Cruz, John Harley A.
IT342 - G1

FULL REGRESSION TEST REPORT

Project Information

RateFlow is a service-based rating and feedback management platform designed to allow users to submit ratings and reviews for various services while enabling administrators to manage, monitor, and analyze user feedback efficiently. The system includes features such as user authentication, service management, rating submission, notification handling, and administrative access control across web, backend, and mobile platforms.

Refactoring Summary

The system was refactored from a traditional layered architecture into a Vertical Slice Architecture to improve maintainability, scalability, modularity, and feature organization across the backend, web frontend, and mobile application.

Previously, the project was organized using technical layers such as:

controller/

service/

repository/

entity/

This structure was refactored into feature-based modules where related components such as controllers, services, repositories, entities, activities, APIs, and frontend pages are grouped together according to their respective system features.

The following feature modules were implemented during the refactoring process:

- Authentication
- Ratings
- Services
- Notifications
- Dashboard
- Profile
- Admin Management

For the backend system, all related controllers, services, repositories, and entities were reorganized into dedicated feature folders under the `features` package.

For the web frontend, React components and pages were reorganized into feature-based folders such as `auth`, `ratings`, `services`, `notifications`, and `admin` to improve modularity and simplify maintenance.

For the Android mobile application, Kotlin classes including activities, adapters, APIs, and models were grouped into feature-based packages following the same architectural pattern.

The refactoring process also included:

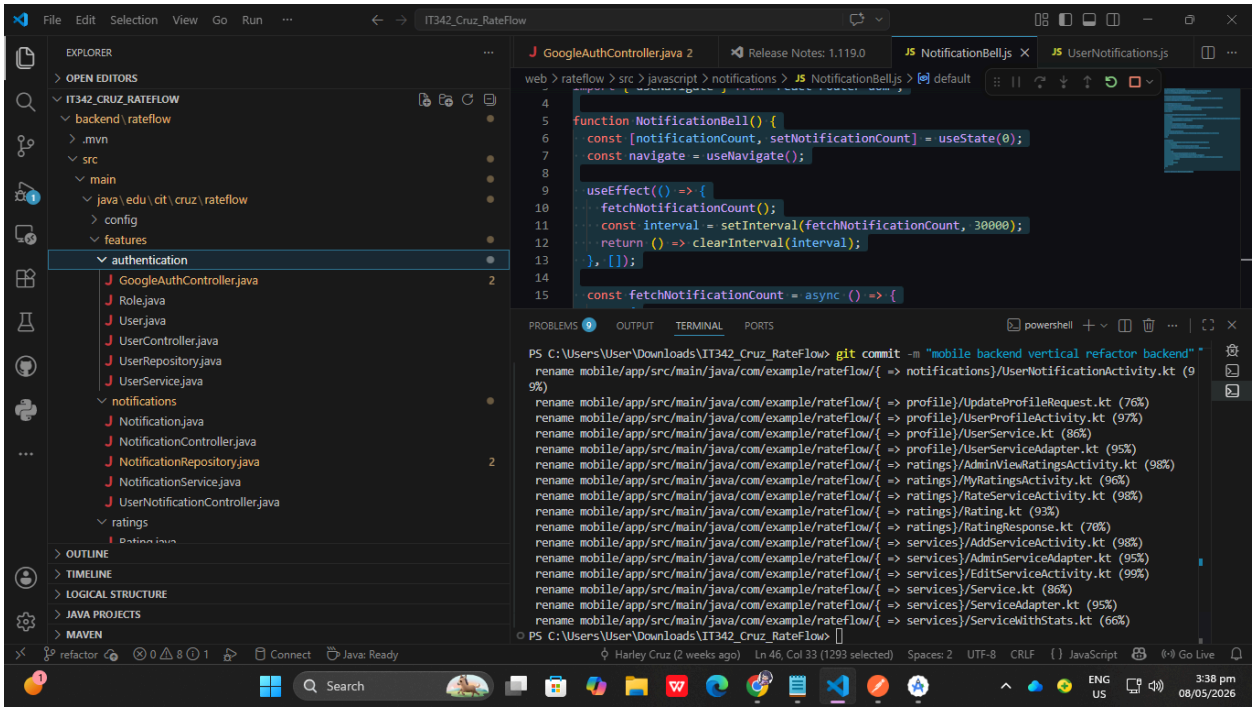
- Updating package declarations
- Fixing import dependencies
- Updating routing and module references
- Resolving component scanning issues
- Updating API references and service integrations

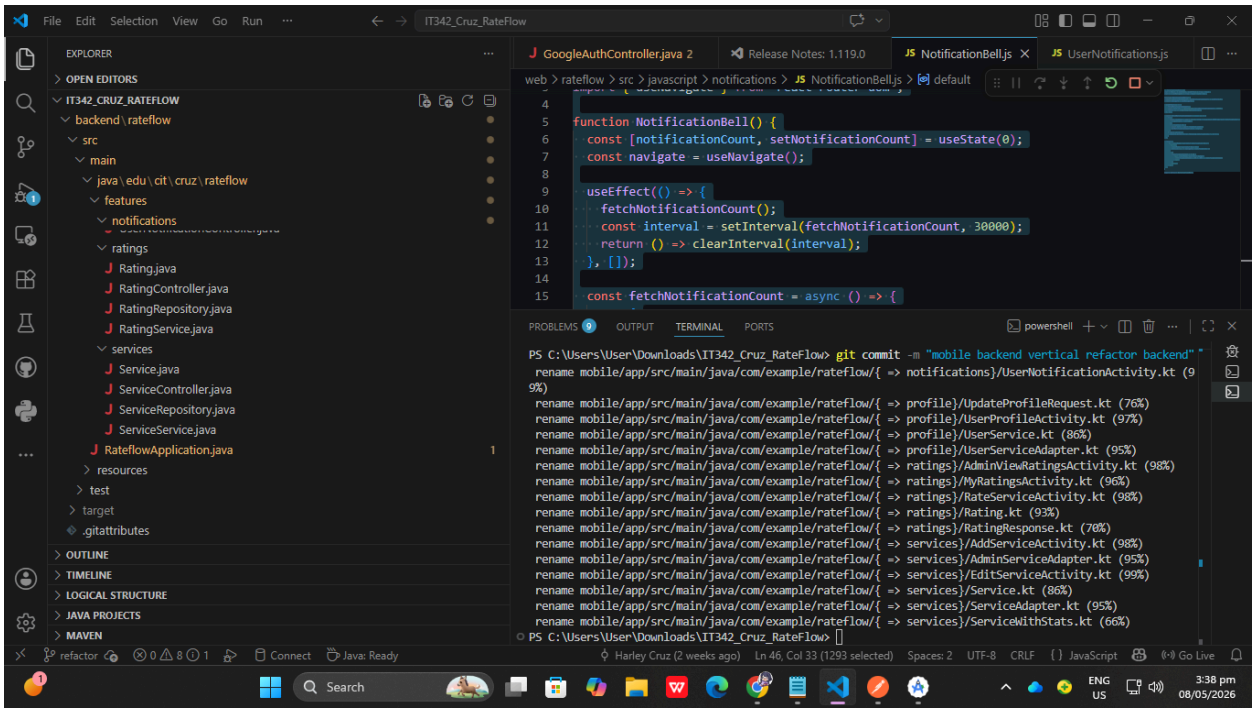
No major functional changes were introduced during the refactoring process. The primary objective of the refactor was to improve the internal structure and organization of the system while preserving all existing functionalities.

After the refactoring process, full regression testing was conducted to ensure that all implemented functional requirements continued to operate correctly without introducing regressions or system failures.

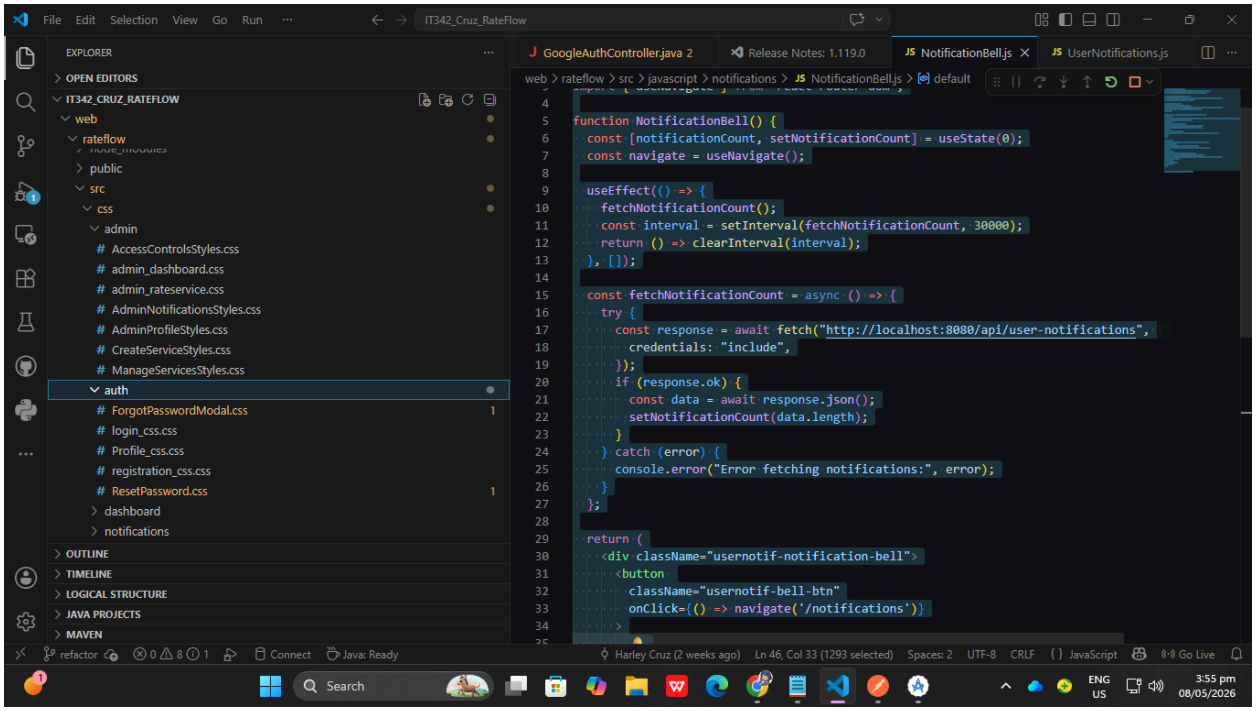
Updated Project Structure

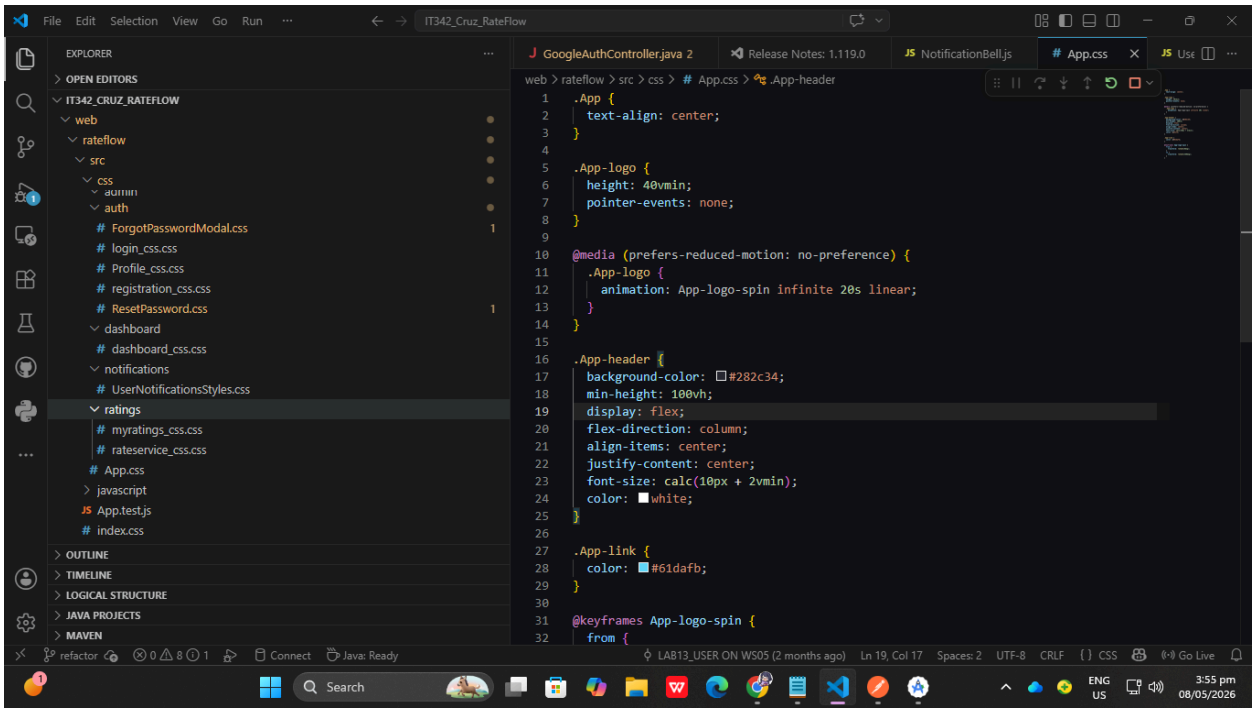
BACKEND:


File Edit Selection View Go Run
IT342_Cruz_RateFlow
EXPLORER
OPEN EDITORS
IT342_CRUZ_RATEFLOW
backend \ rateflow
.mvn
src
main
java \ edu \ cit \ cruz \ rateflow
config
features
authentication
GoogleAuthController.java
Role.java
User.java
UserController.java
UserRepository.java
UserService.java
notifications
Notification.java
NotificationController.java
NotificationRepository.java
NotificationService.java
UserNotificationController.java
ratings
OUTLINE
TIMELINE
LOGICAL STRUCTURE
JAVA PROJECTS
MAVEN
GoogleAuthController.java 2
Release Notes: 1.119.0
NotificationBell.js
UserNotifications.js
web > rateflow > src > javascript > notifications > NotificationBell.js > default
function NotificationBell() {
const [notificationCount, setNotificationCount] = useState(0);
const navigate = useNavigate();
useEffect(() => {
fetchNotificationCount();
const interval = setInterval(fetchNotificationCount, 30000);
return () => clearInterval(interval);
}, []);
const fetchNotificationCount = async () => {
PROBLEMS 9
OUTPUT
TERMINAL
PORTS
powershell
PS C:\Users\User\Downloads\IT342_Cruz_RateFlow> git commit -m "mobile backend vertical refactor backend"
rename mobile/app/src/main/java/com/example/rateflow/{ => notifications}/UserNotificationActivity.kt (9
9%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UpdateProfileRequest.kt (76%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UserProfileActivity.kt (97%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UserService.kt (86%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UserServiceAdapter.kt (95%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/AdminViewRatingsActivity.kt (98%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/MyRatingsActivity.kt (96%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/RateServiceActivity.kt (98%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/Rating.kt (93%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/RatingResponse.kt (70%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/AddServiceActivity.kt (98%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/AdminServiceAdapter.kt (95%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/EditServiceActivity.kt (99%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/Service.kt (86%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/ServiceAdapter.kt (95%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/ServiceWithStats.kt (66%)
PS C:\Users\User\Downloads\IT342_Cruz_RateFlow>
refactor
Connect
Java: Ready
Harley Cruz (2 weeks ago)
Ln 46, Col 33 (1293 selected)
Spaces: 2
UTF-8
CRLF
JavaScript
Go Live
Search
ENG
US
3:38 pm
08/05/2026

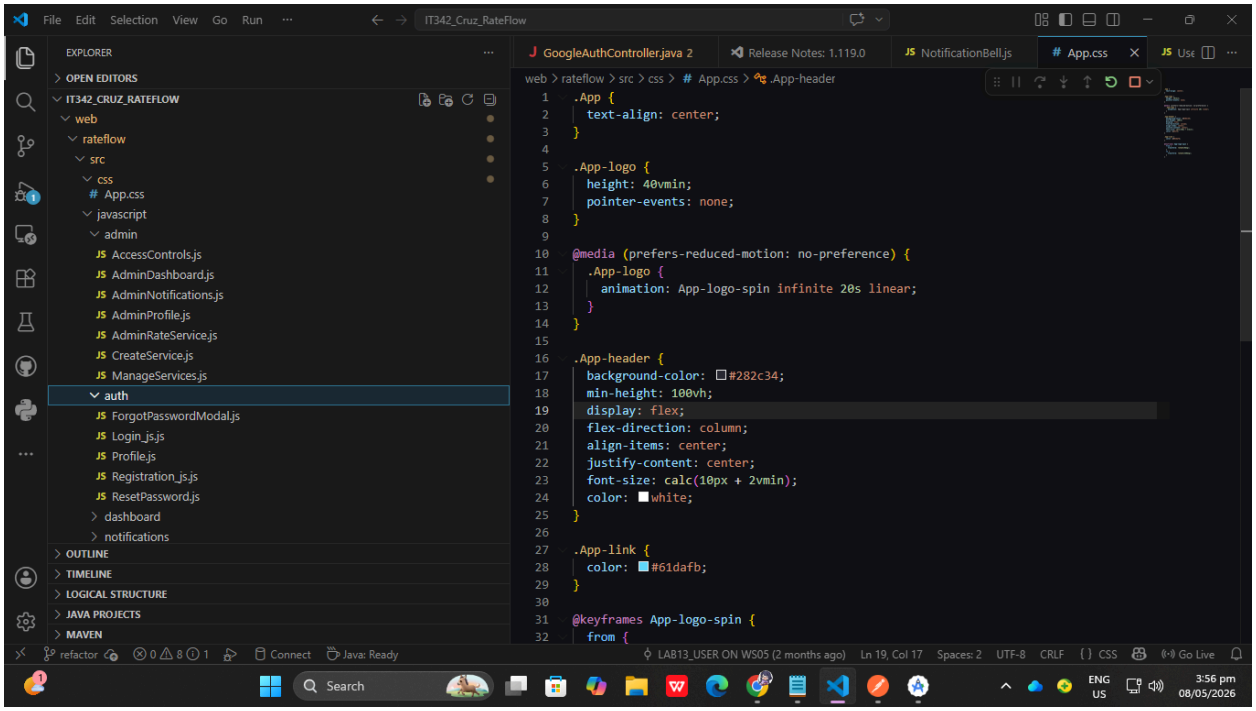

File Edit Selection View Go Run
IT342_Cruz_RateFlow
EXPLORER
OPEN EDITORS
IT342_CRUZ_RATEFLOW
backend \ rateflow
src
main
java \ edu \ cit \ cruz \ rateflow
features
notifications
ratings
Rating.java
RatingController.java
RatingRepository.java
RatingService.java
services
Service.java
ServiceController.java
ServiceRepository.java
ServiceService.java
RateflowApplication.java
resources
test
target
.gitattributes
OUTLINE
TIMELINE
LOGICAL STRUCTURE
JAVA PROJECTS
MAVEN
GoogleAuthController.java 2
Release Notes: 1.119.0
NotificationBell.js
UserNotifications.js
web > rateflow > src > javascript > notifications > NotificationBell.js > default
function NotificationBell() {
const [notificationCount, setNotificationCount] = useState(0);
const navigate = useNavigate();
useEffect(() => {
fetchNotificationCount();
const interval = setInterval(fetchNotificationCount, 30000);
return () => clearInterval(interval);
}, []);
const fetchNotificationCount = async () => {
PROBLEMS 9
OUTPUT
TERMINAL
PORTS
powershell
PS C:\Users\User\Downloads\IT342_Cruz_RateFlow> git commit -m "mobile backend vertical refactor backend"
rename mobile/app/src/main/java/com/example/rateflow/{ => notifications}/UserNotificationActivity.kt (9
9%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UpdateProfileRequest.kt (76%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UserProfileActivity.kt (97%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UserService.kt (86%)
rename mobile/app/src/main/java/com/example/rateflow/{ => profile}/UserServiceAdapter.kt (95%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/AdminViewRatingsActivity.kt (98%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/MyRatingsActivity.kt (96%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/RateServiceActivity.kt (98%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/Rating.kt (93%)
rename mobile/app/src/main/java/com/example/rateflow/{ => ratings}/RatingResponse.kt (70%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/AddServiceActivity.kt (98%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/AdminServiceAdapter.kt (95%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/EditServiceActivity.kt (99%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/Service.kt (86%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/ServiceAdapter.kt (95%)
rename mobile/app/src/main/java/com/example/rateflow/{ => services}/ServiceWithStats.kt (66%)
PS C:\Users\User\Downloads\IT342_Cruz_RateFlow>
refactor
Connect
Java: Ready
Harley Cruz (2 weeks ago)
Ln 46, Col 33 (1293 selected)
Spaces: 2
UTF-8
CRLF
JavaScript
Go Live
Search
ENG
US
3:38 pm
08/05/2026

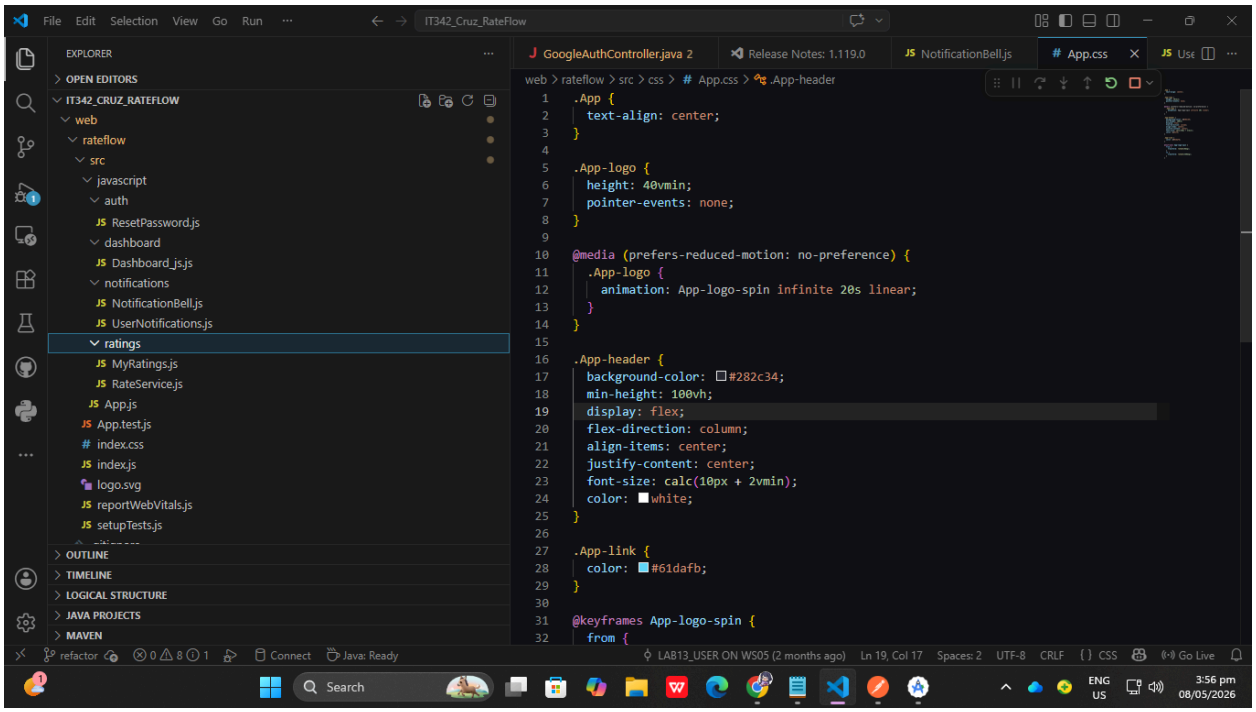
FRONTEND (WEB CCS):



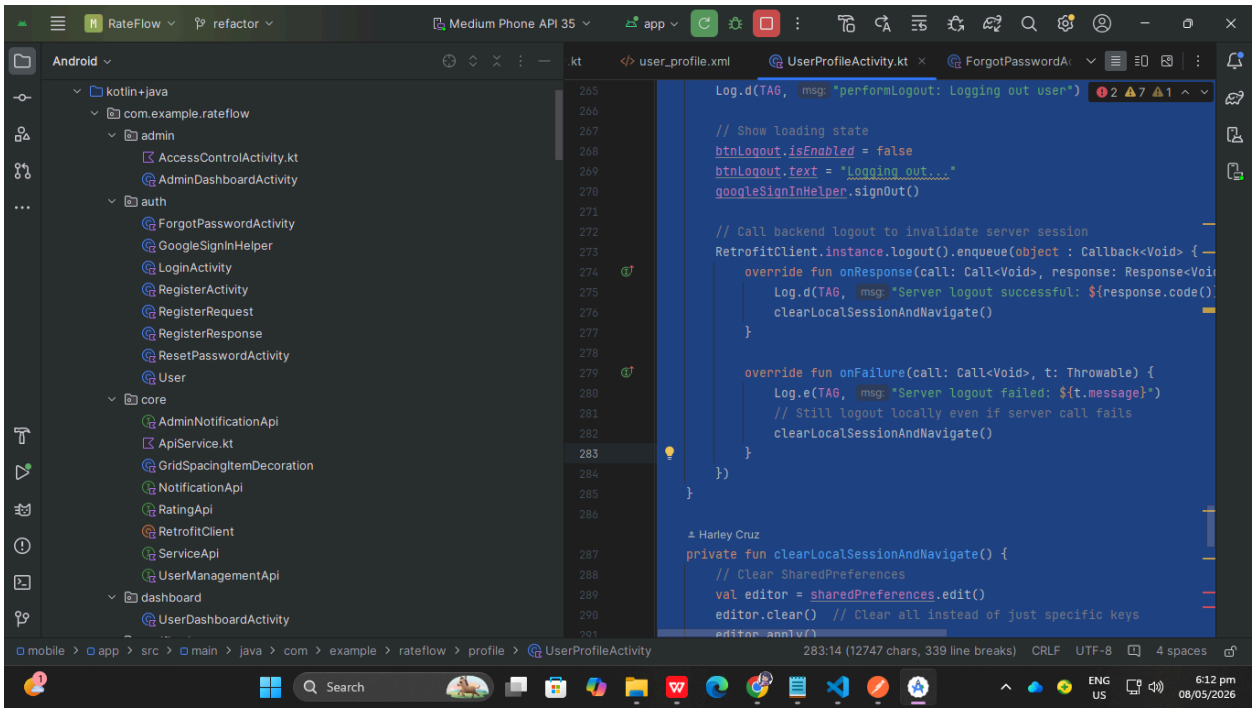


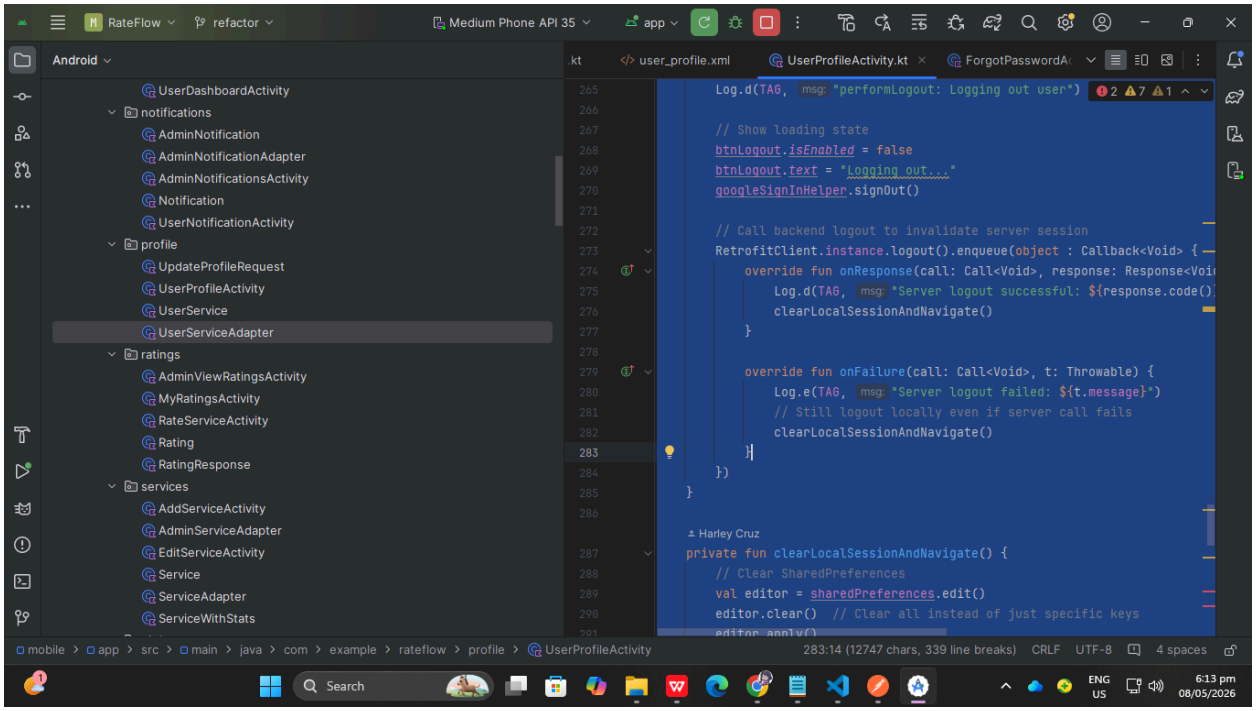
WEB (JAVASCRIPT):





MOBILE (ACTIVITIES):

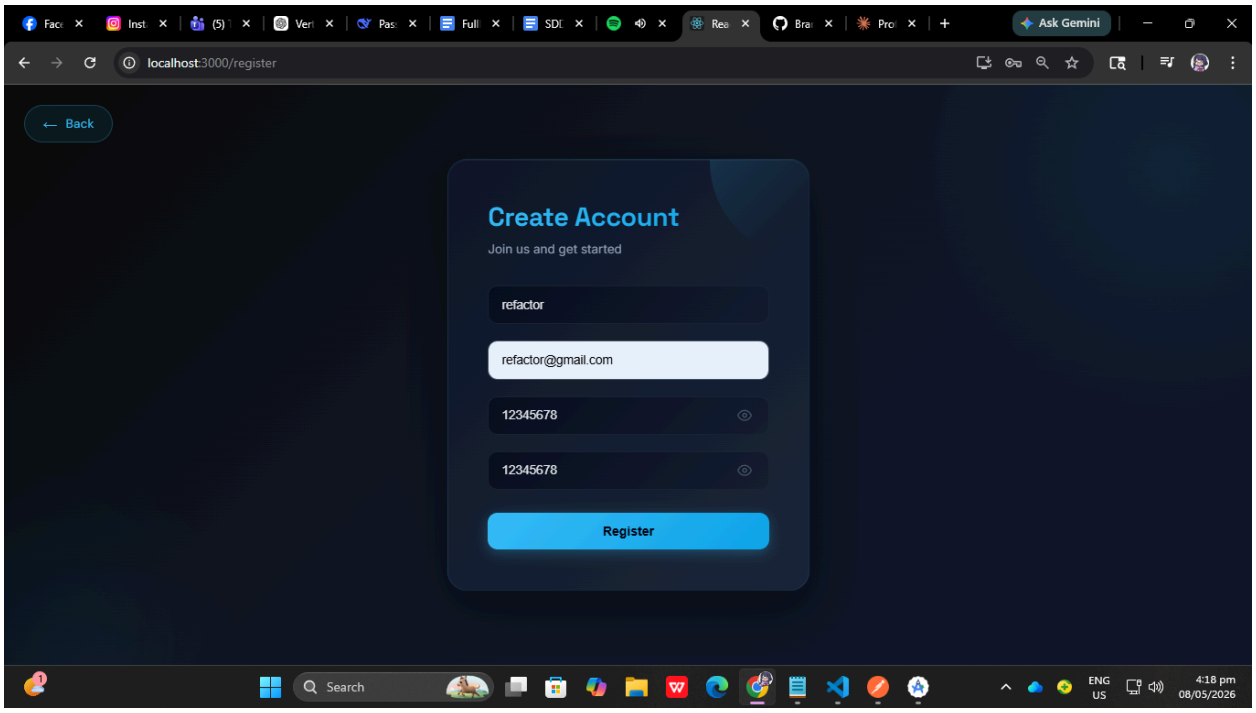


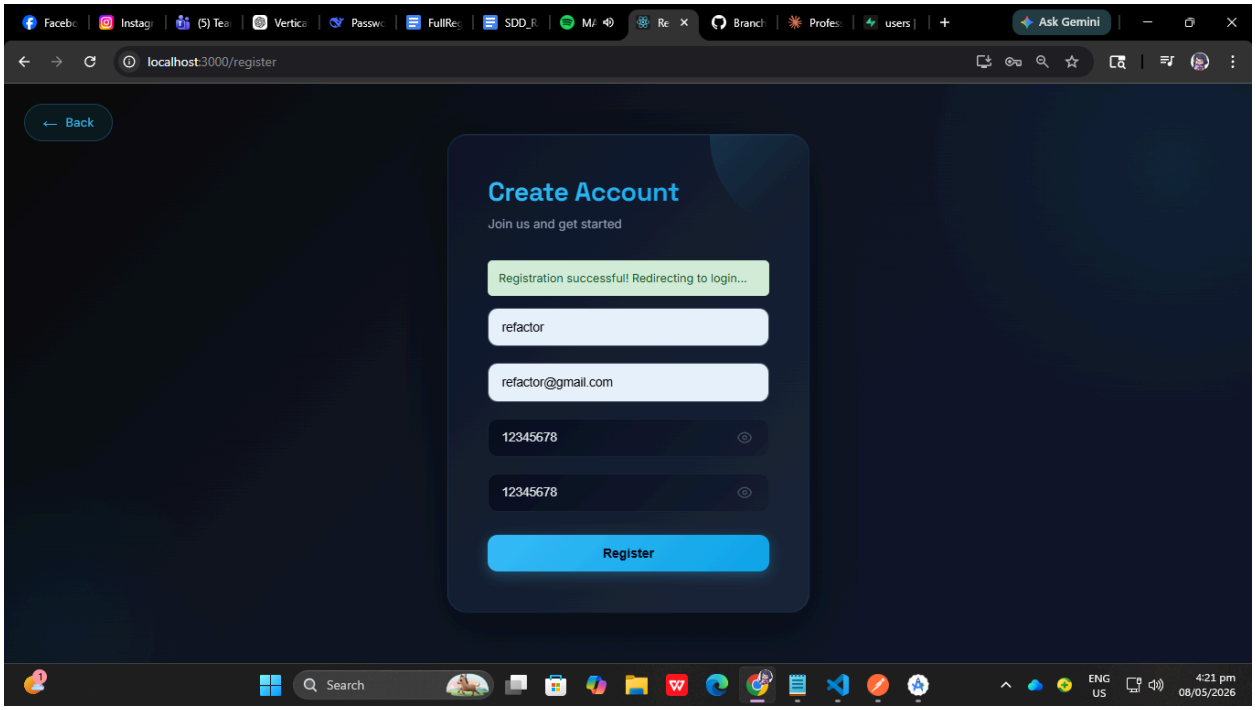


Test Plan Documentation

TS-AUTH-001 — User Registration

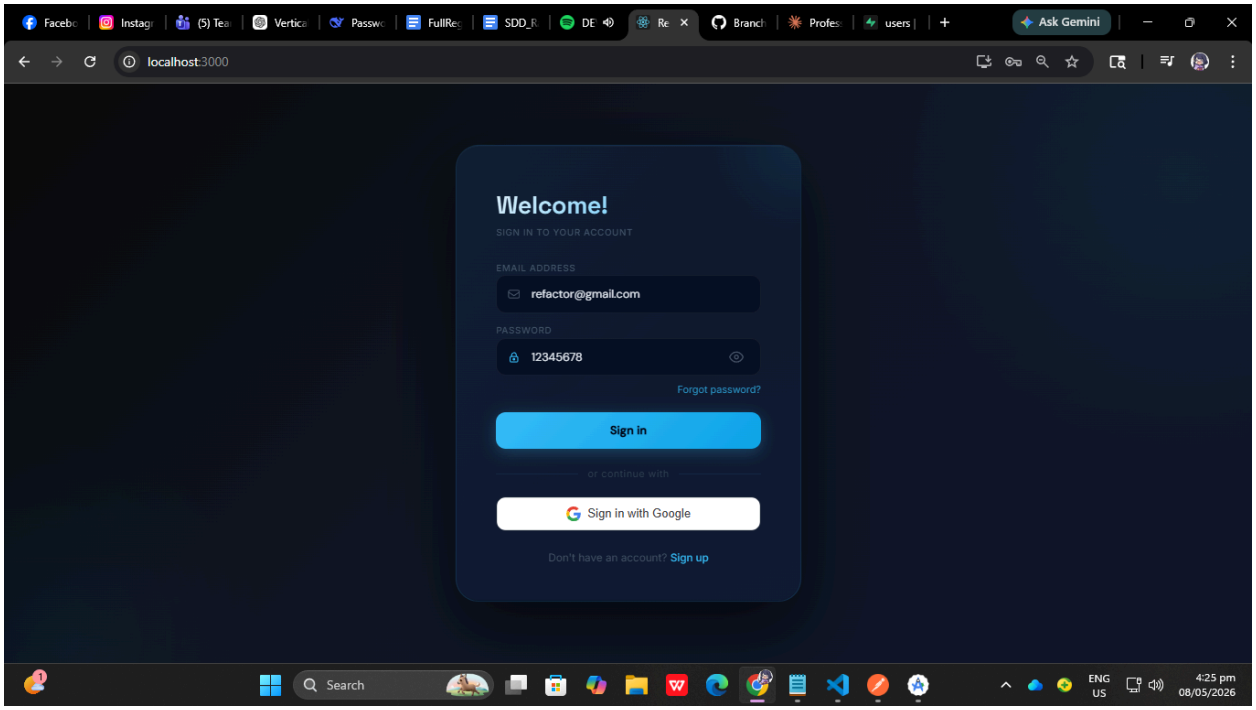
Field	Details
Module	Authentication
Objective	Verify that a new user can register successfully
Preconditions	User is not yet registered
Test Steps	1. Open the Registration Page 2. Enter valid username 3. Enter valid email 4. Enter password 5. Confirm password correctly 6. Click “Register” button
Expected Result	User account is created successfully and redirected to login page

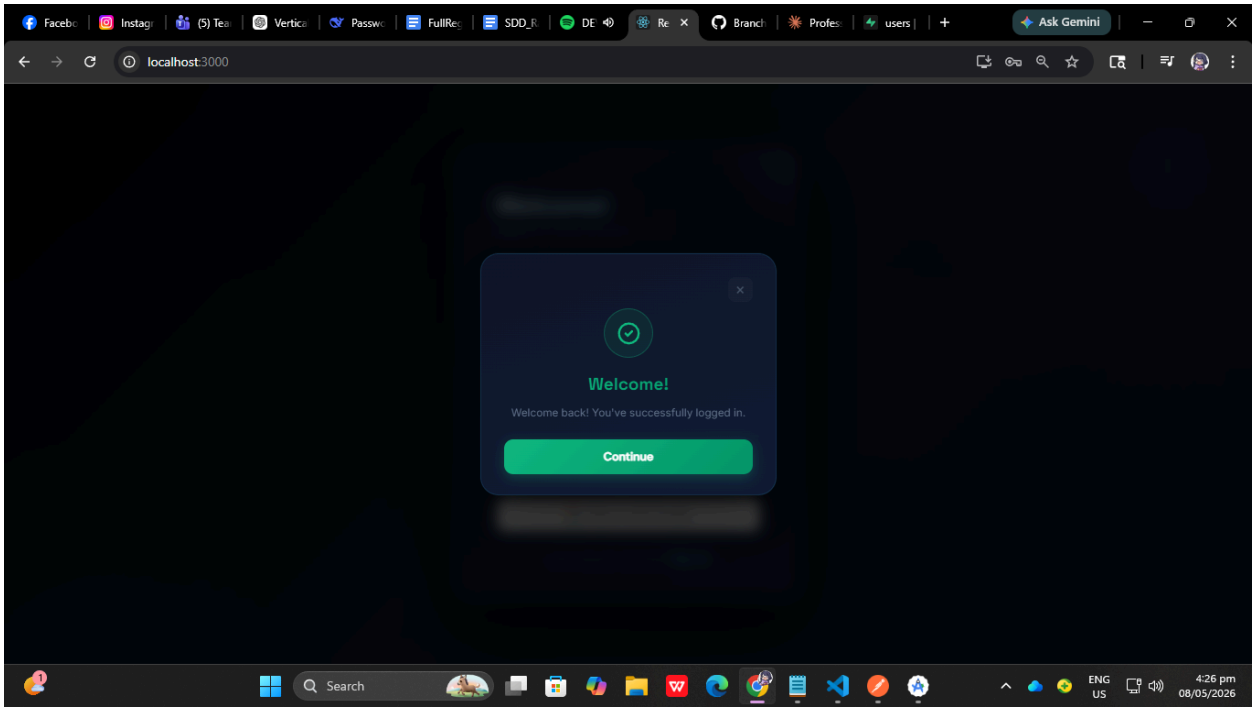




TS-AUTH-002 — User Login

Field	Details
Module	Authentication
Objective	Verify user login functionality
Preconditions	Registered user account exists
Test Steps	1. Open Login Page 2. Enter registered email 3. Enter correct password 4. Click “Login”
Expected Result	User successfully logs in and dashboard page loads

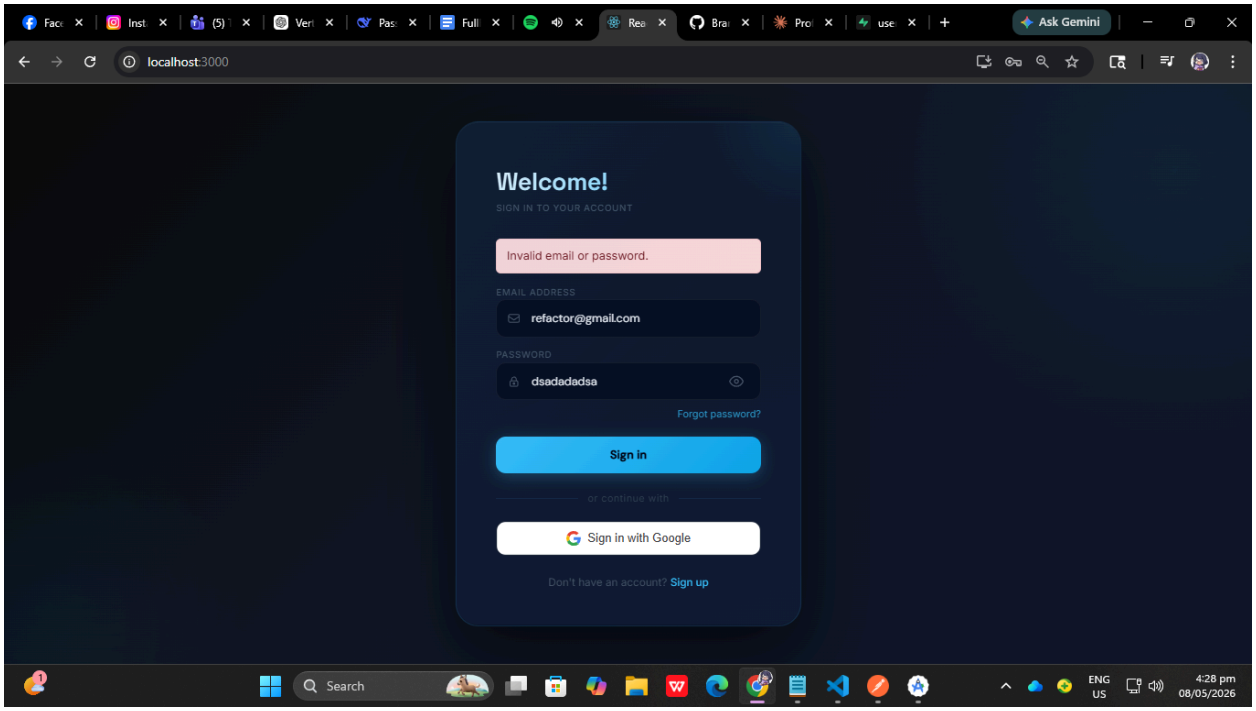




TS-AUTH-003 — Invalid Login Credentials

Field	Details

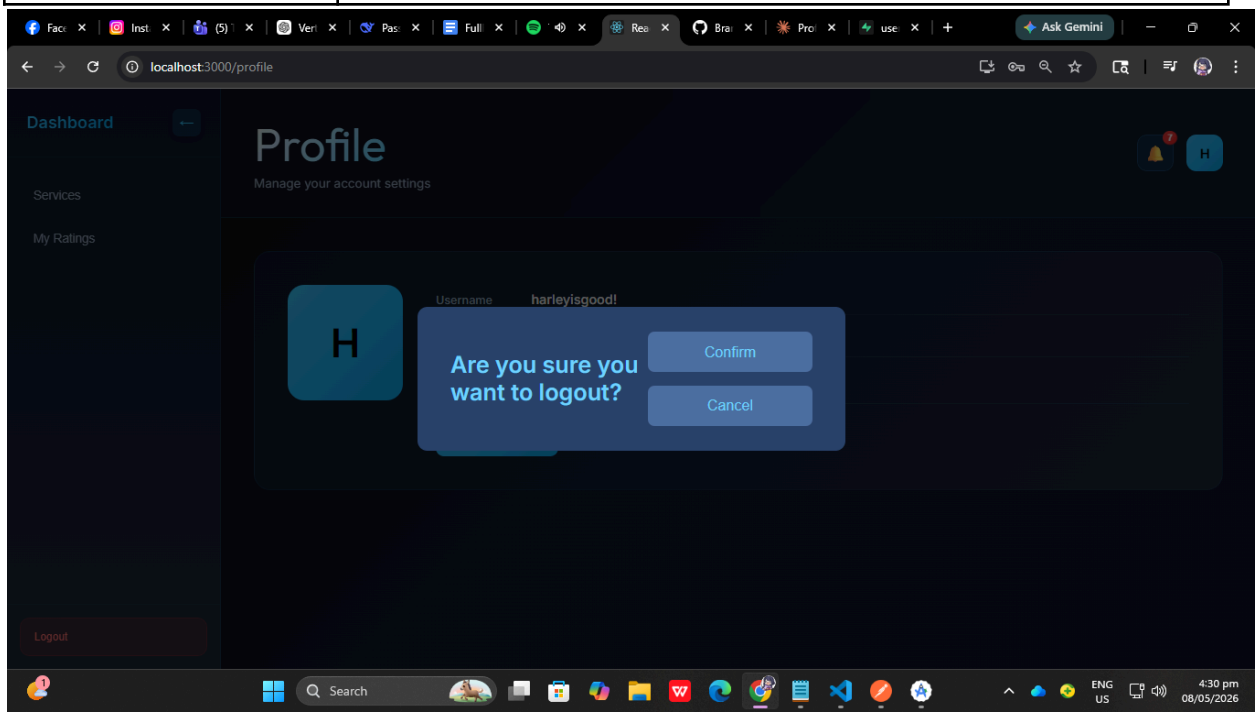
Module	Authentication
Objective	Verify system rejects invalid login
Preconditions	Registered user account exists
Test Steps	1. Open Login Page 2. Enter invalid password 3. Click “Login”
Expected Result	Error message “Invalid credentials” is displayed

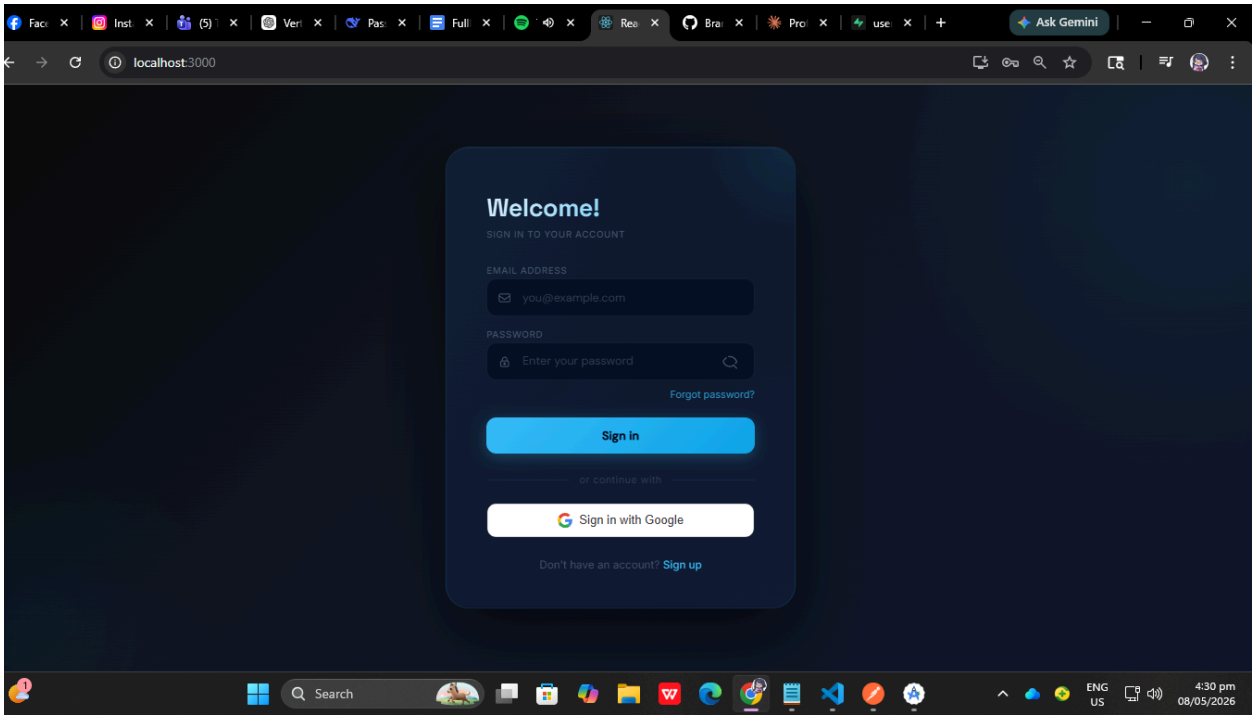


TS-AUTH-004 — User Logout

Field	Details
Module	Authentication
Objective	Verify logout functionality
Preconditions	User is logged in
Test Steps	1. Click Logout button

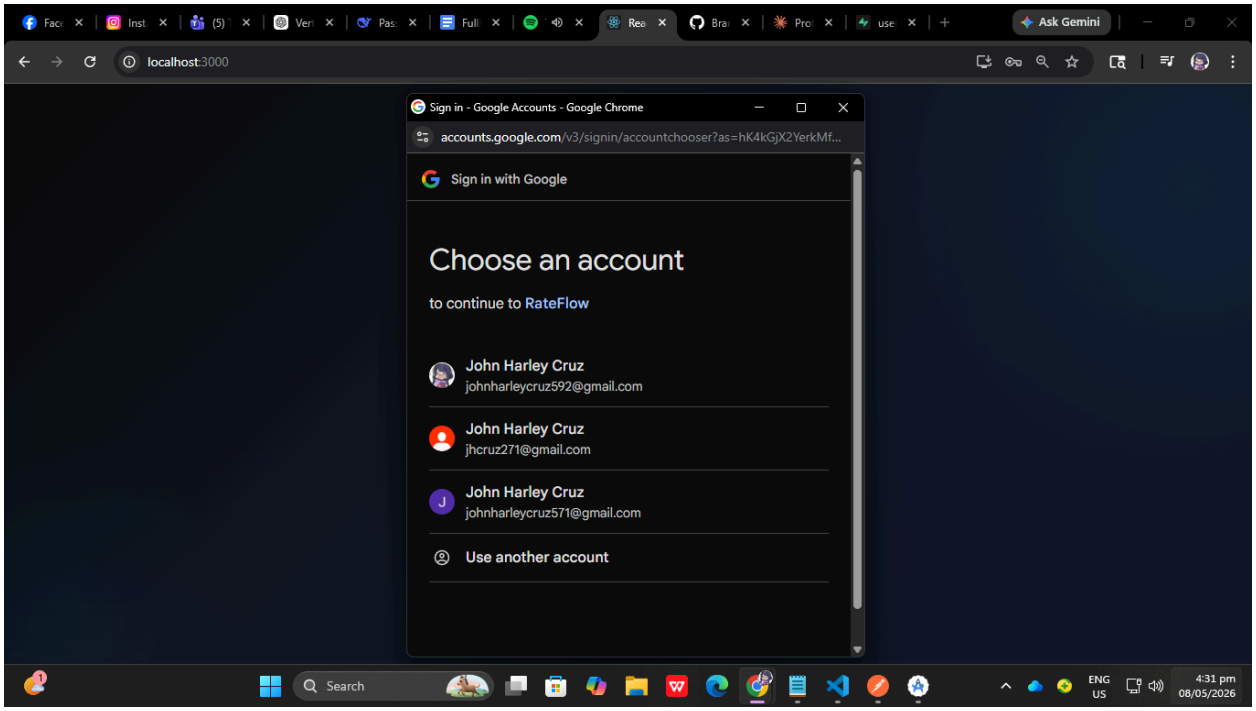
	2. Confirm logout
Expected Result	User session is terminated and redirected to login page

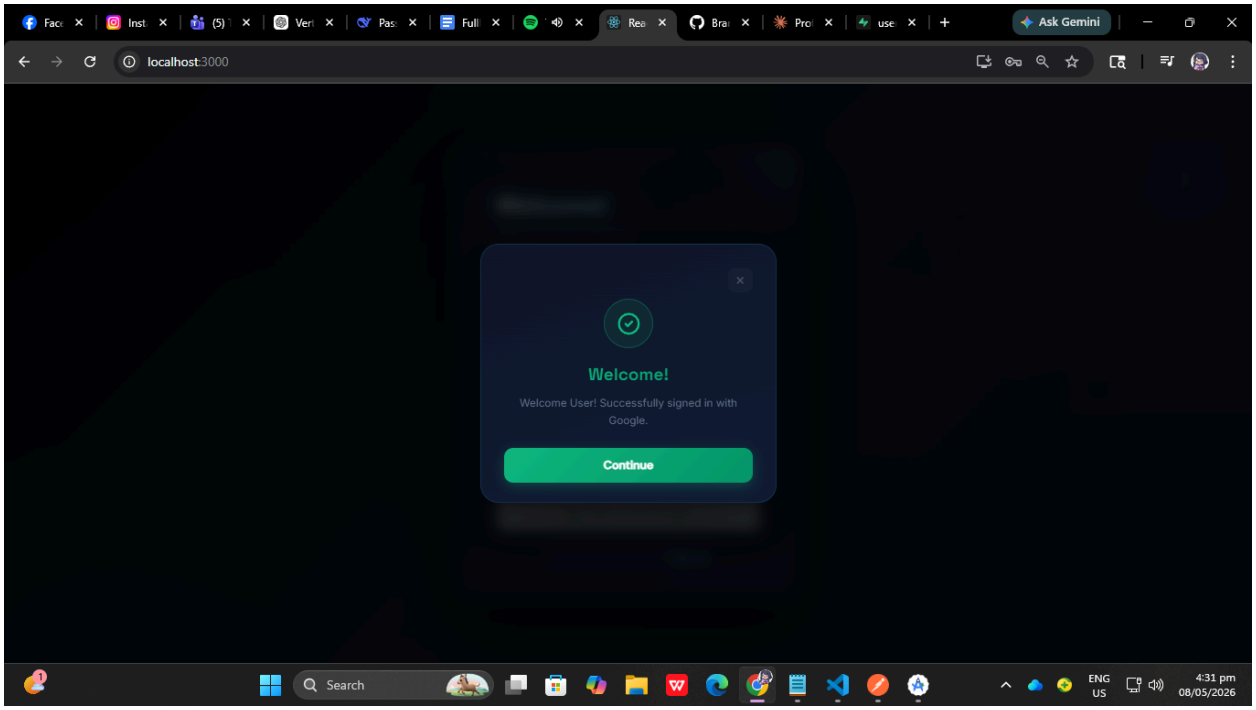




TS-AUTH-005 — Google Sign-In

Field	Details
Module	Authentication
Objective	Verify Google authentication login
Preconditions	Google account already registered in system
Test Steps	1. Open Login Page 2. Click “Sign in with Google” 3. Select registered Google account
Expected Result	User successfully authenticated and redirected to dashboard

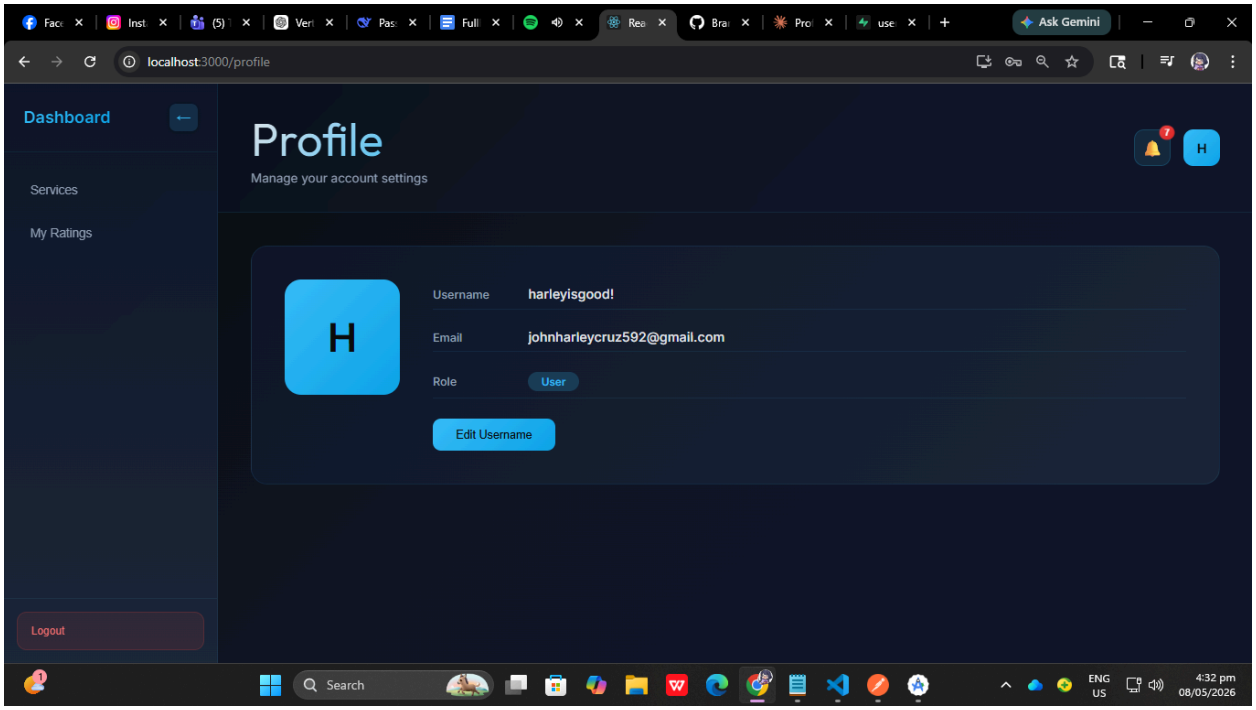




USER PROFILE MODULE

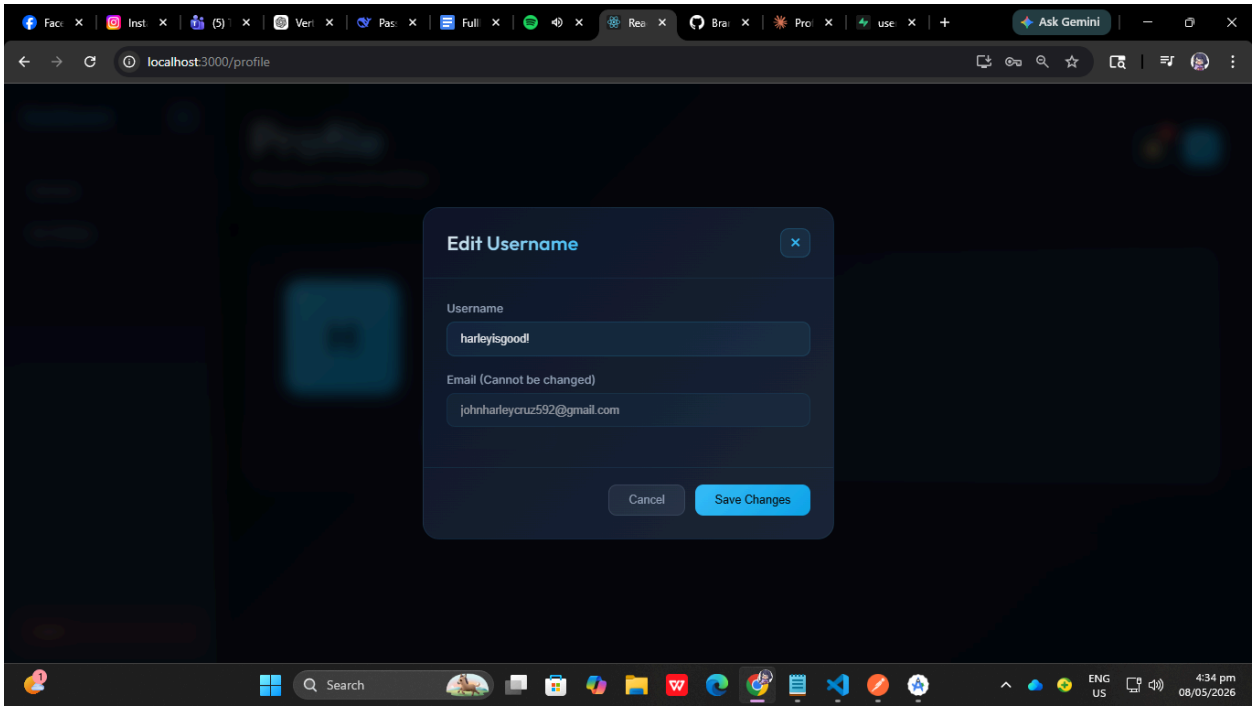
TS-PROFILE-001 — View User Profile

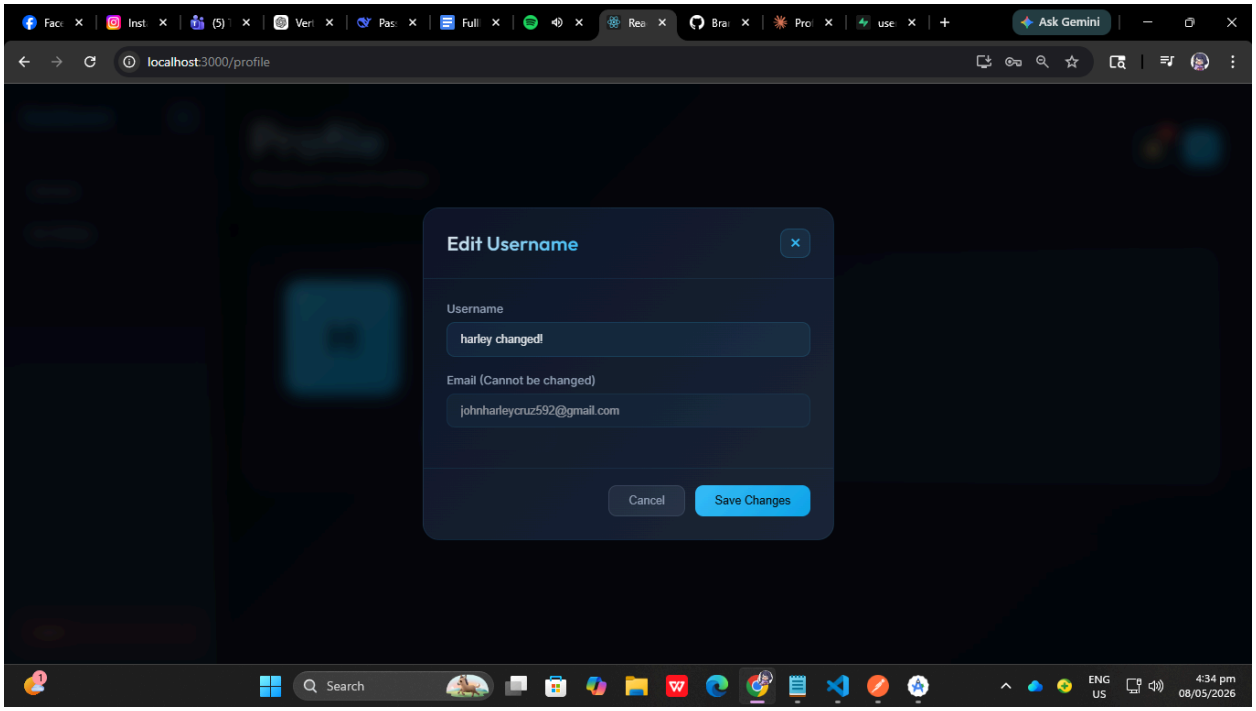
Field	Details
Module	User Profile
Objective	Verify profile information is displayed
Preconditions	User is logged in
Test Steps	1. Open Profile Page
Expected Result	Username, email, and role are displayed correctly

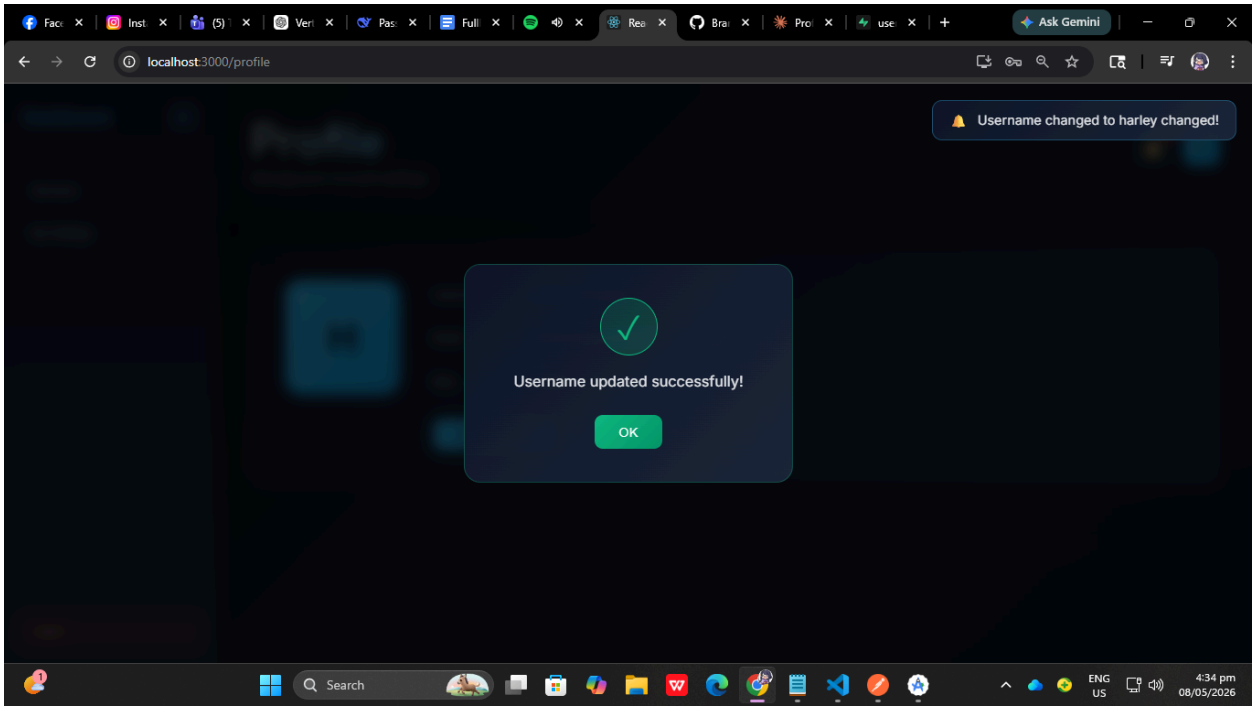


TS-PROFILE-002 — Edit User Profile

Field	Details
Module	User Profile
Objective	Verify user can edit profile
Preconditions	User is logged in
Test Steps	1. Open Profile Page 2. Click “Edit Username” 3. Update profile username 4. Save changes
Expected Result	Updated profile information is saved successfully

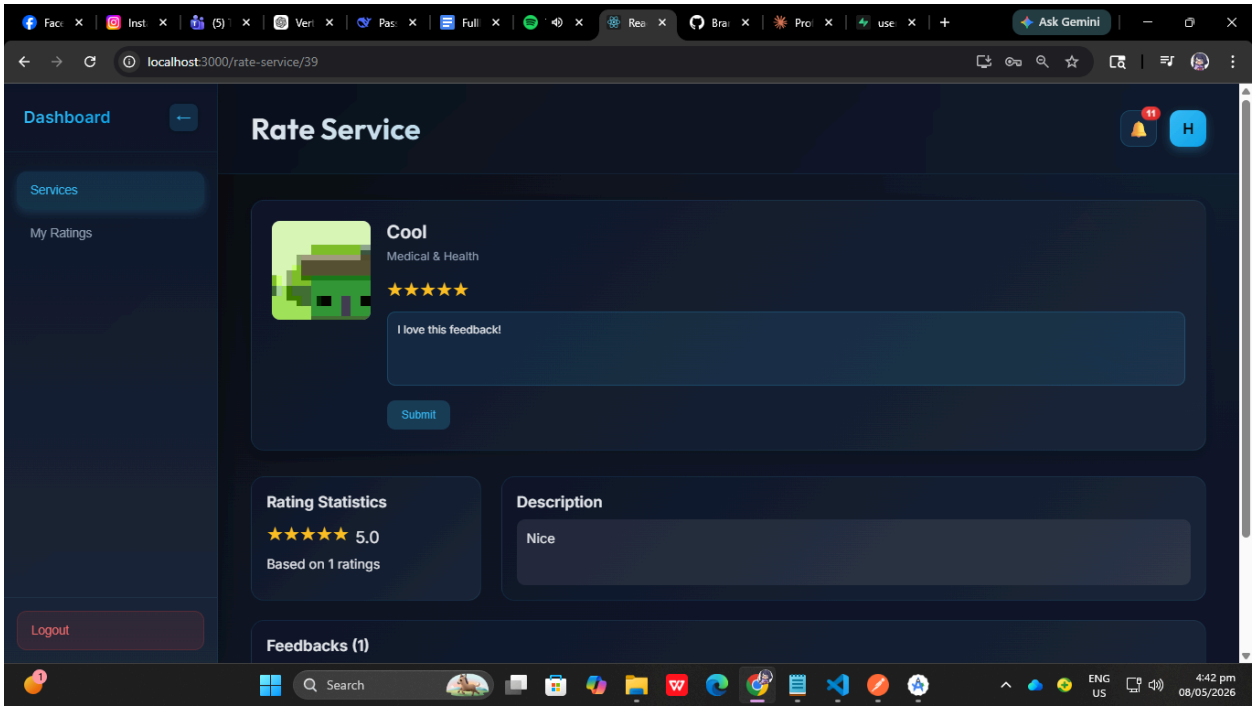

Ask Gemini
localhost:3000/profile
Edit Username
Username
harleyisgood!
Email (Cannot be changed)
johnharleycruz592@gmail.com
Cancel
Save Changes
Search
ENG
US
4:34 pm
08/05/2026

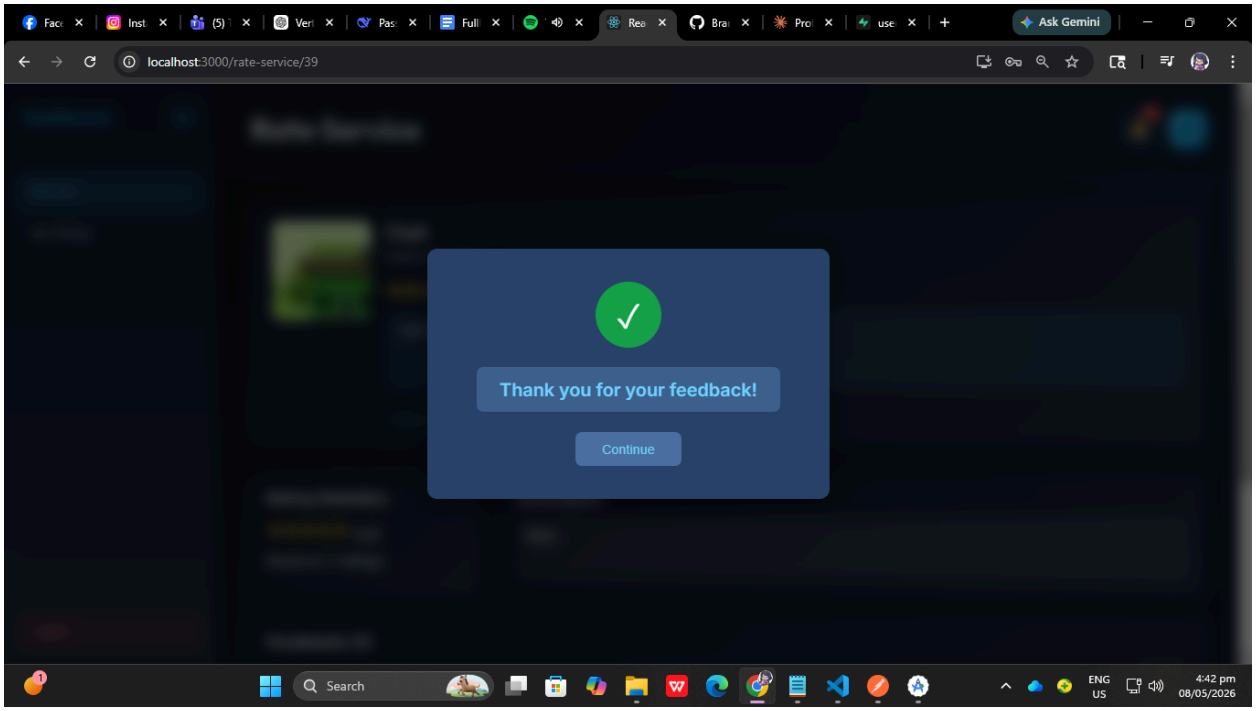

Ask Gemini
localhost:3000/profile
Edit Username
Username
harley changed!
Email (Cannot be changed)
johnharleycruz592@gmail.com
Cancel
Save Changes
Search
ENG
US
4:34 pm
08/05/2026



TS-RATE-001 — Submit Rating

Field	Details
Module	Rating System
Objective	Verify user can submit rating
Preconditions	User logged in and service exists
Test Steps	1. Open Service Page 2. Select star rating 3. Enter feedback text 4. Click “Submit Rating”
Expected Result	Rating is stored successfully and confirmation message appears

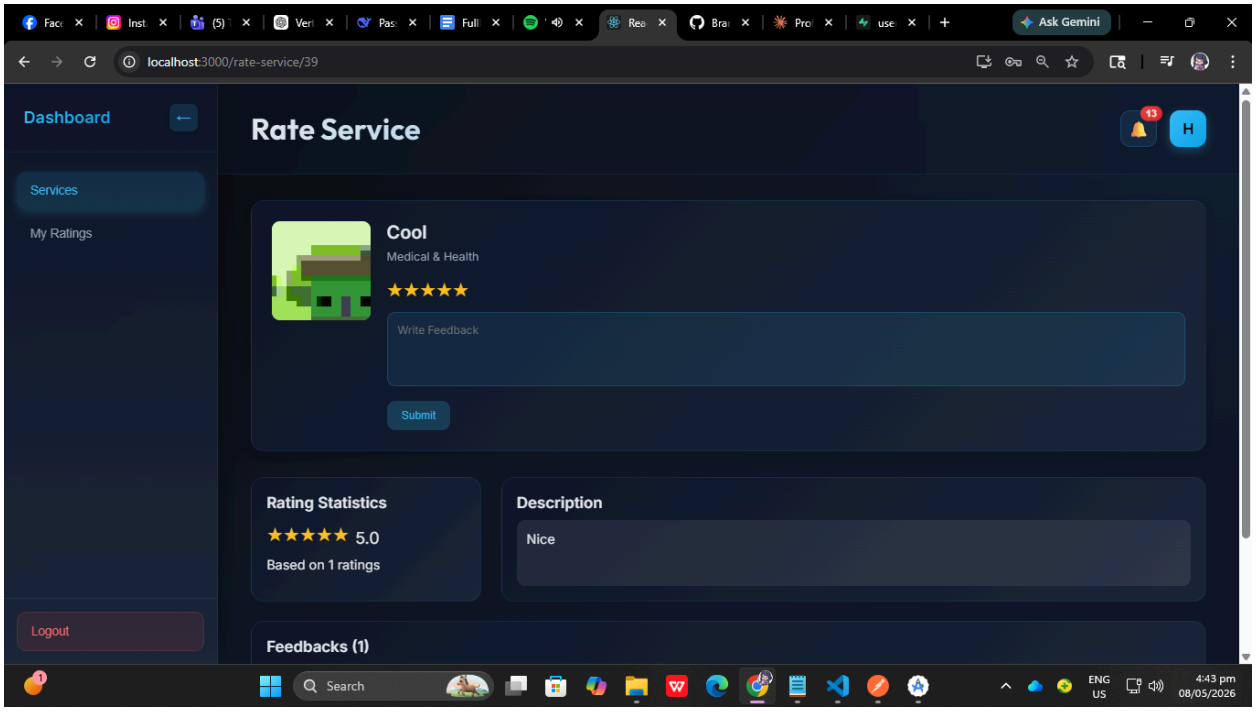


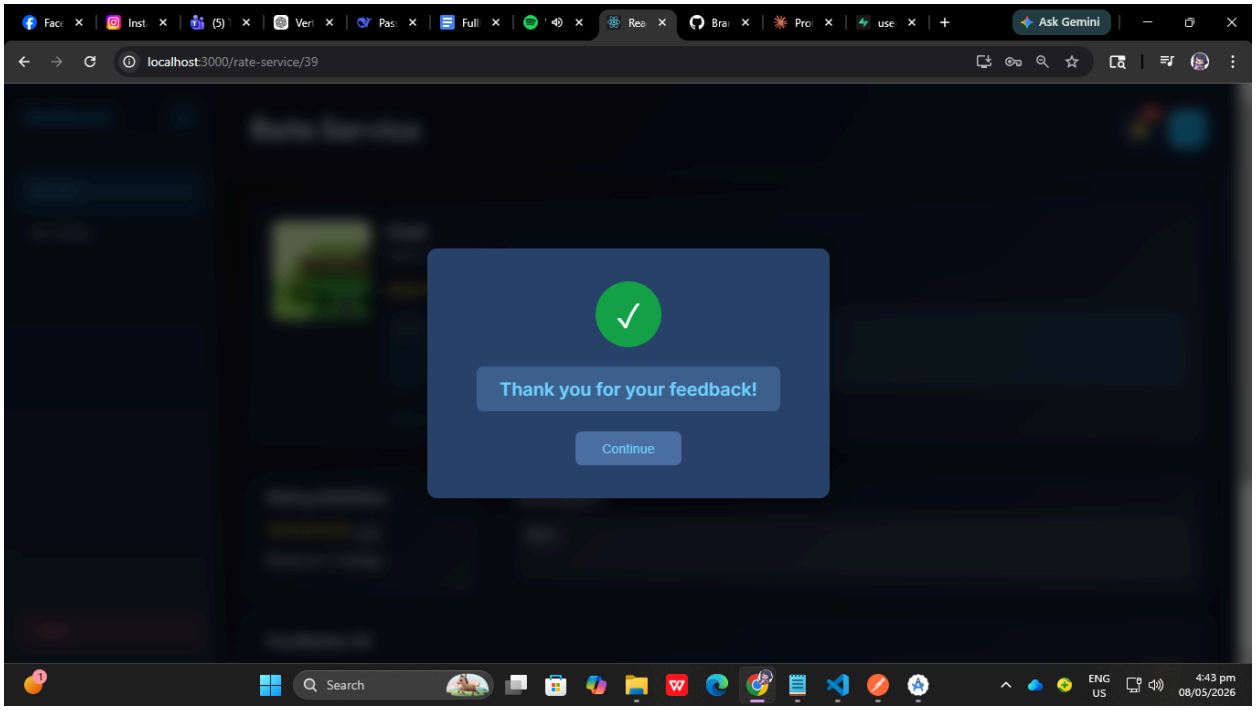


TS-RATE-002 — Submit Rating Without Feedback

Field	Details
Module	Rating System

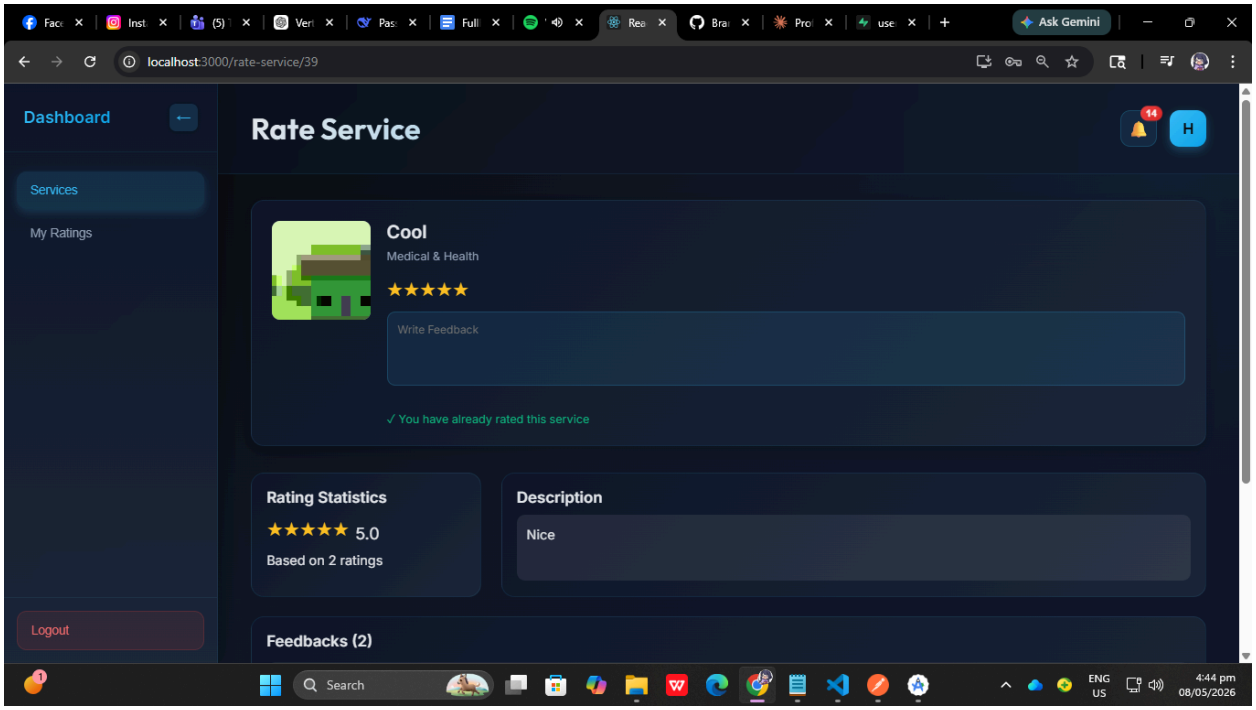
Objective	Verify optional feedback submission
Preconditions	User logged in
Test Steps	1. Open Service Page 2. Select star rating only 3. Click “Submit Rating”
Expected Result	Rating successfully submitted without written feedback





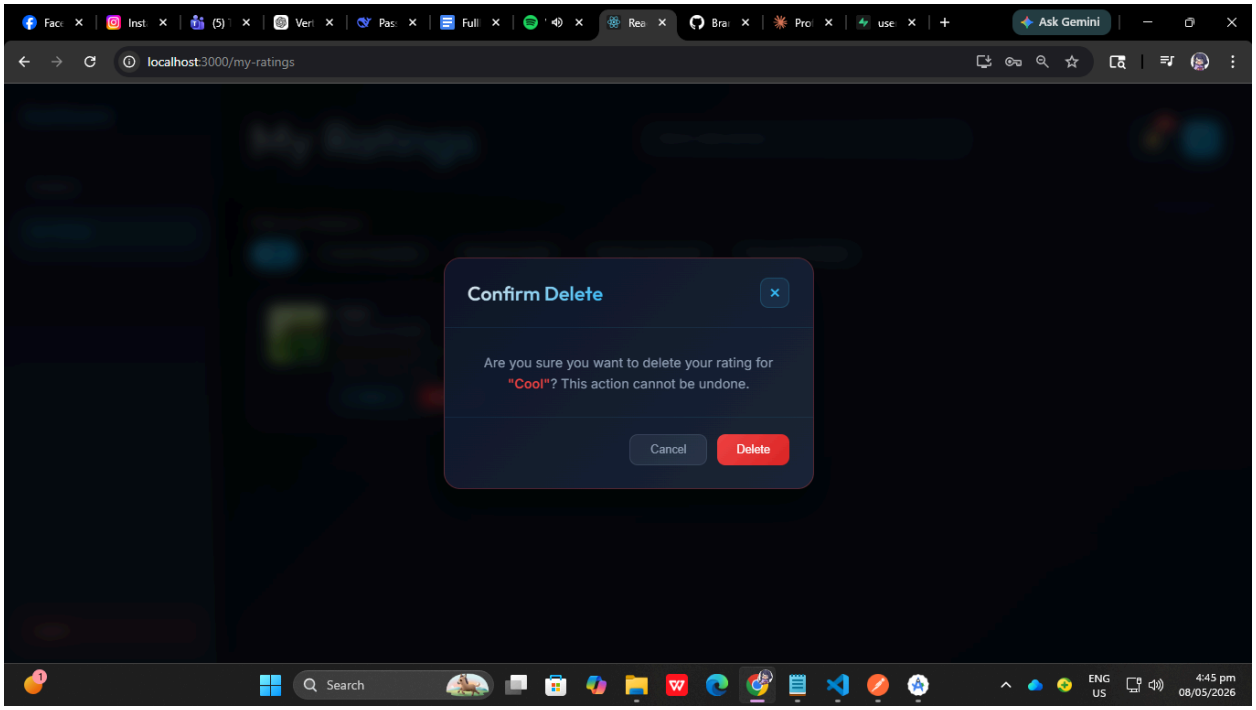
TS-RATE-003 — Prevent Duplicate Rating

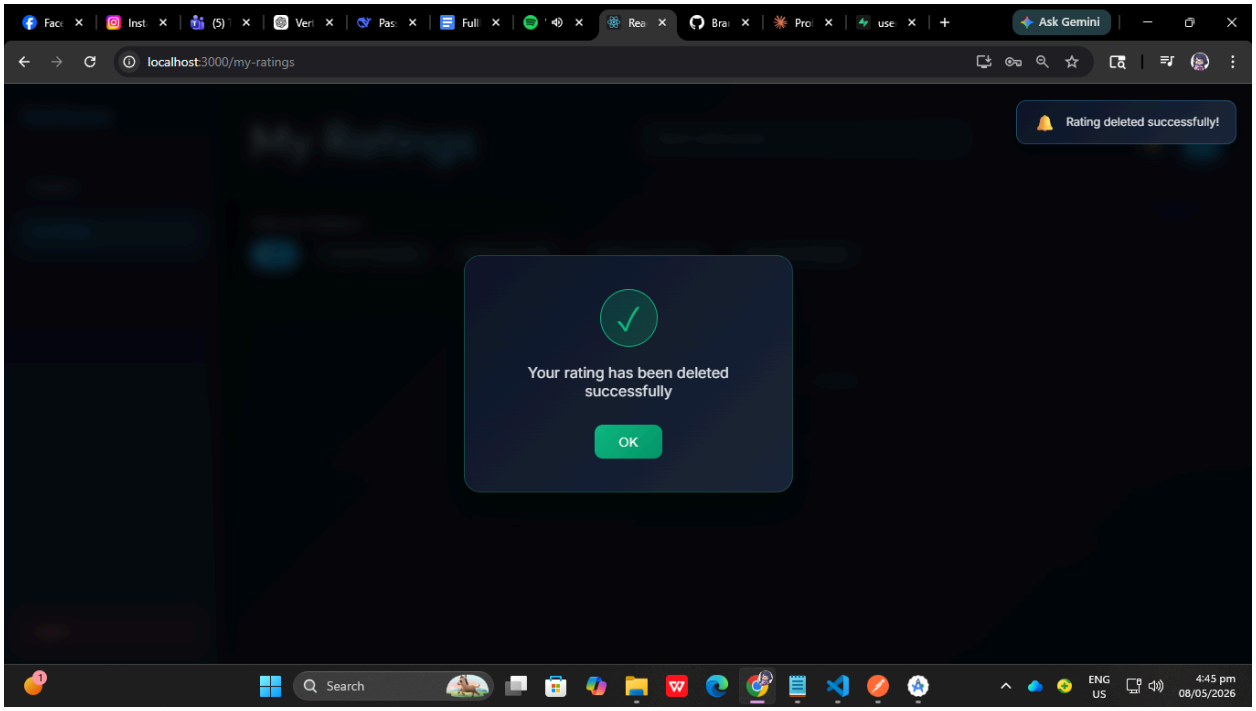
Field	Details
Module	Rating System
Objective	Verify user cannot rate same service twice
Preconditions	User already rated service
Test Steps	1. Open already-rated service 2. Attempt second rating submission
Expected Result	System prevents duplicate rating submission

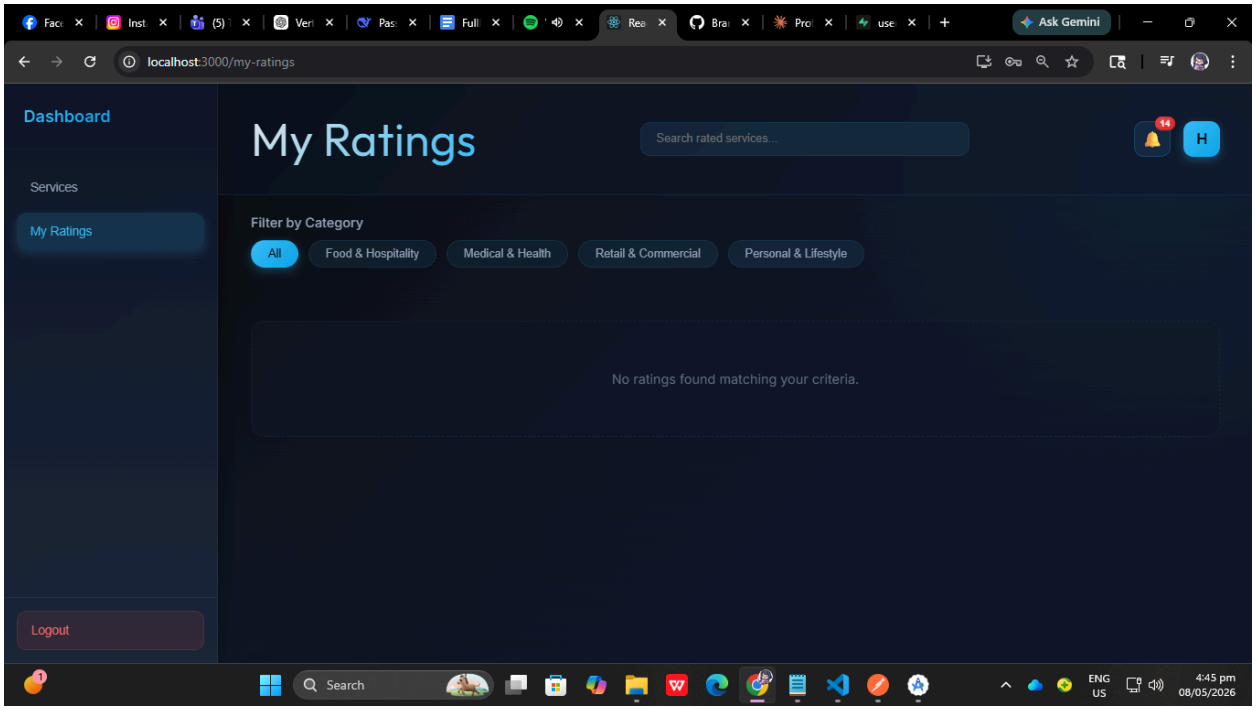


TS-RATE-004 — Delete Own Rating

Field	Details
Module	Rating System
Objective	Verify user can delete own rating
Preconditions	User has existing rating
Test Steps	1. Open My Ratings Page 2. Select rating 3. Click Delete button 4. Confirm deletion
Expected Result	Rating deleted successfully


Ask Gemini
localhost:3000/my-ratings
Confirm Delete
Are you sure you want to delete your rating for "Cool"? This action cannot be undone.
Cancel
Delete
Search
ENG US
4:45 pm 08/05/2026


Ask Gemini
localhost:3000/my-ratings
Rating deleted successfully!
Your rating has been deleted successfully
OK
Search
ENG US
4:45 pm 08/05/2026

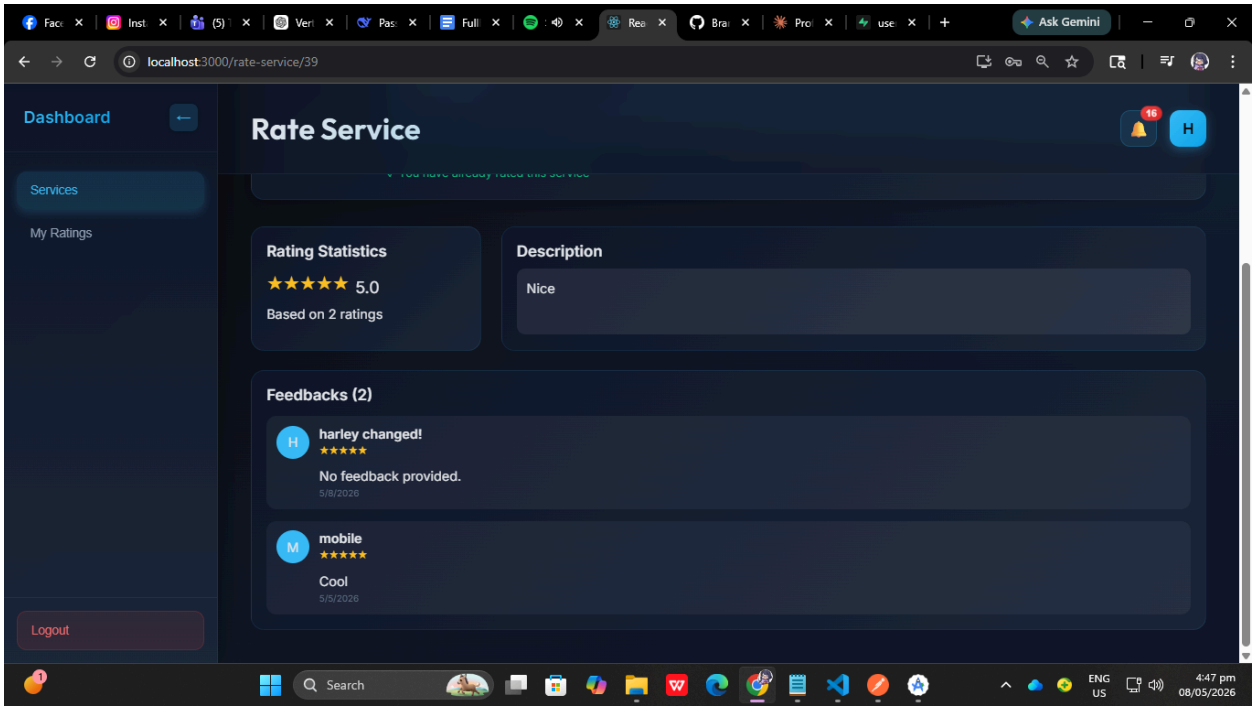


TS-RATE-005 — View Feedback History

Field	Details
Module	Rating System
Objective	Verify user feedback history display
Preconditions	User has submitted ratings
Test Steps	1. Open My Ratings Page
Expected Result	All submitted ratings displayed correctly

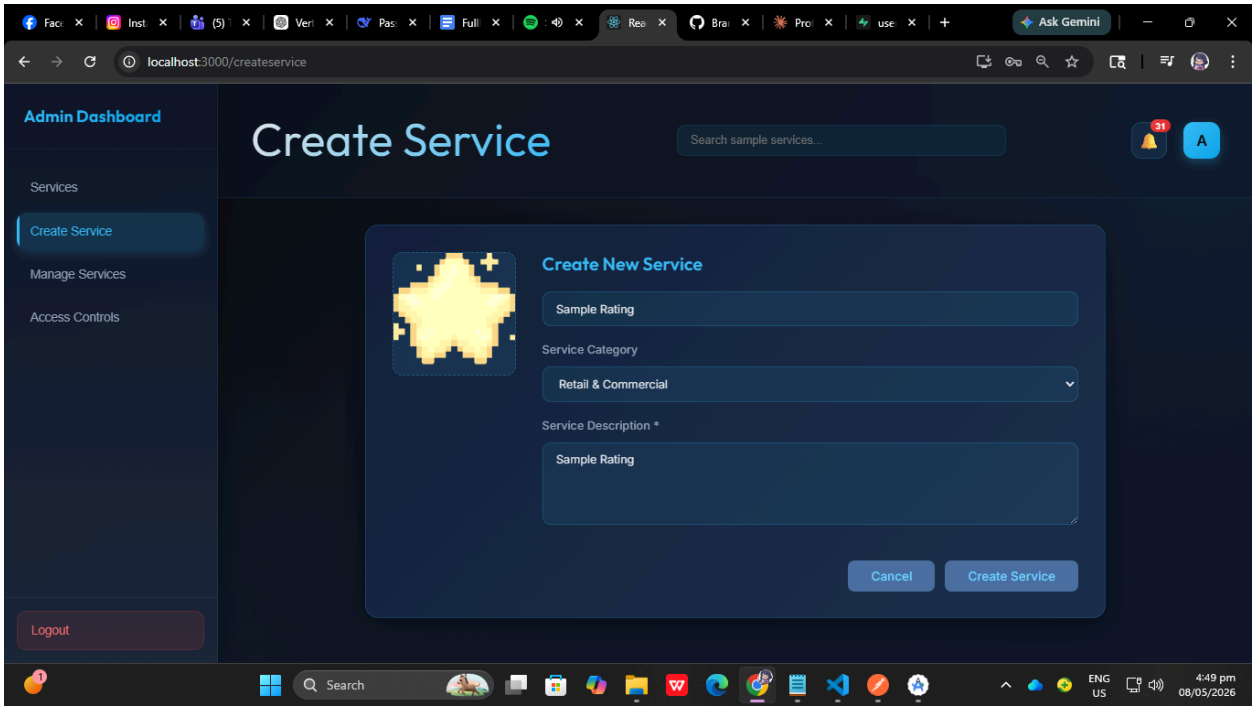
TS-RATE-006 — View Rating Statistics

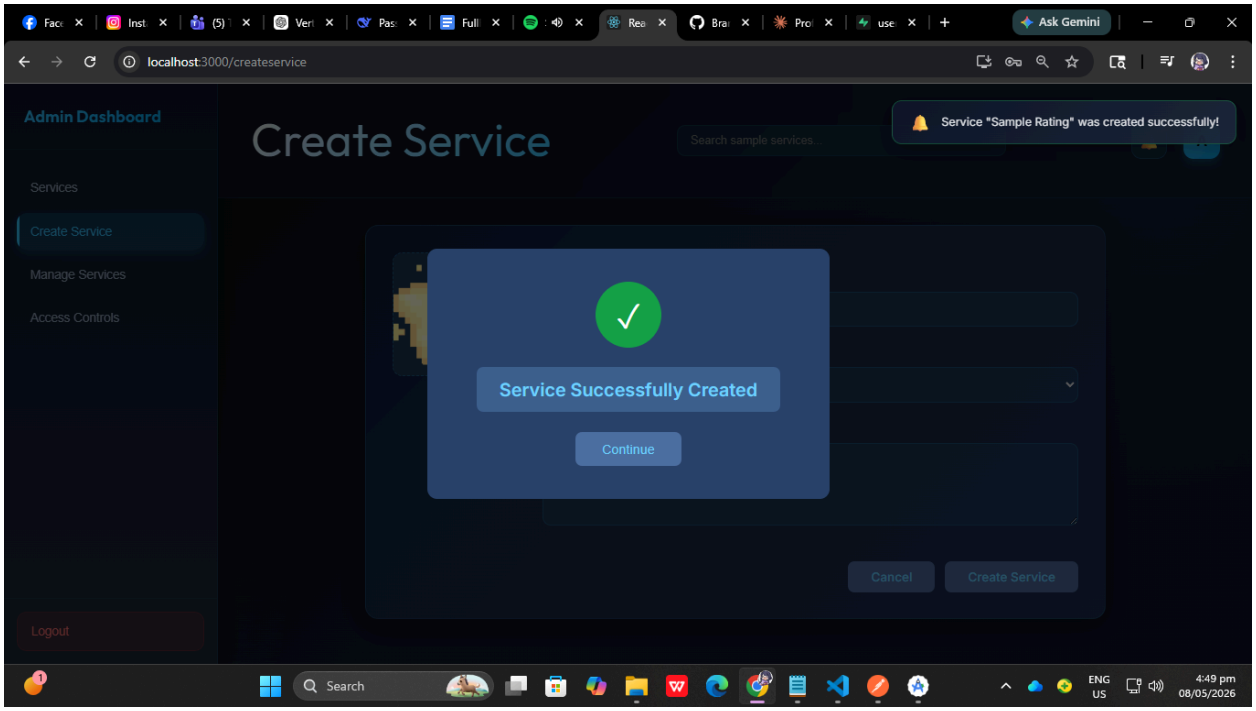
Field	Details
Module	Rating System
Objective	Verify average rating statistics display
Preconditions	Service has ratings
Test Steps	1. Open Service Detail Page
Expected Result	Average rating and rating distribution displayed correctly

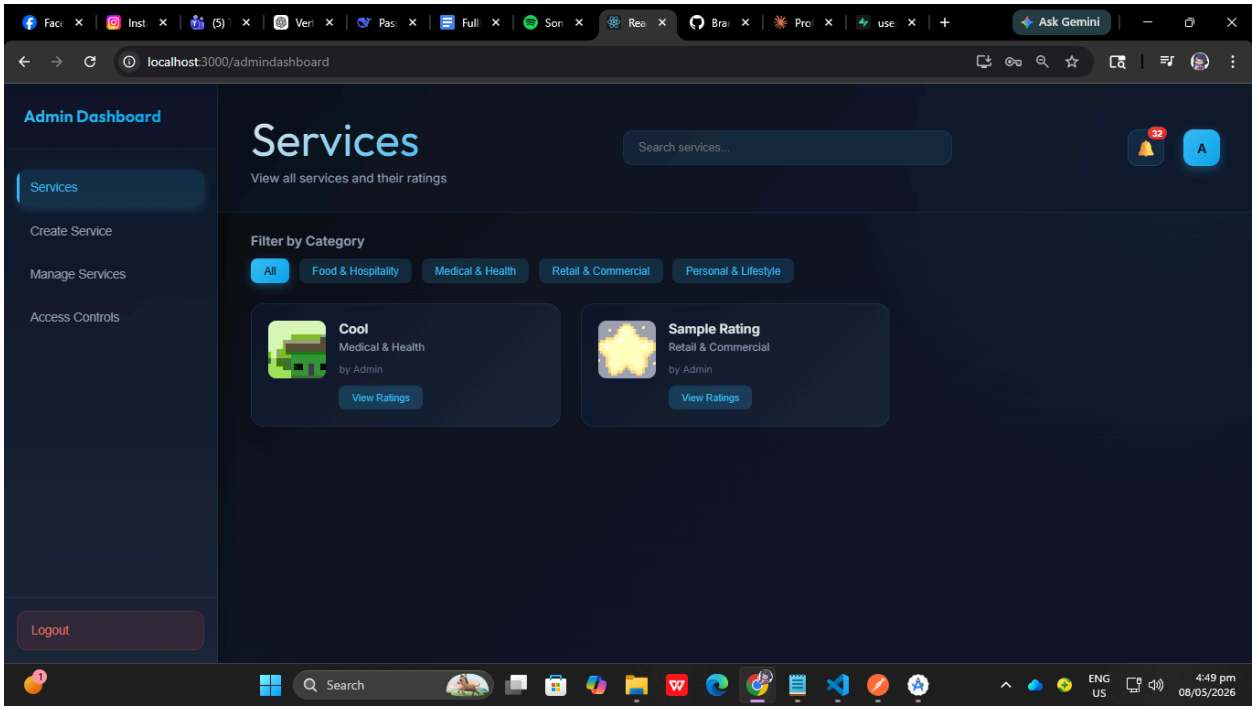


TS-SERV-001 — Create Service

Field	Details
Module	Admin Service Management
Objective	Verify admin can create service
Preconditions	Admin logged in
Test Steps	1. Open Admin Dashboard 2. Click “Create Service” 3. Fill required fields 4. Upload image 5. Click “Create”
Expected Result	Service created successfully and displayed in dashboard

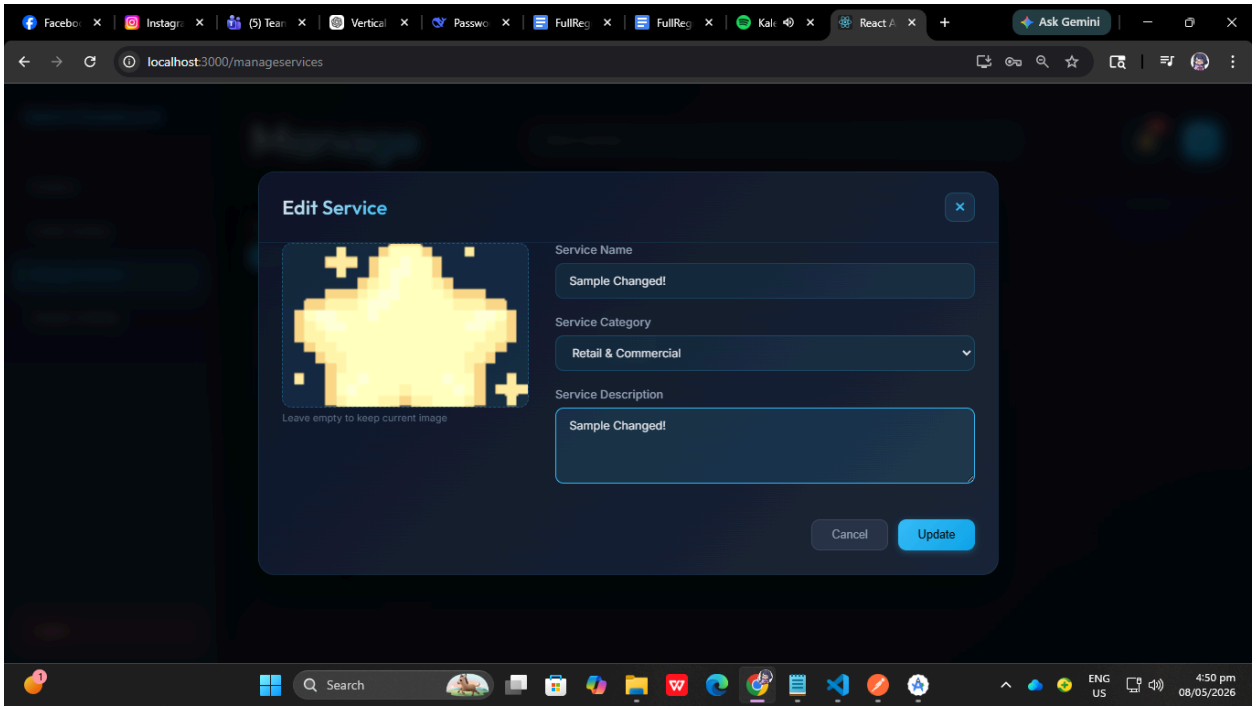

Ask Gemini
localhost:3000/createservice
Admin Dashboard
Create Service
Search sample services...
Services
Create Service
Manage Services
Access Controls
Logout
Create New Service
Sample Rating
Service Category
Retail & Commercial
Service Description *
Sample Rating
Cancel
Create Service
Search
ENG
US
4:49 pm
08/05/2026

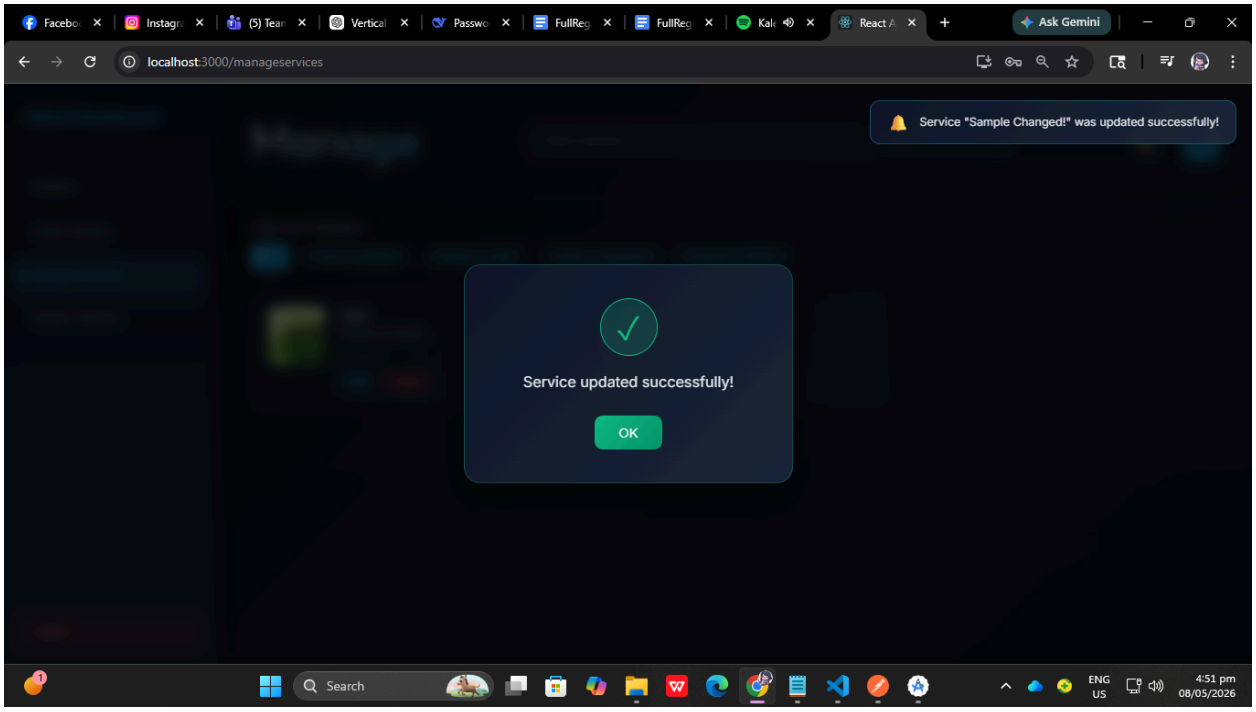

Ask Gemini
localhost:3000/createservice
Admin Dashboard
Create Service
Service "Sample Rating" was created successfully!
Services
Create Service
Manage Services
Access Controls
Logout
Service Successfully Created
Continue
Cancel
Create Service
Search
ENG
US
4:49 pm
08/05/2026

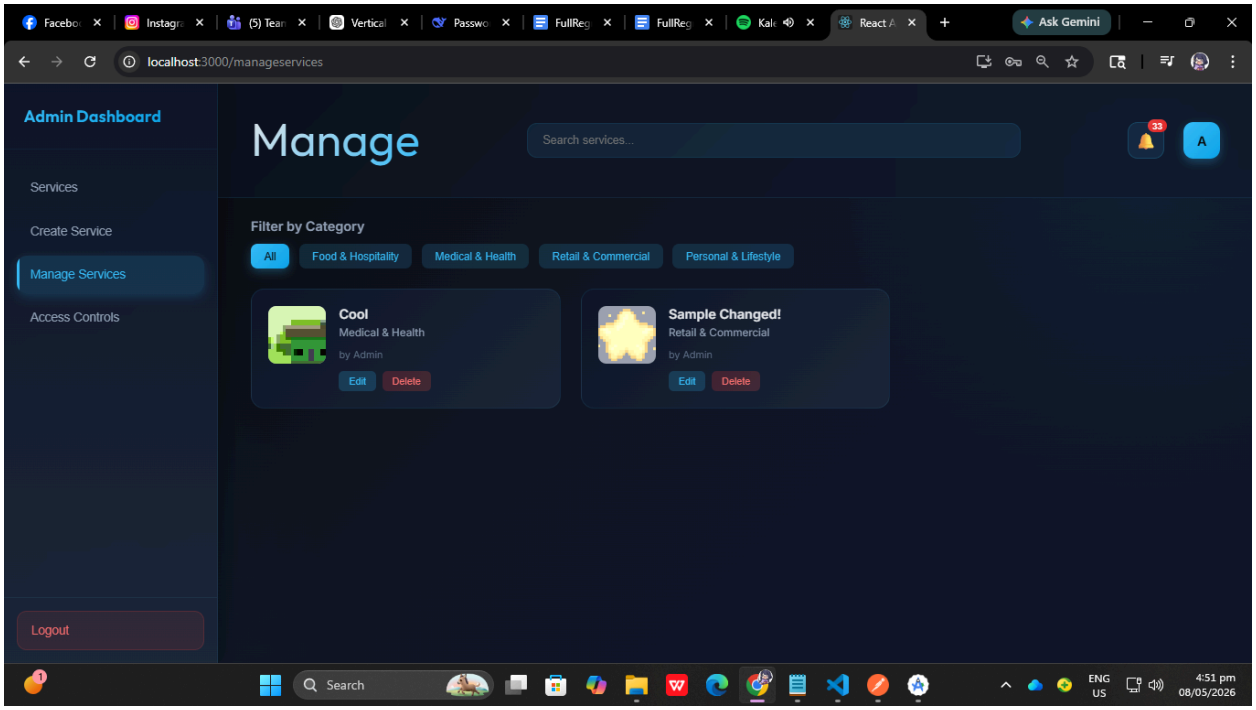


TS-SERV-002 — Edit Service

Field	Details
Module	Admin Service Management
Objective	Verify admin can edit service
Preconditions	Existing service available
Test Steps	1. Open Manage Services Page 2. Select service 3. Click “Edit” 4. Update fields 5. Save changes
Expected Result	Updated service information displayed successfully

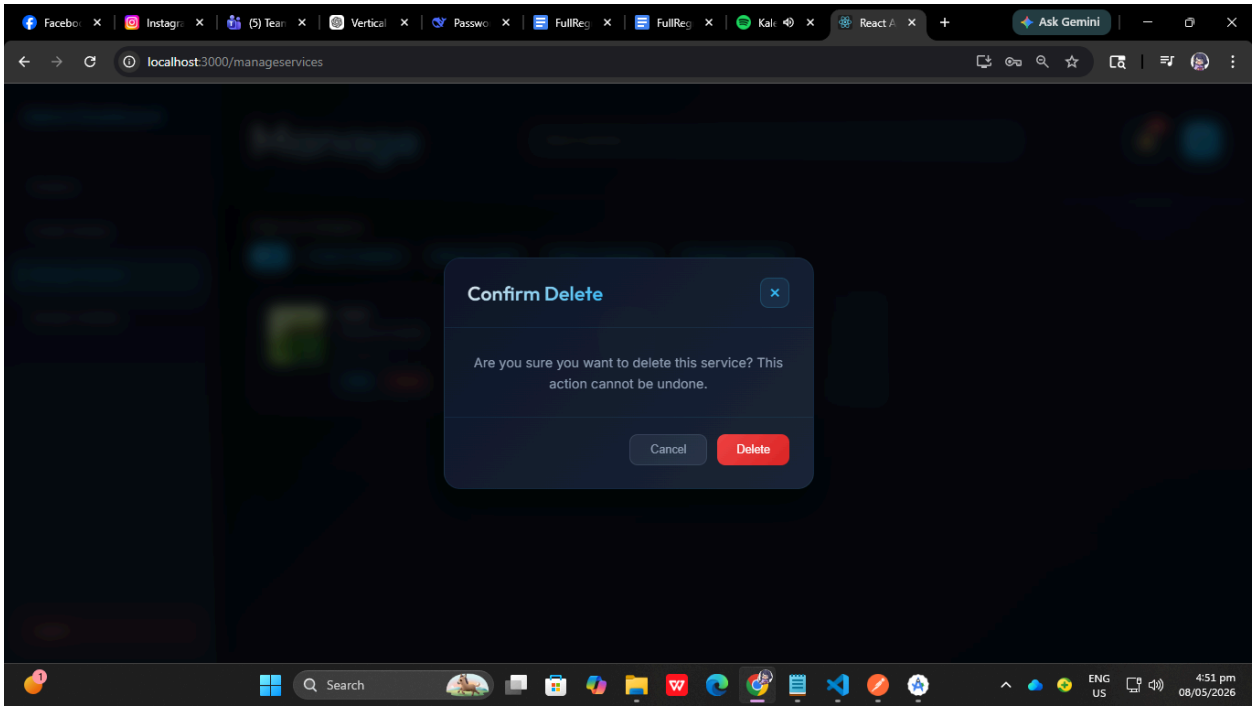

localhost:3000/manageservices
Edit Service
Service Name
Sample Changed!
Service Category
Retail & Commercial
Service Description
Sample Changed!
Leave empty to keep current image
Cancel
Update
Search
ENG
US
4:50 pm
08/05/2026

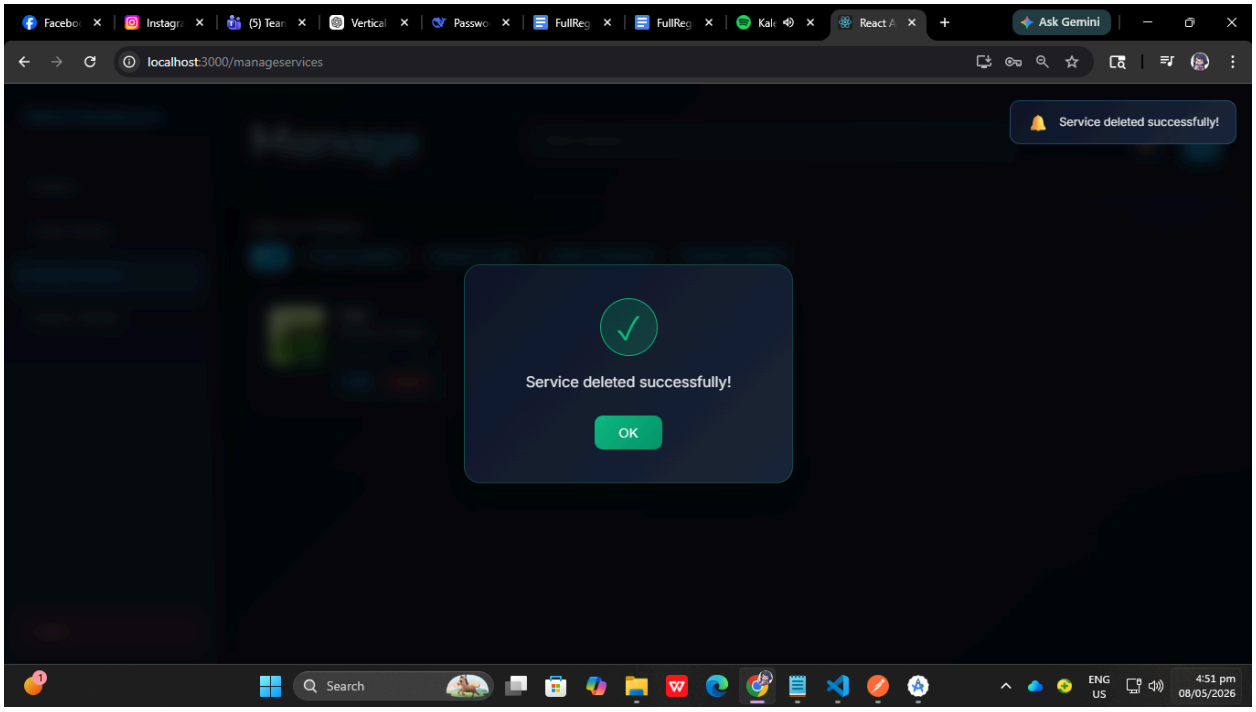

localhost:3000/manageservices
Service "Sample Changed!" was updated successfully!
Service updated successfully!
OK
Search
ENG
US
4:51 pm
08/05/2026

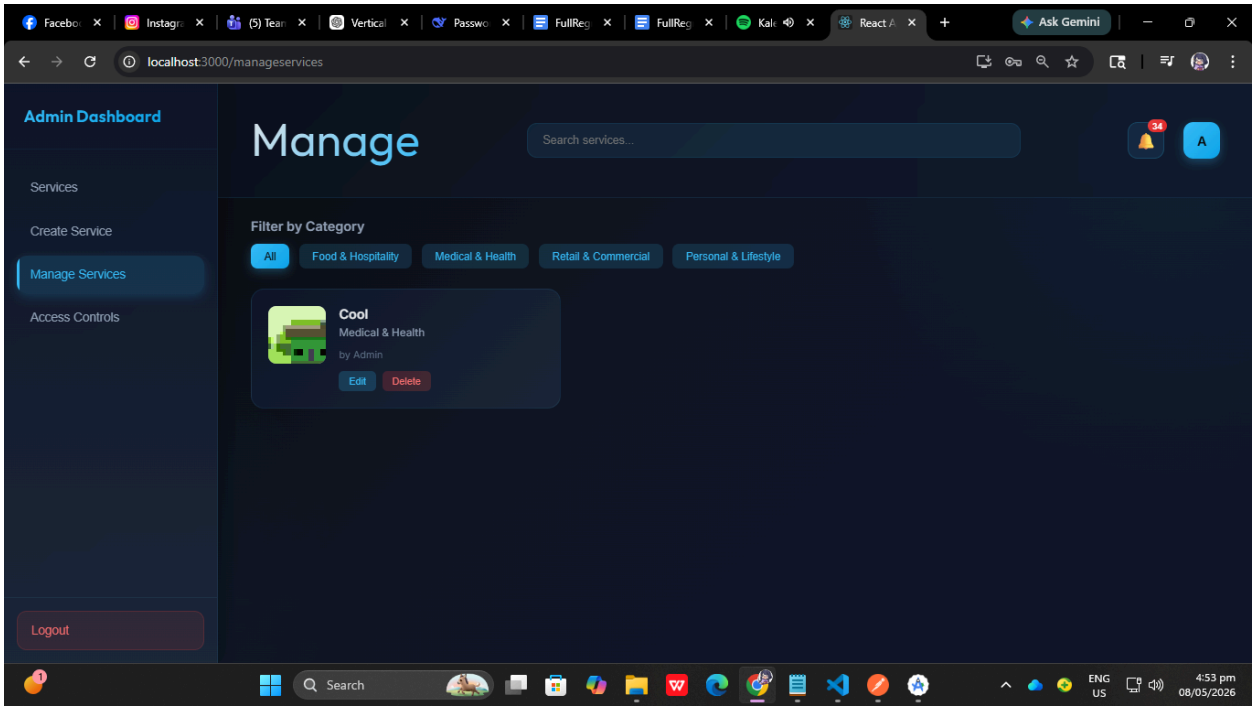


TS-SERV-003 — Delete Service

Field	Details
Module	Admin Service Management
Objective	Verify admin can delete service
Preconditions	Existing service available
Test Steps	1. Open Manage Services Page 2. Select service 3. Click Delete button 4. Confirm deletion
Expected Result	Service removed successfully from system

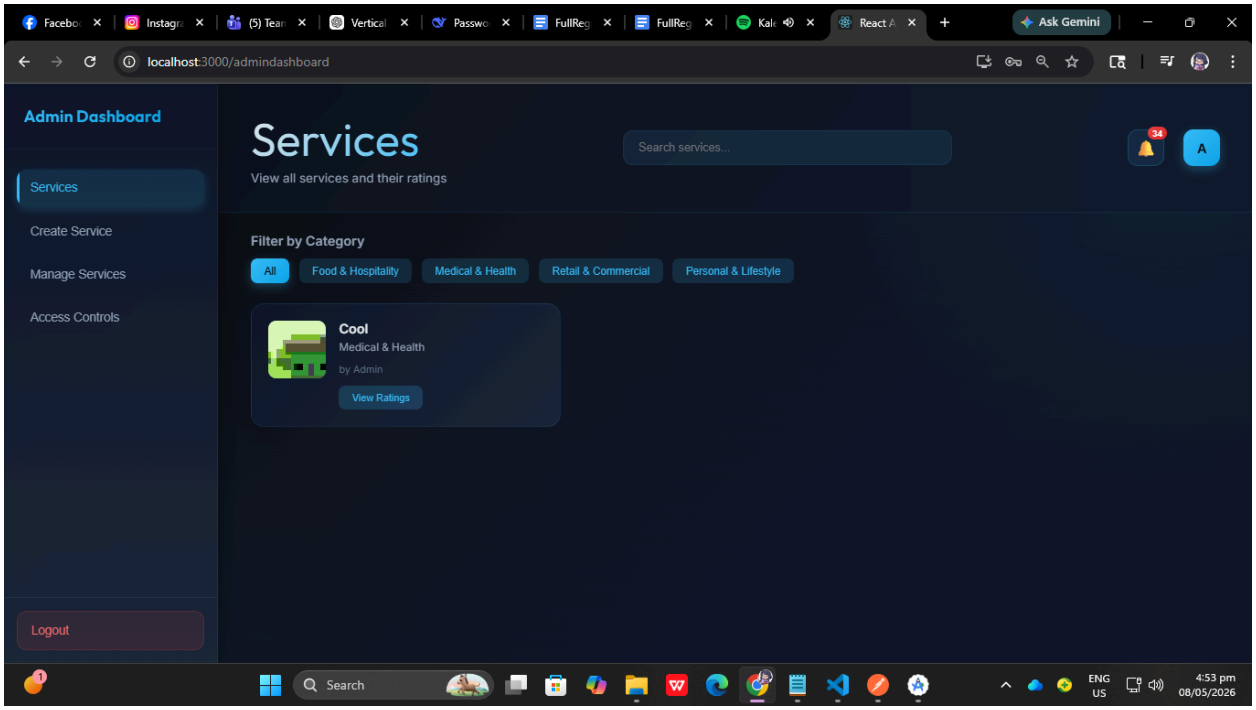

localhost:3000/manageservices
Confirm Delete
Are you sure you want to delete this service? This action cannot be undone.
Cancel
Delete
Search
ENG
US
4:51 pm
08/05/2026


localhost:3000/manageservices
Service deleted successfully!
Service deleted successfully!
OK
Search
ENG
US
4:51 pm
08/05/2026



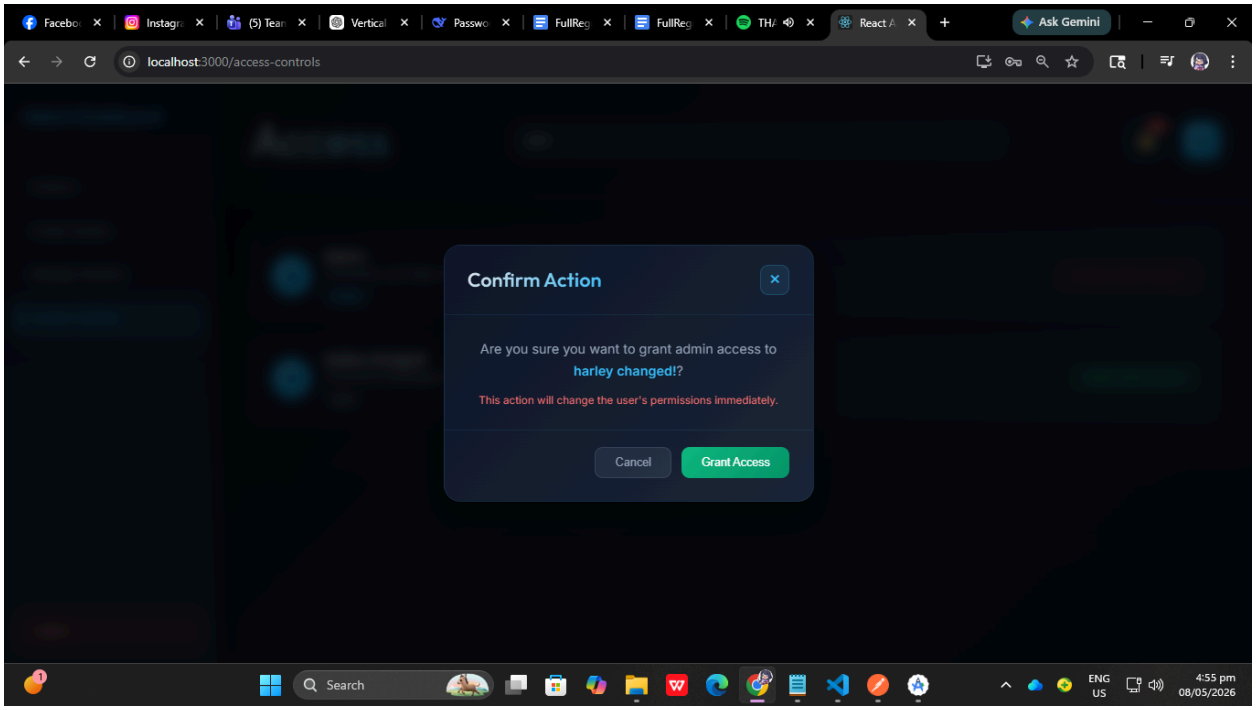
TS-SERV-004 — View Created Services

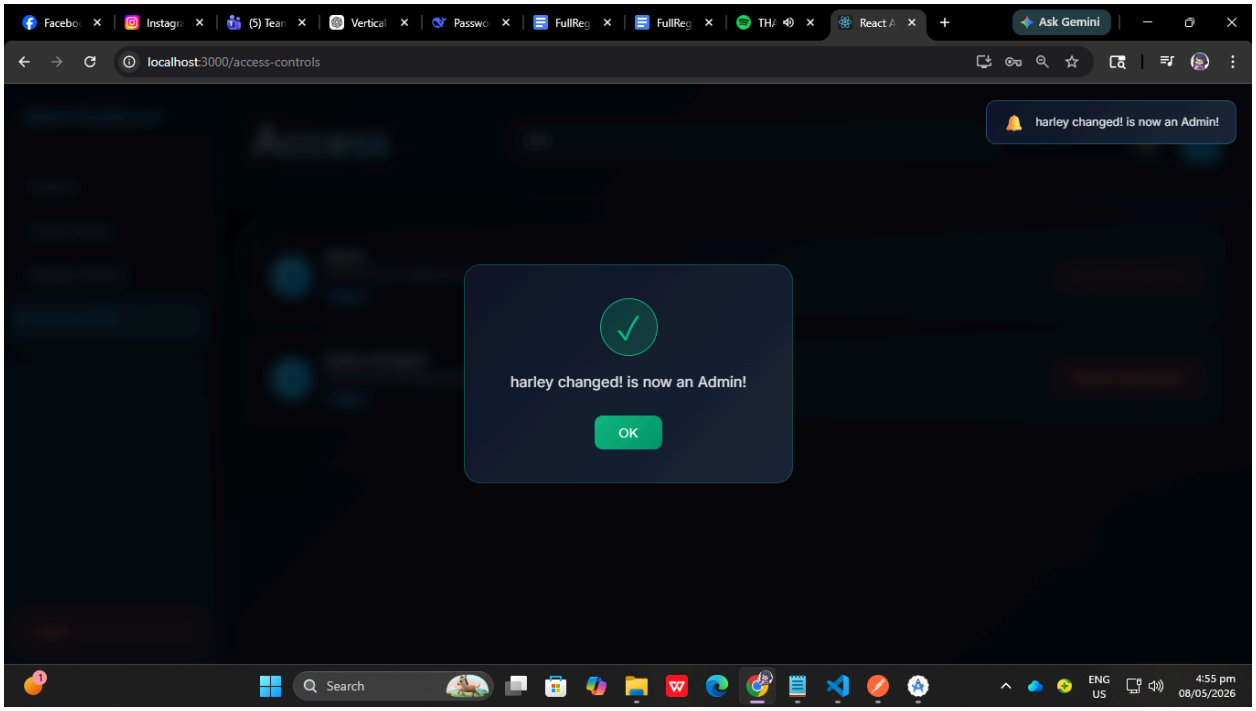
Field	Details
Module	Admin Service Management
Objective	Verify admin can view created services
Preconditions	Services exist
Test Steps	1. Open Admin Dashboard
Expected Result	All admin-created services displayed



TS-ACCESS-001 — Grant Admin Access

Field	Details
Module	Access Control
Objective	Verify admin can grant admin privileges
Preconditions	Admin logged in
Test Steps	1. Open Access Control Page 2. Select user 3. Click “Grant Admin Access” 4. Confirm action
Expected Result	User role updated to ADMIN successfully

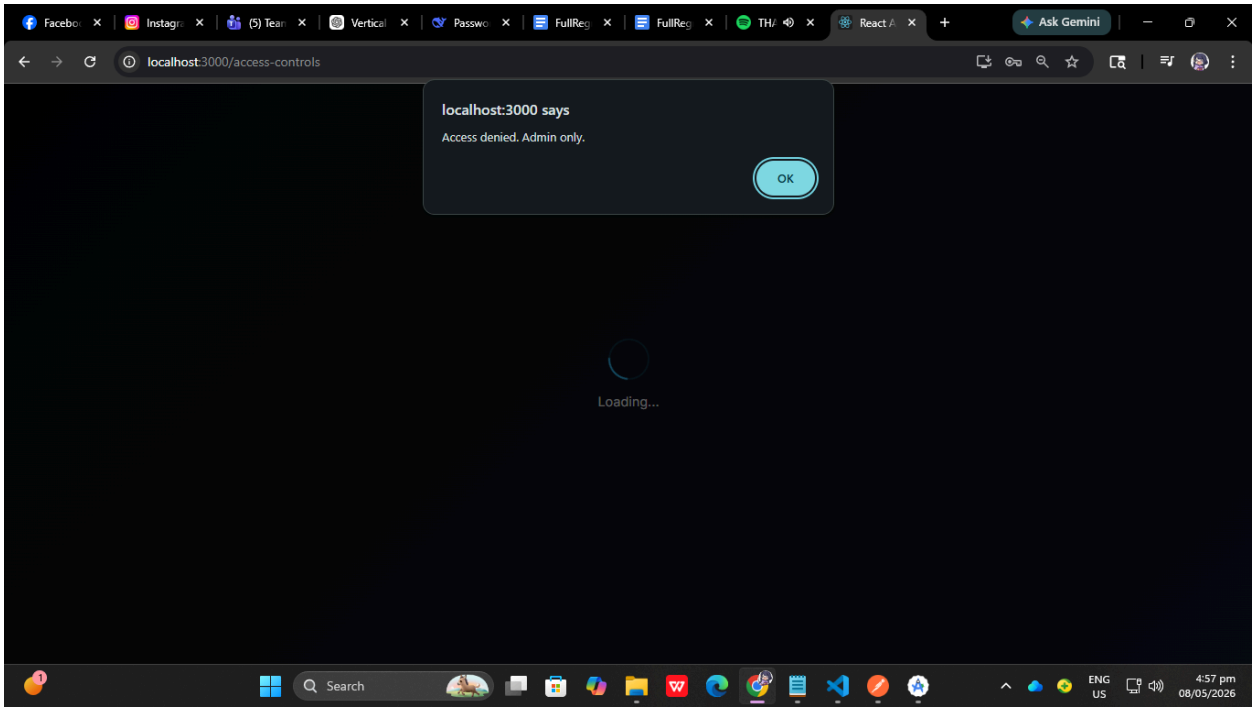




TS-ACCESS-002 — Prevent Unauthorized Access Control

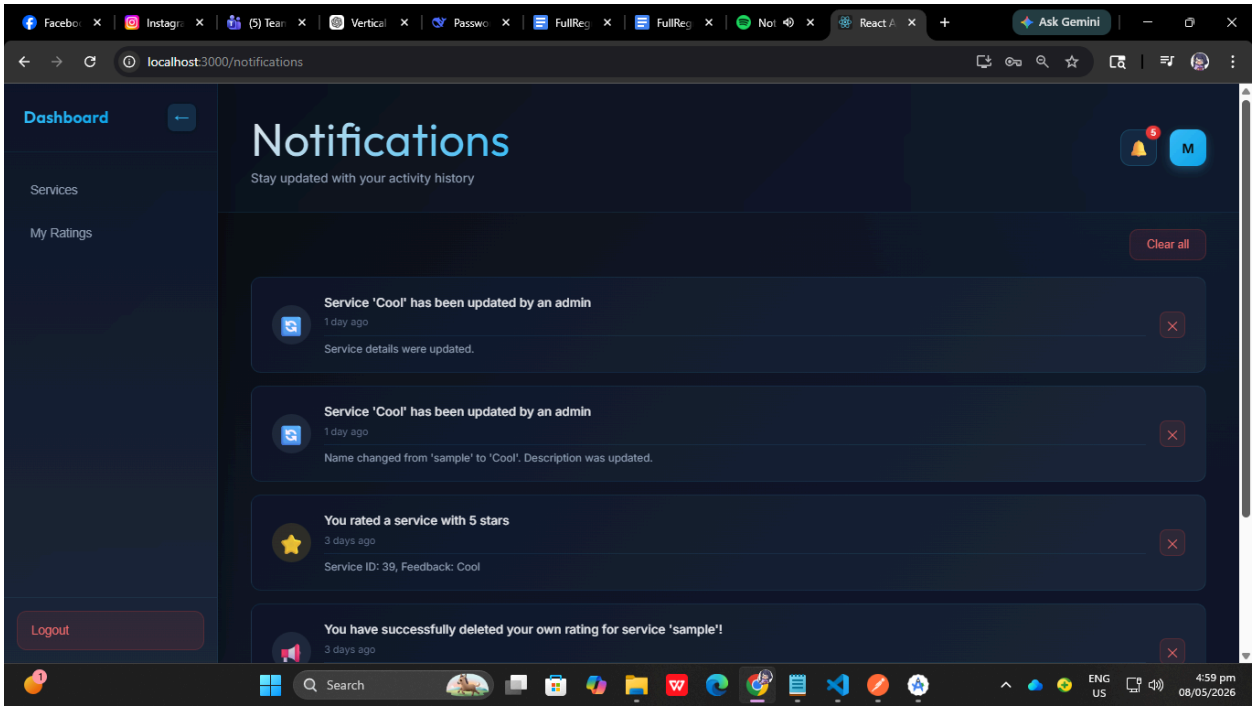
Field	Details
Module	Access Control

Objective	Verify normal user cannot access admin control
Preconditions	User logged in as regular user
Test Steps	1. Attempt to access Access Control page URL
Expected Result	Access denied or redirected



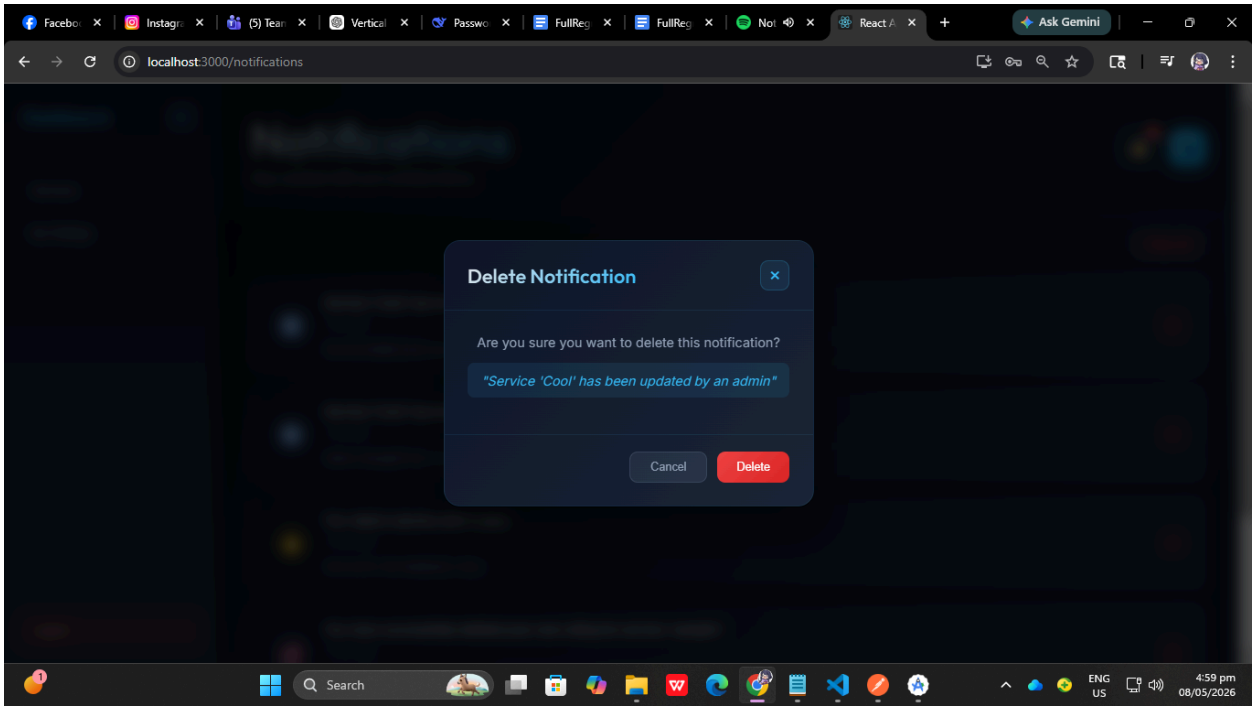
TS-NOTIF-001 — View Notifications

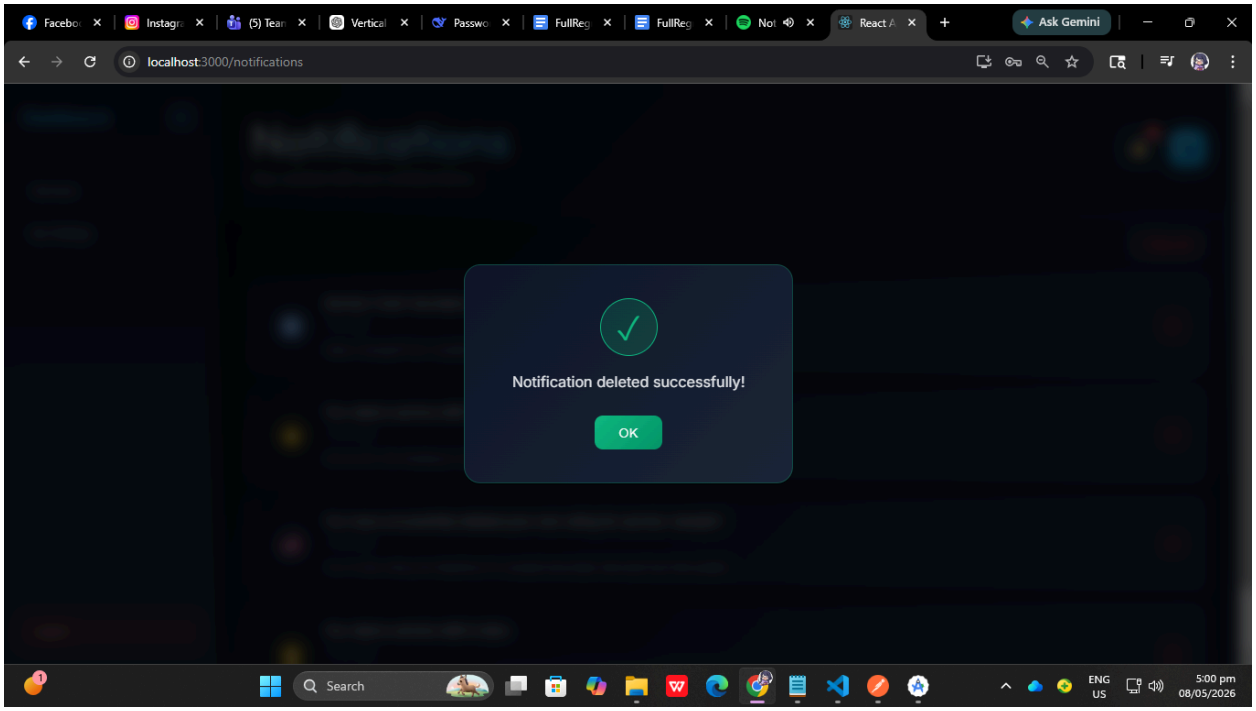
Field	Details
Module	Notifications
Objective	Verify notifications are displayed
Preconditions	User has notifications
Test Steps	1. Open Notifications Page
Expected Result	Notifications displayed correctly



TS-NOTIF-002 — Delete Notification

Field	Details
Module	Notifications
Objective	Verify single notification deletion
Preconditions	Existing notification available
Test Steps	1. Open Notifications Page 2. Click Delete icon
Expected Result	Notification removed successfully

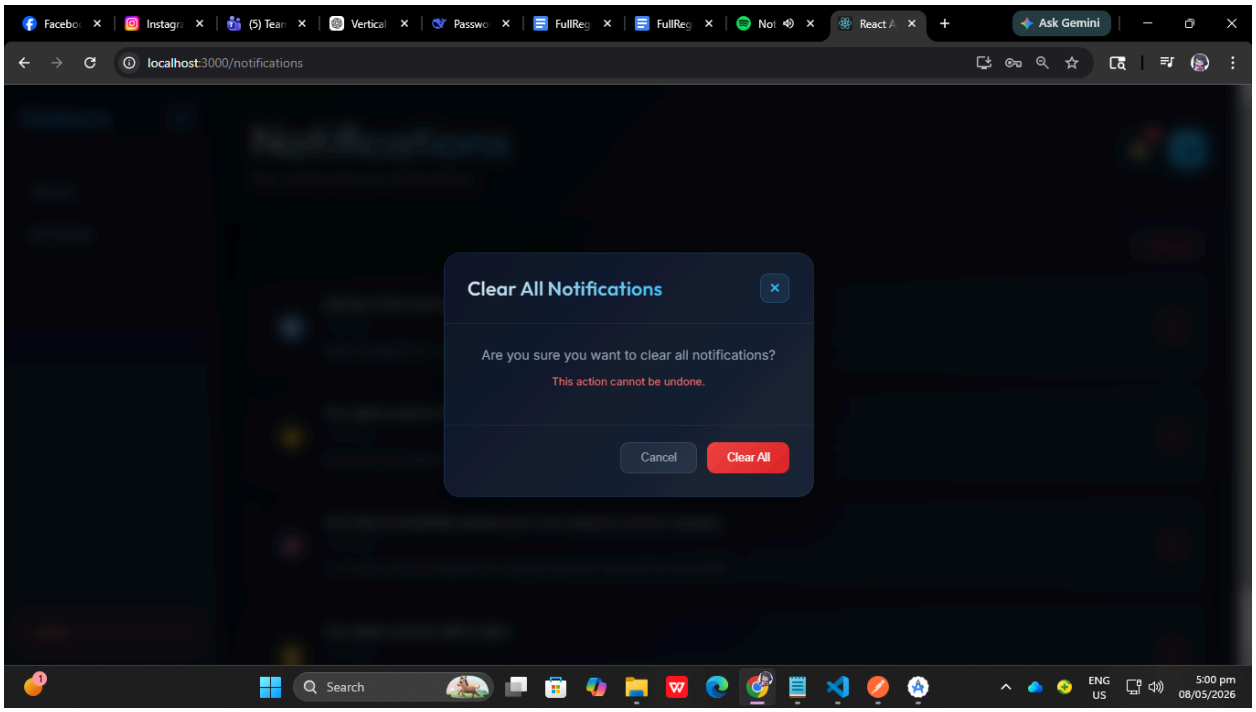


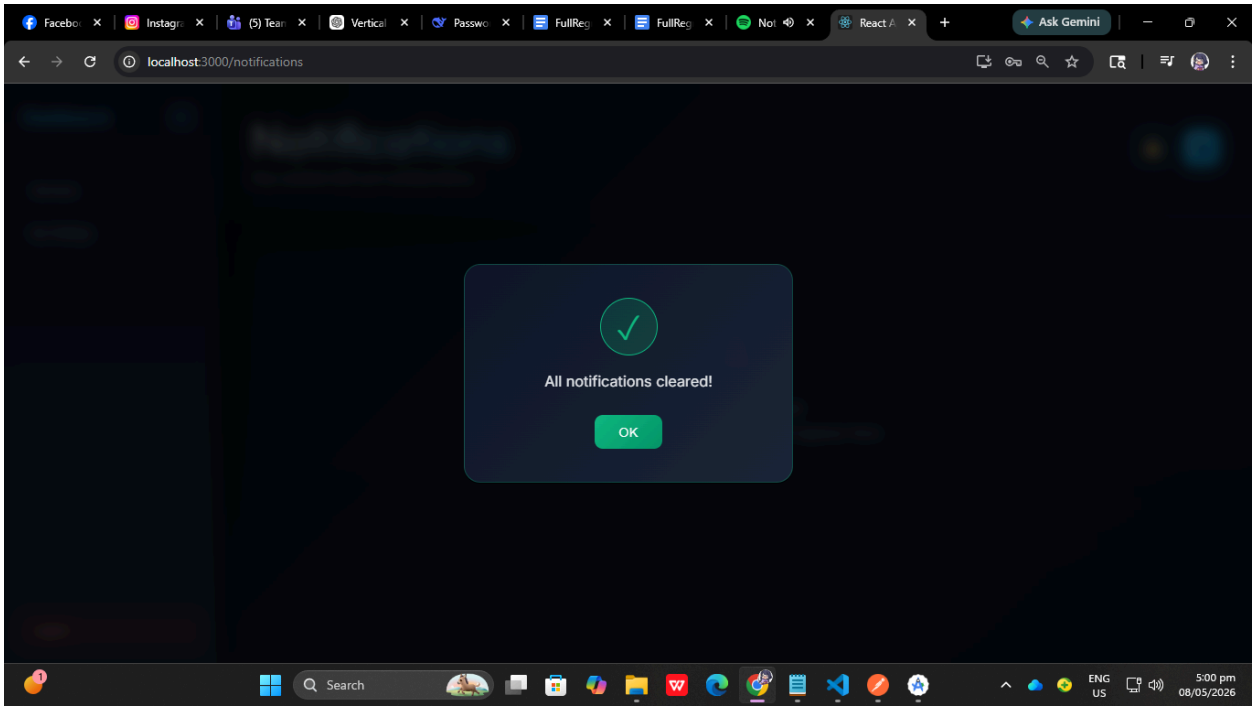


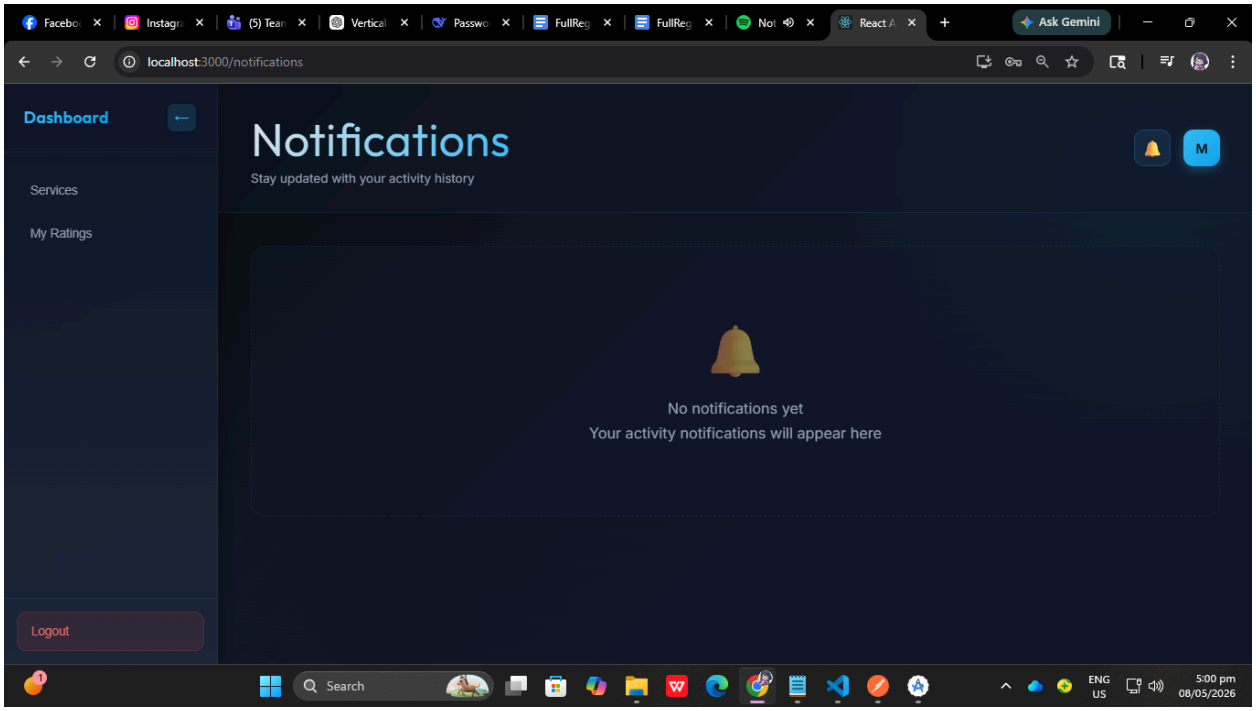
TS-NOTIF-003 — Clear All Notifications

Field	Details
Module	Notifications

Objective	Verify clearing all notifications
Preconditions	Multiple notifications available
Test Steps	1. Open Notifications Page 2. Click “Clear All”
Expected Result	All notifications removed successfully

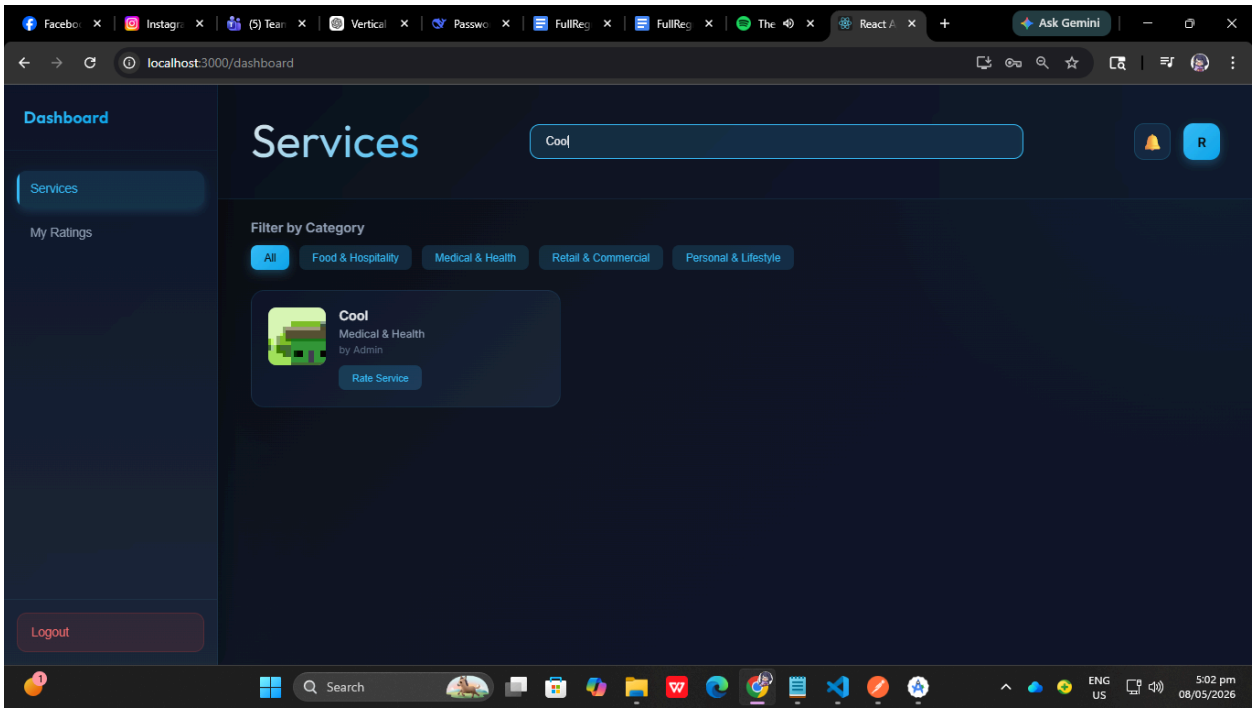


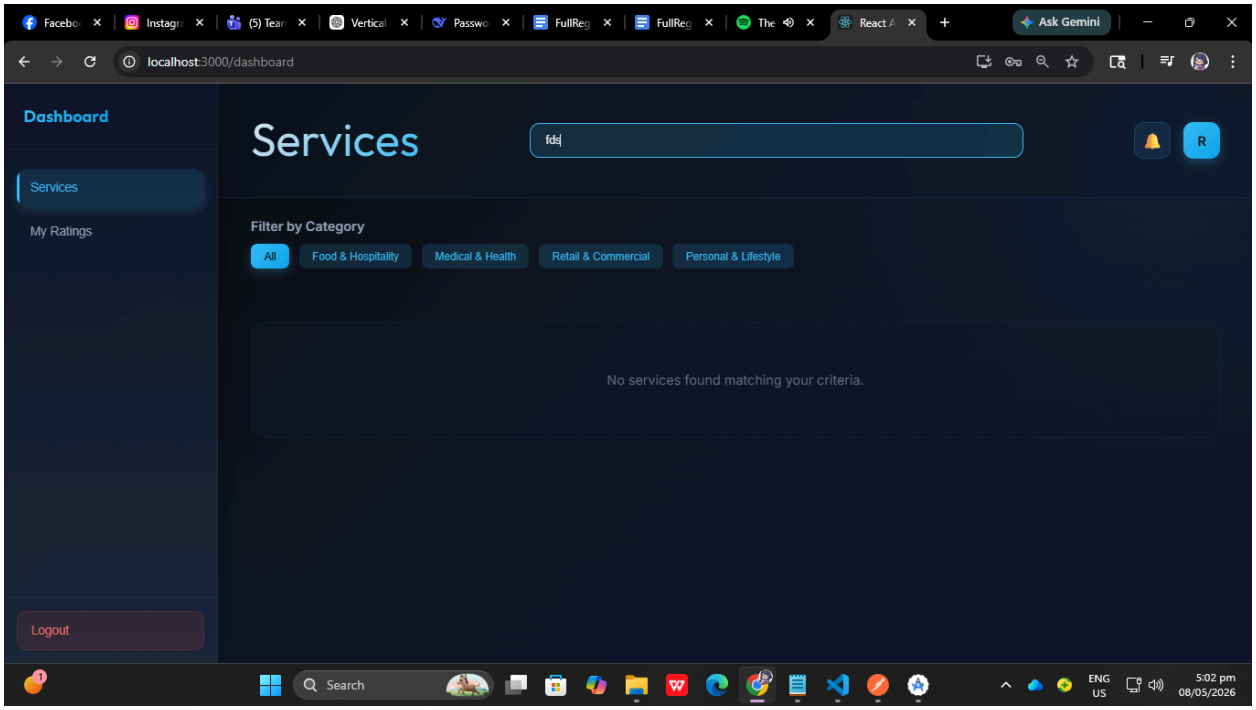

localhost:3000/notifications
All notifications cleared!
OK


localhost:3000/notifications
Dashboard
Services
My Ratings
Logout
Notifications
Stay updated with your activity history
No notifications yet
Your activity notifications will appear here

TS-SEARCH-001 — Search Service

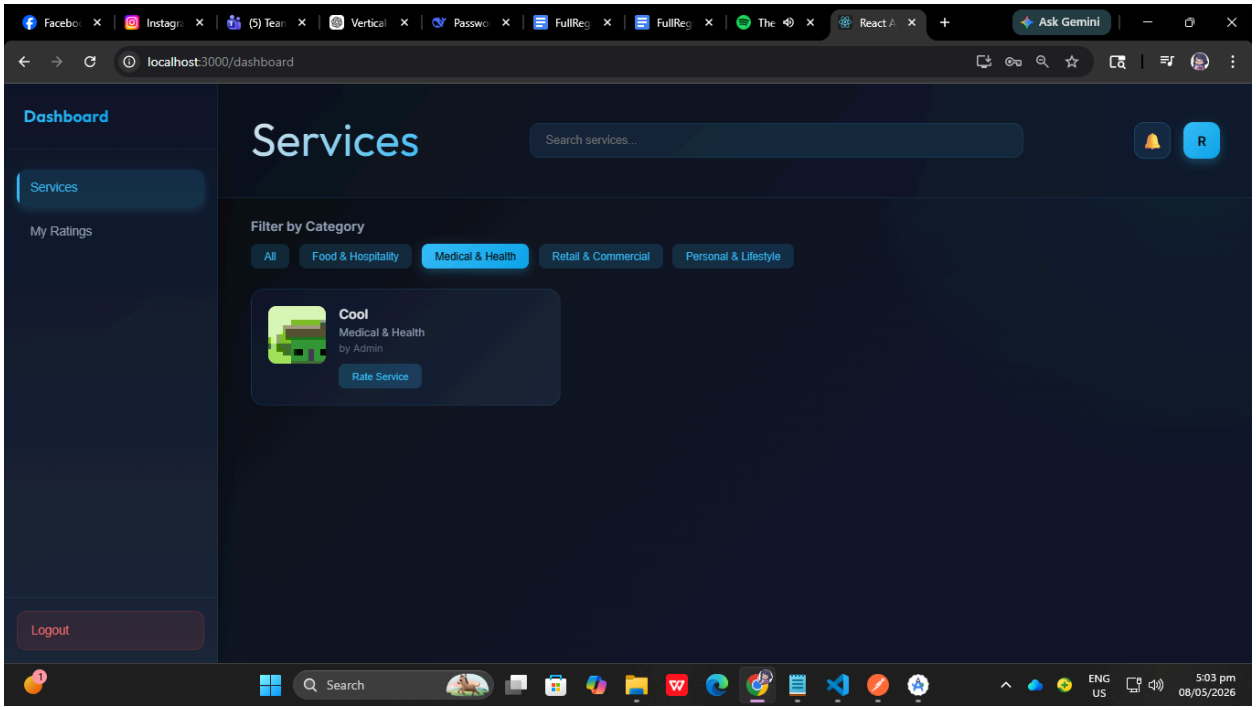
Field	Details
Module	Search
Objective	Verify search functionality
Preconditions	Services exist
Test Steps	1. Enter service keyword in search bar
Expected Result	Matching services displayed





TS-SEARCH-002 — Filter Services by Category

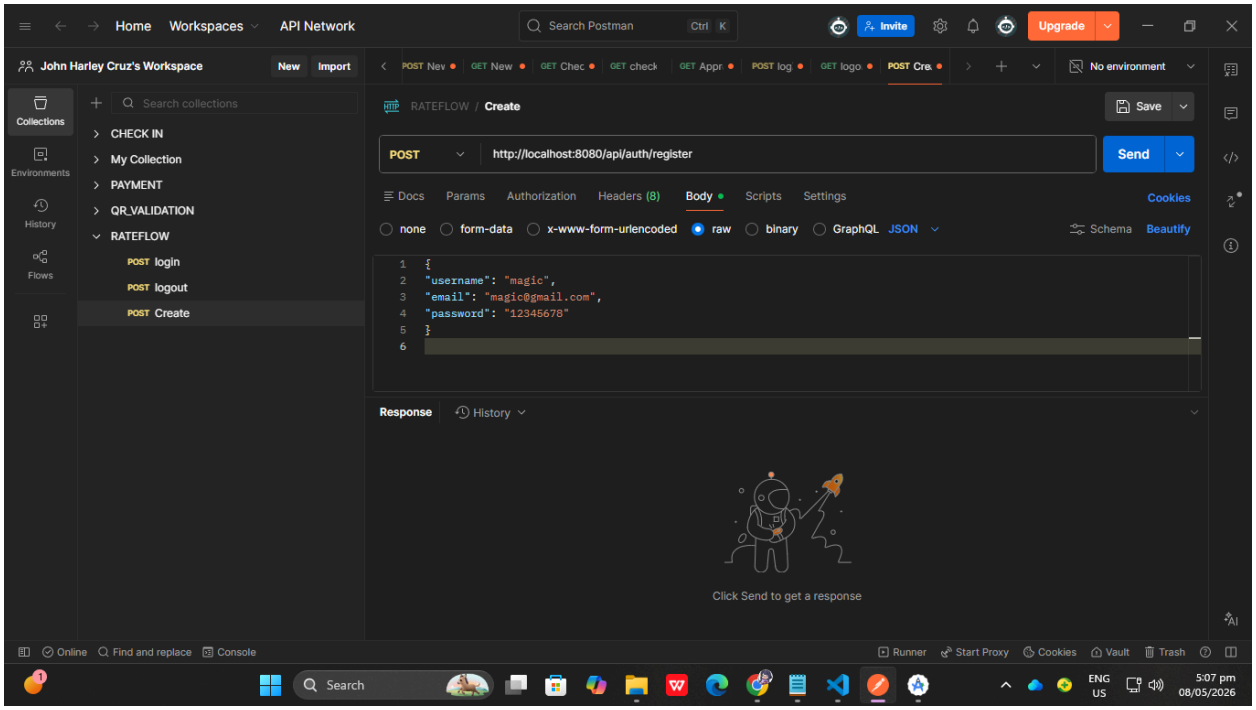
Field	Details
Module	Filter
Objective	Verify category filtering
Preconditions	Multiple categorized services exist
Test Steps	1. Click category filter button
Expected Result	Only matching category services displayed

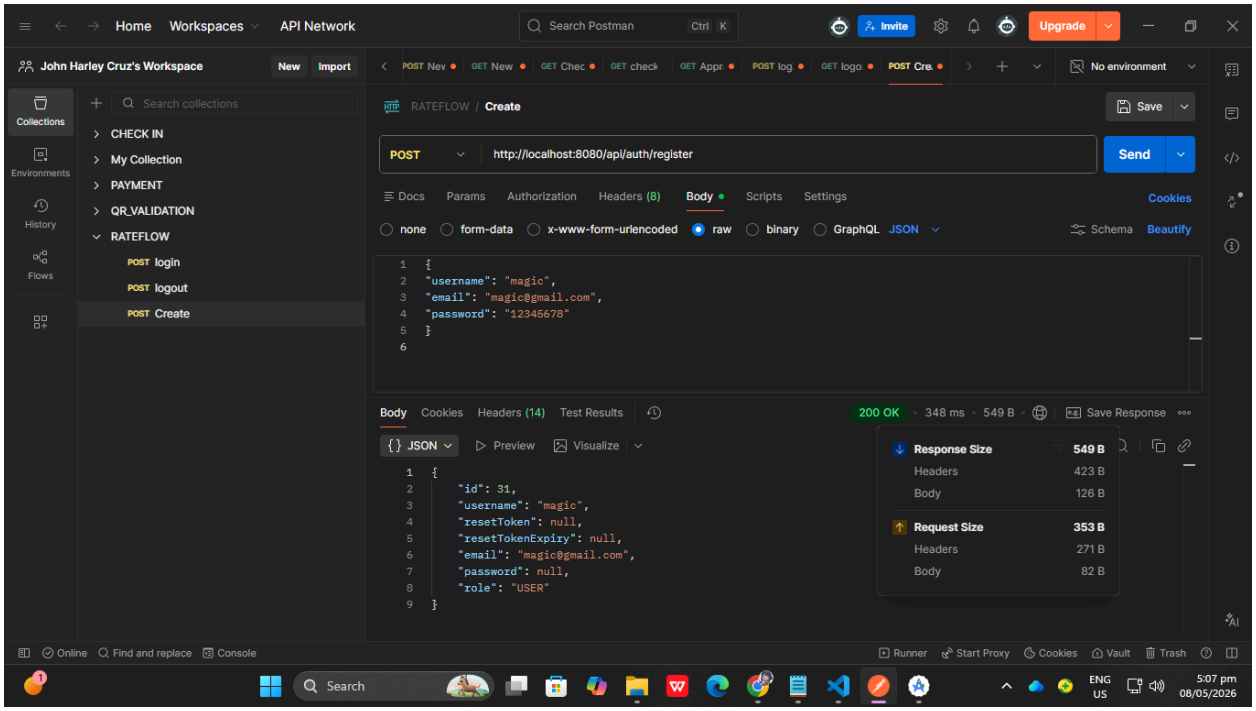


Automated Test Evidence

ATC-AUTH-001 — User Registration API

Field	Details
Test ID	ATC-AUTH-001
Endpoint	POST `/api/auth/register`
Tool	Postman
Objective	Verify successful user registration
Request Body	username, email, password
Automated Validation	Check if status code is 201
Expected Result	User account created successfully
Status	Passed

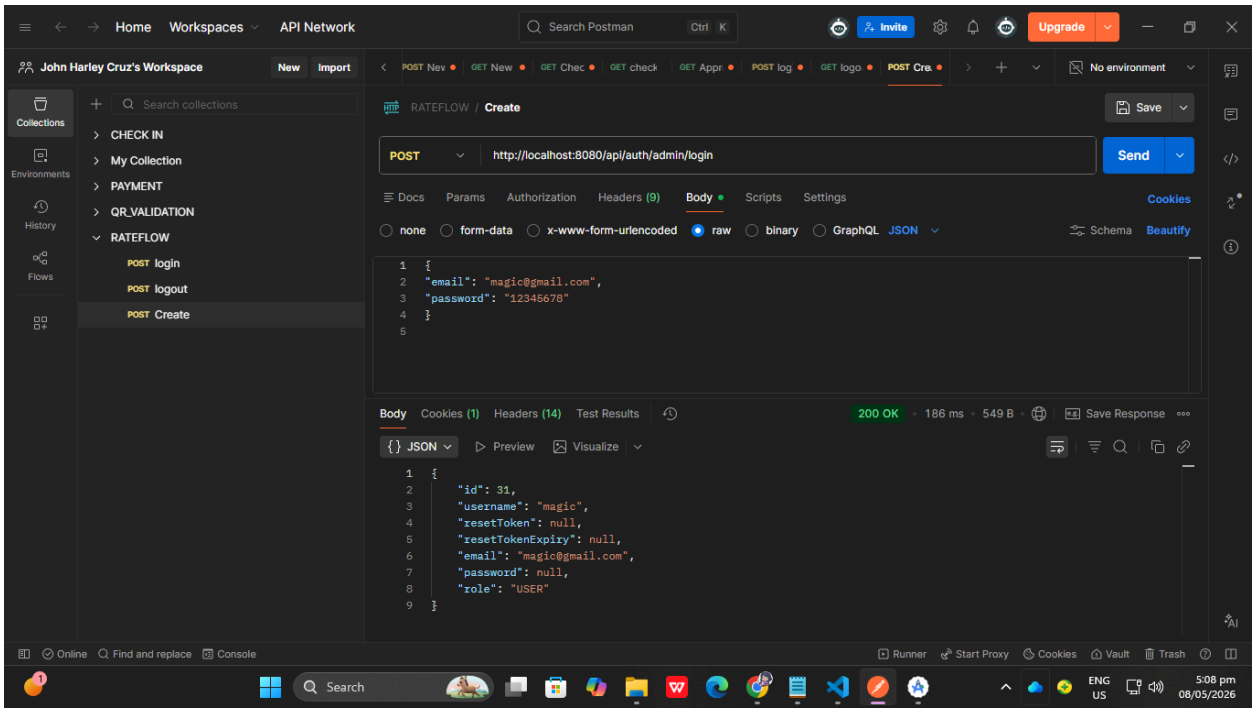




ATC-AUTH-002 — User Login API

Field	Details
Test ID	ATC-AUTH-002

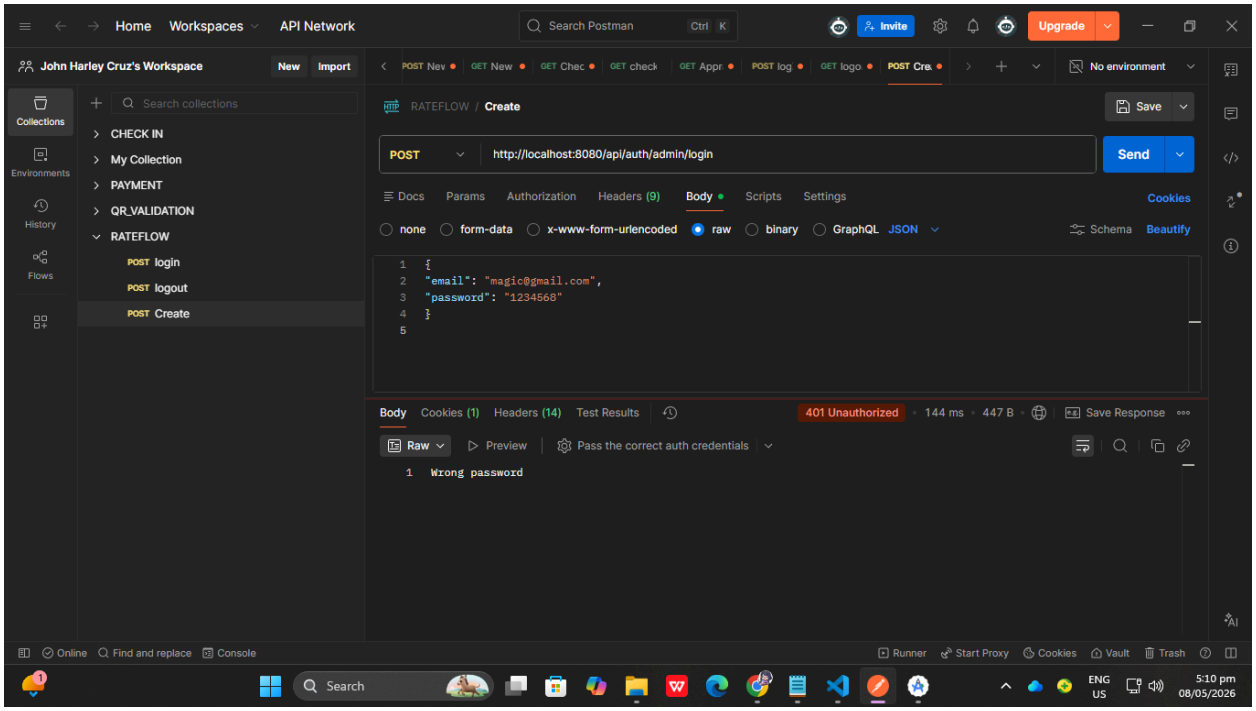
Endpoint	POST `/api/auth/admin/login`
Tool	Postman
Objective	Verify successful user login
Automated Validation	Verify status code and returned role
Expected Result	User authenticated successfully
Status	Passed



ATC-AUTH-003 — Invalid Login Credentials

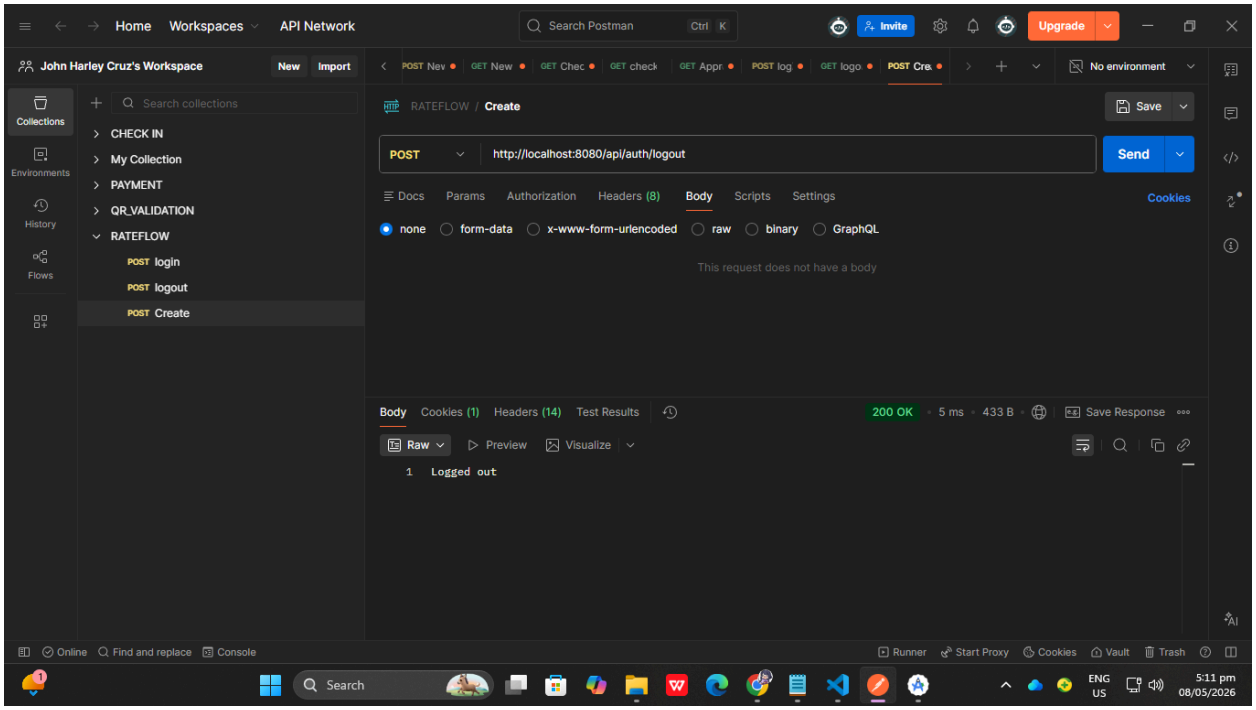
Field	Details
Test ID	ATC-AUTH-003
Endpoint	POST `/api/auth/admin/login`
Tool	Postman
Objective	Verify invalid login rejected

Automated Validation	Check unauthorized response
Expected Result	HTTP 401 Unauthorized
Status	Passed



ATC-AUTH-004 — User Logout

Field	Details
Test ID	ATC-AUTH-004
Endpoint	POST `/api/auth/logout`
Tool	Postman
Objective	Verify session invalidation
Automated Validation	Check successful logout
Expected Result	Session terminated
Status	Passed



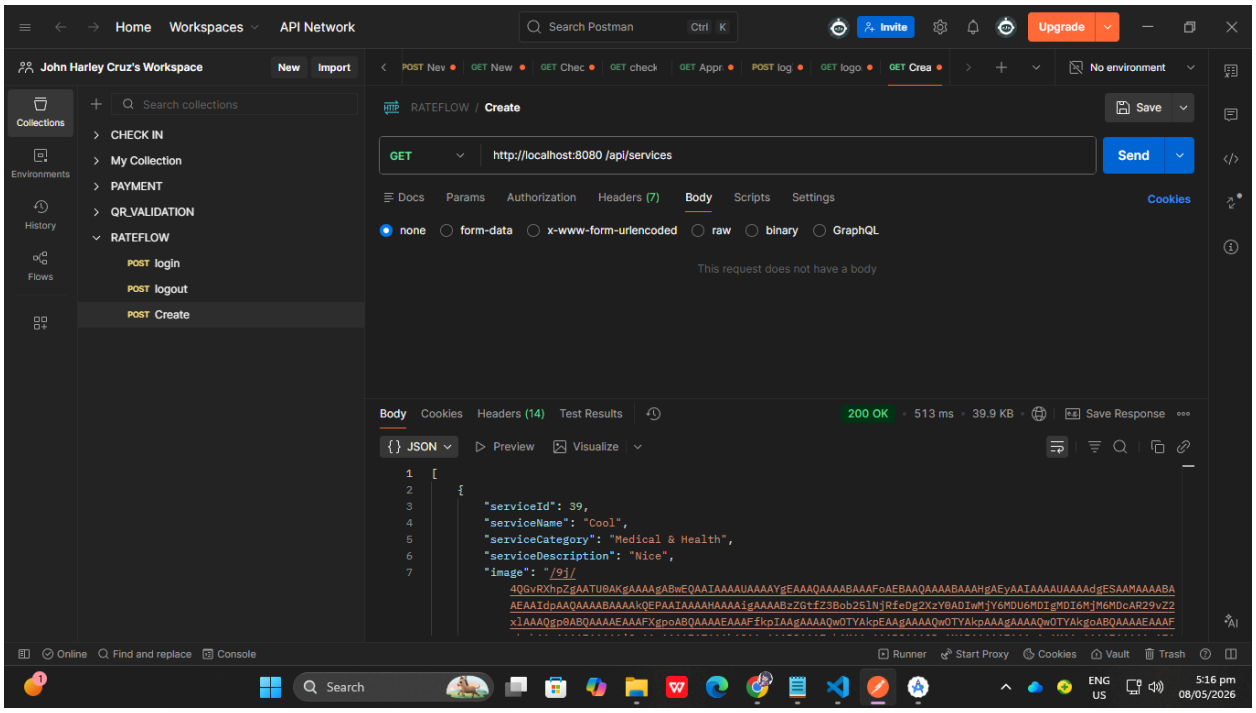
ATC-AUTH-005 — Google Authentication Login

Field	Details
Test ID	ATC-AUTH-005
Endpoint	POST `/api/auth/google`
Tool	Postman
Objective	Verify Google login
Automated Validation	Check success response
Expected Result	User authenticated via Google
Status	Passed

ATC-SERV-001 — Get All Services

Field	Details
Test ID	ATC-SERV-001

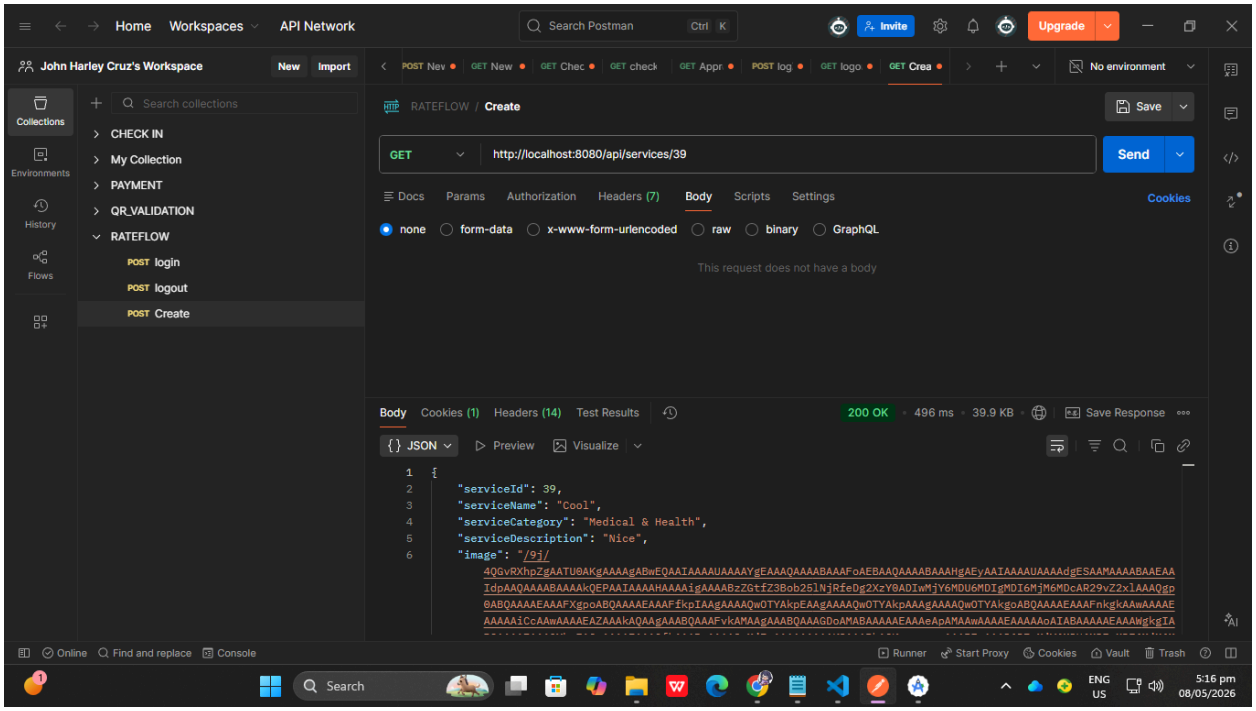
Endpoint	GET `/api/services`
Tool	Postman
Objective	Verify retrieval of services
Automated Validation	Check returned array
Expected Result	Services list returned successfully
Status	Passed



ATC-SERV-002 — Get Service By ID

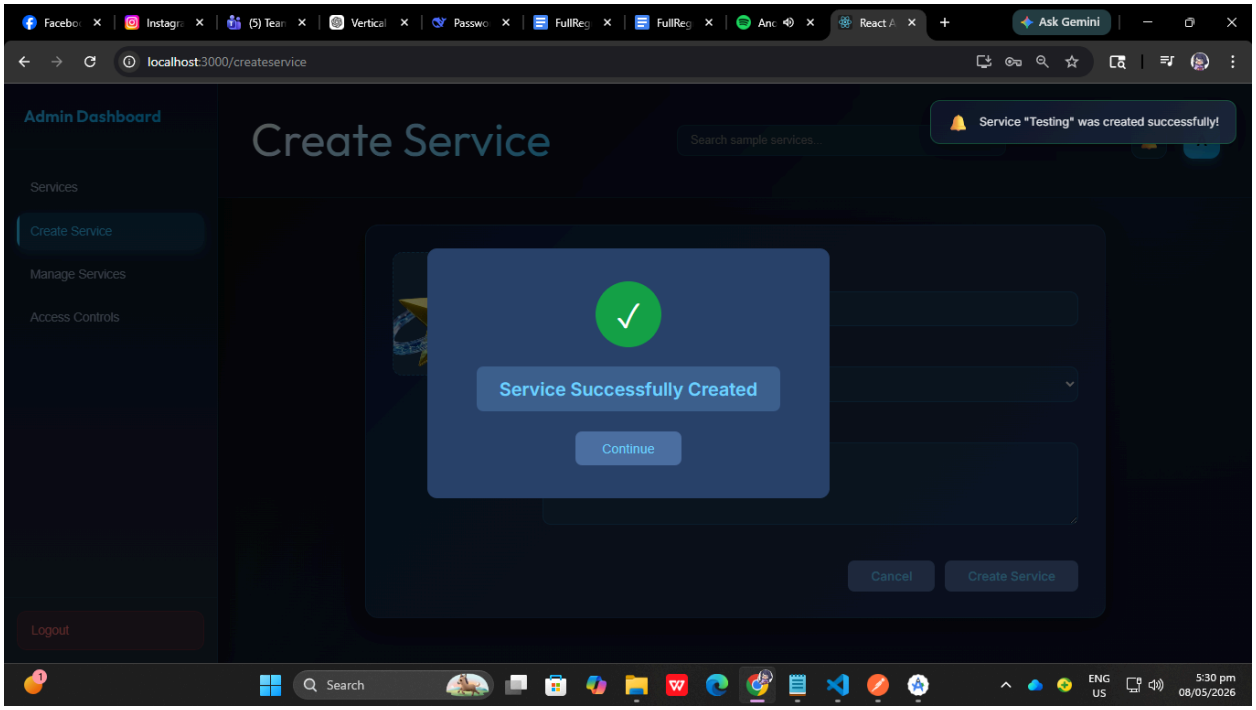
Field	Details
Test ID	ATC-SERV-002
Endpoint	GET `/api/services/{serviceId}`
Tool	Postman
Objective	Verify service details retrieval

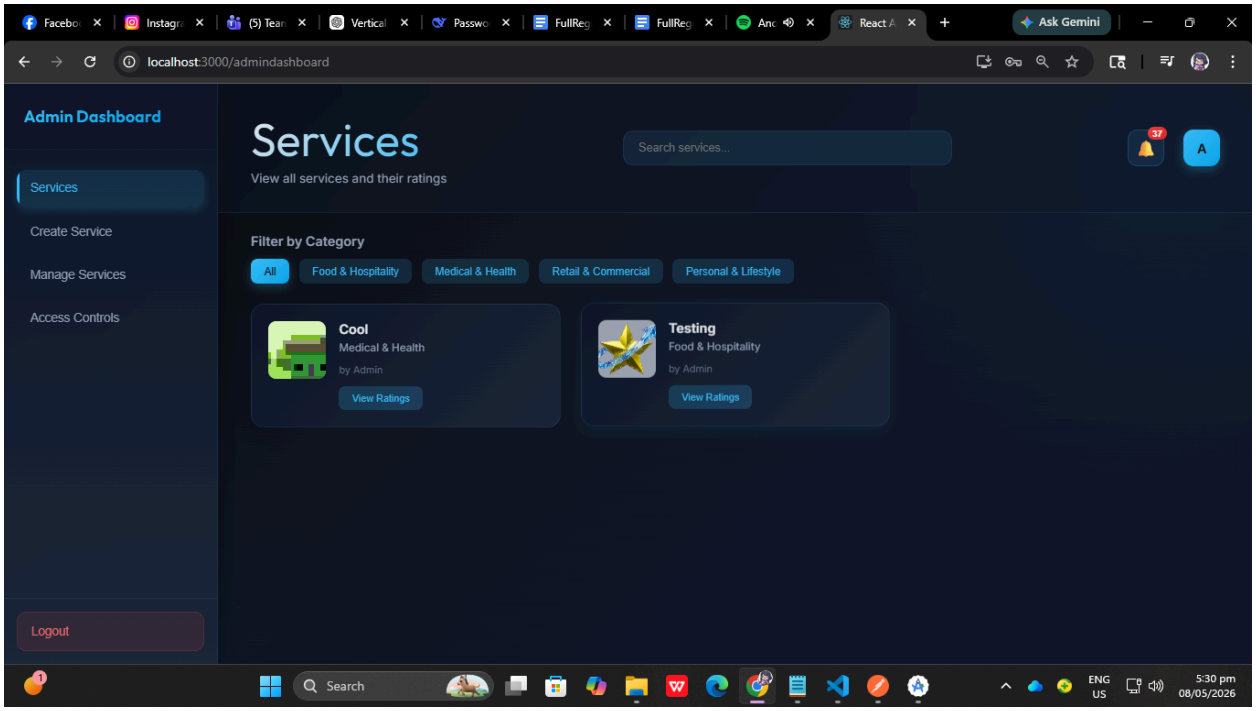
Automated Validation	Check service object returned
Expected Result	Correct service data returned
Status	Passed



ATC-SERV-003 — Create Service

Field	Details
Test ID	ATC-SERV-003
Endpoint	POST `/api/services/create`
Tool	Postman
Objective	Verify admin service creation
Automated Validation	Verify success message
Expected Result	Service created successfully
Status	Passed

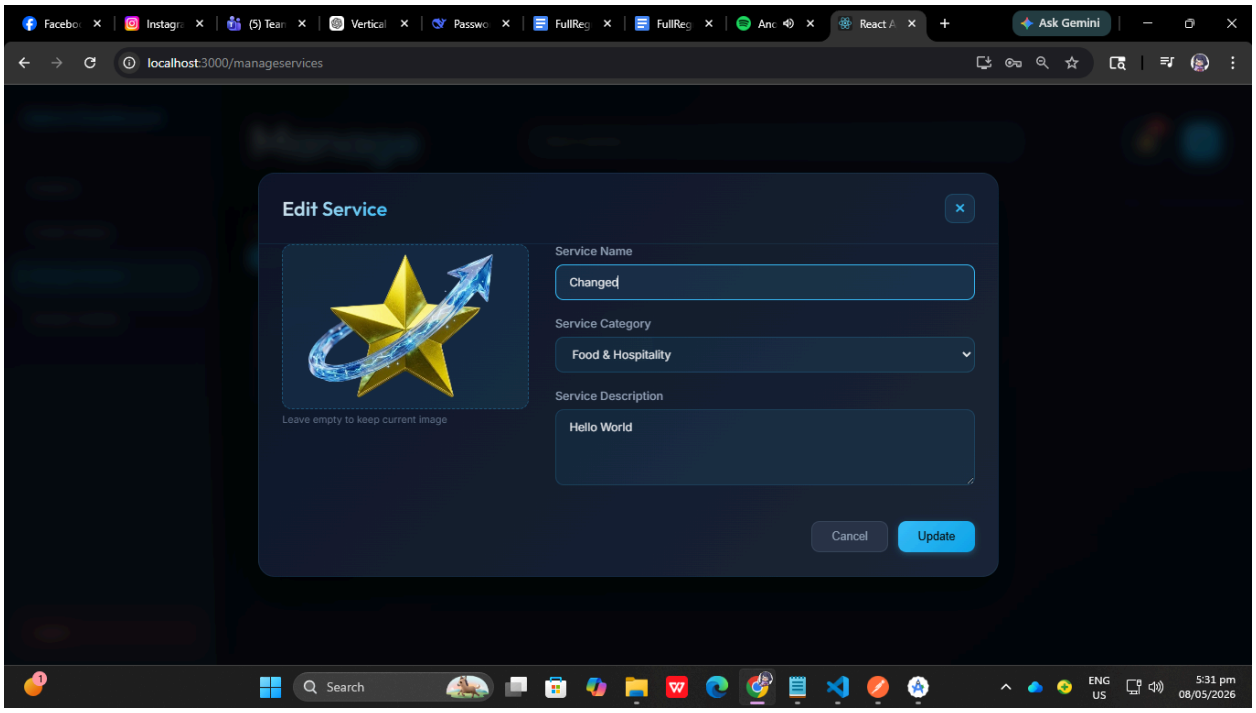


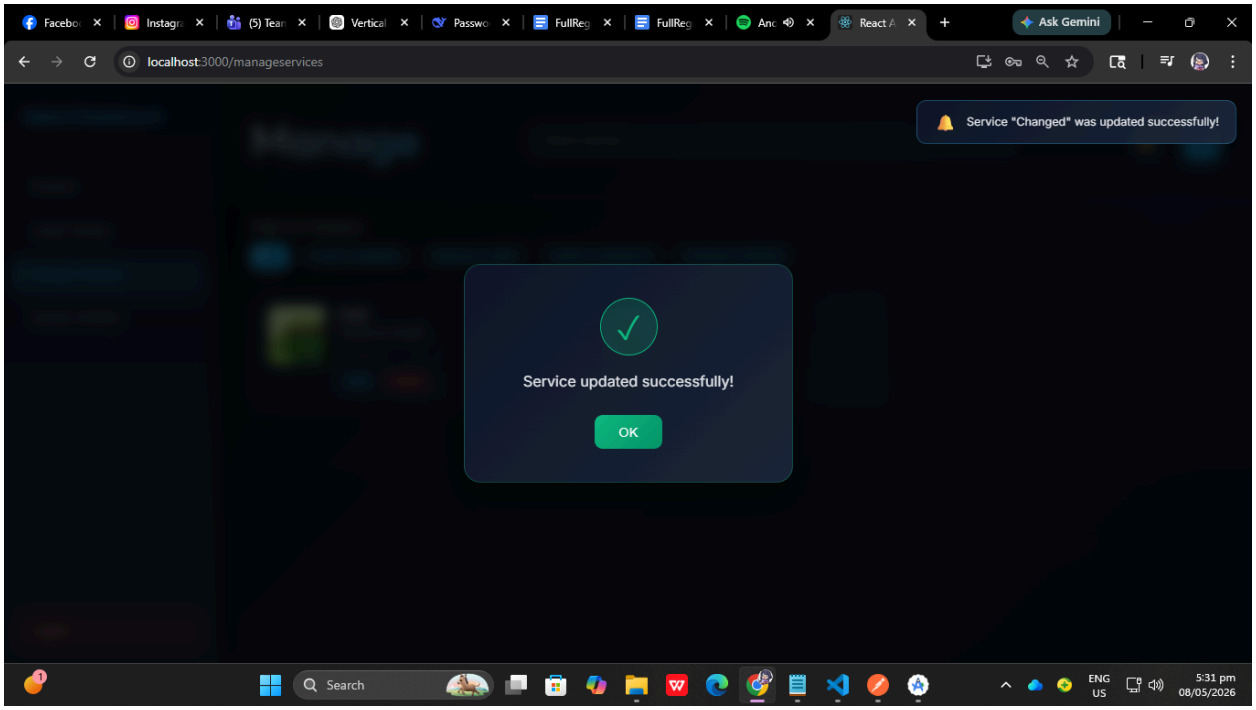


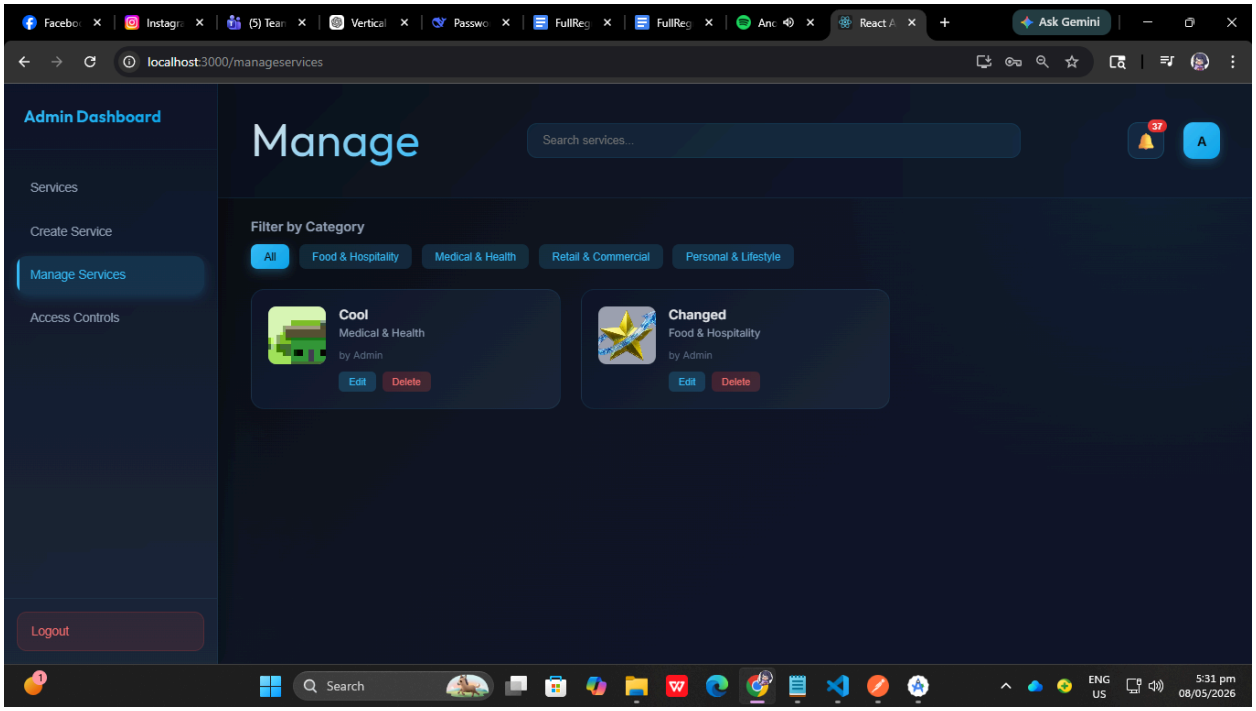
ATC-SERV-004 — Update Service

Field	Details
Test ID	ATC-SERV-004

Endpoint	PUT `/api/services/update/{serviceId}`
Tool	Postman
Objective	Verify service editing
Automated Validation	Check update response
Expected Result	Service updated successfully
Status	Passed



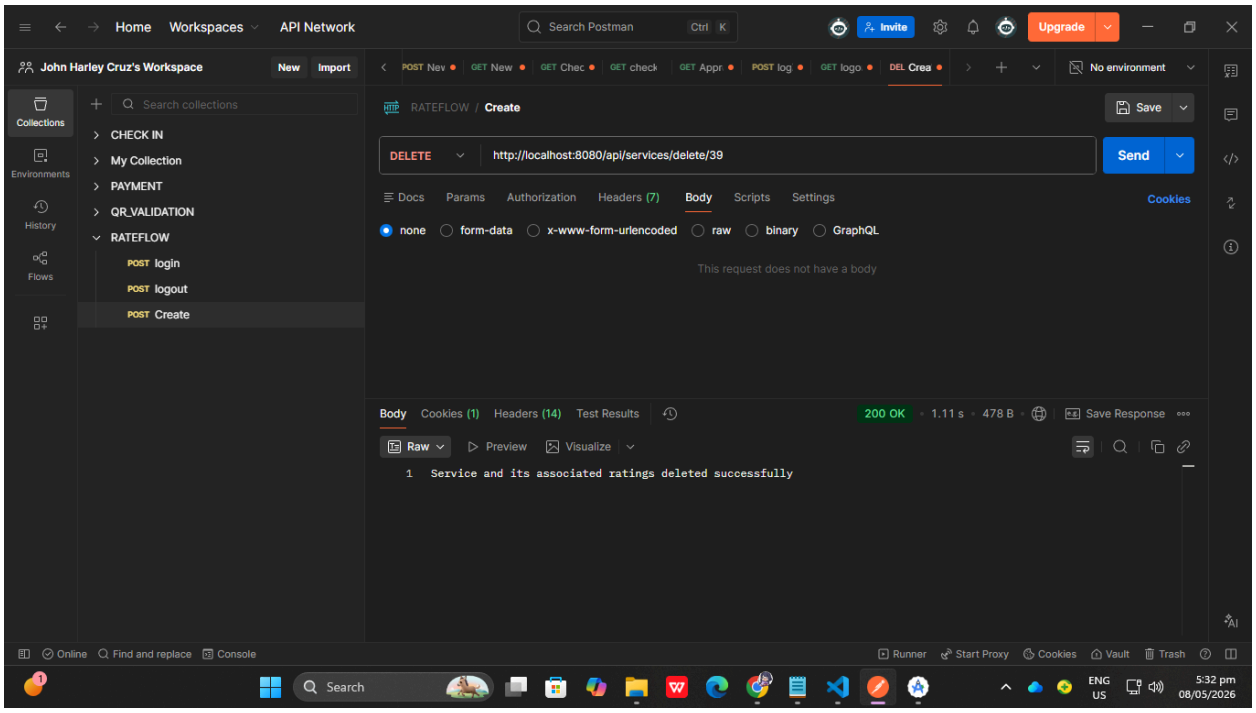




ATC-SERV-005 — Delete Service

Field	Details
Test ID	ATC-SERV-005

Endpoint	DELETE `/api/services/delete/{serviceId}`
Tool	Postman
Objective	Verify service deletion
Automated Validation	Verify deletion message
Expected Result	Service removed successfully
Status	Passed



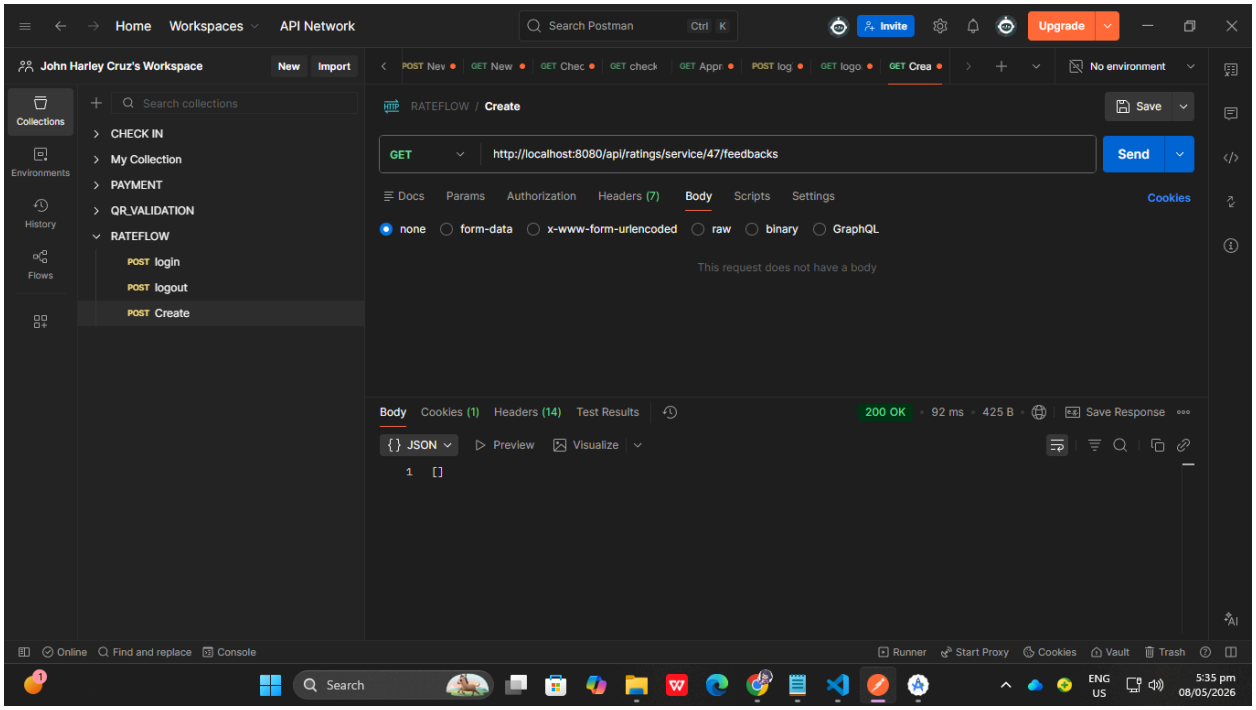
ATC-RATE-001 — Submit Rating

Field	Details
Test ID	ATC-RATE-001
Endpoint	POST `/api/ratings/submit`
Tool	Postman
Objective	Verify rating submission

Automated Validation	Check success response
Expected Result	Rating saved successfully
Status	Passed

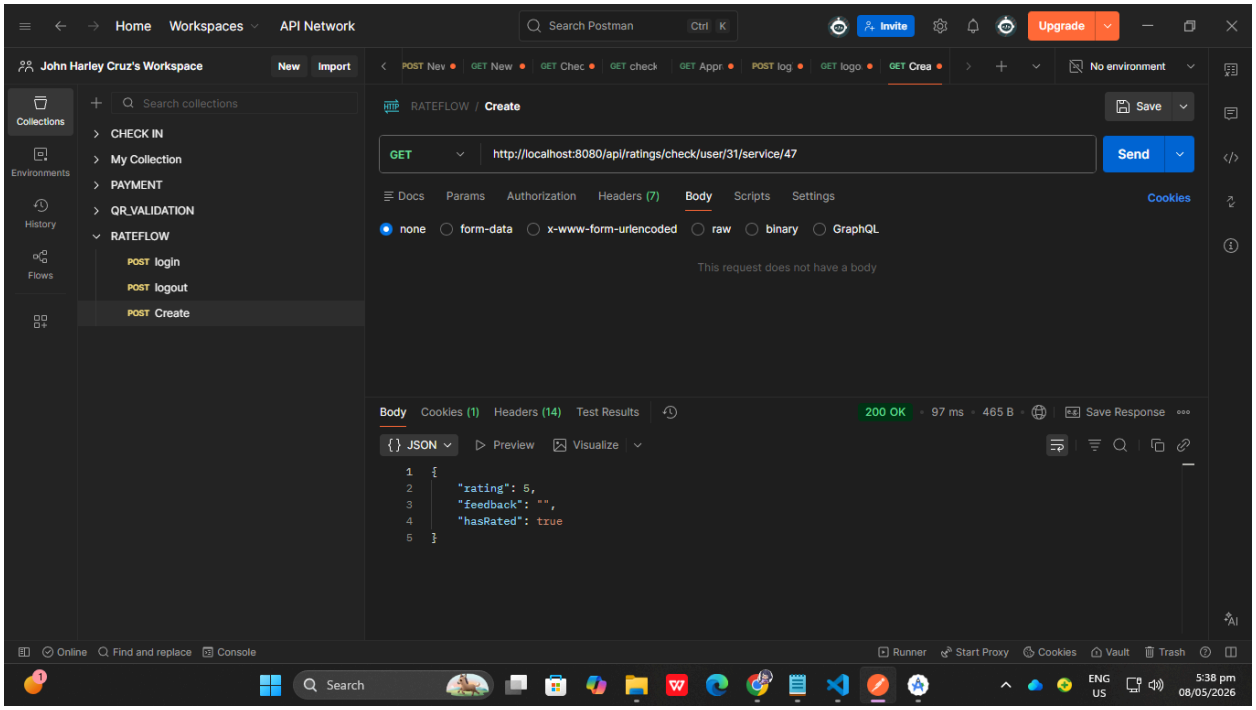
ATC-RATE-002 — Get Service Feedbacks

Field	Details
Test ID	ATC-RATE-002
Endpoint	GET `/api/ratings/service/{serviceId}/feedbacks`
Tool	Postman
Objective	Verify retrieval of feedback list
Automated Validation	Check array response
Expected Result	Feedback list displayed
Status	Passed



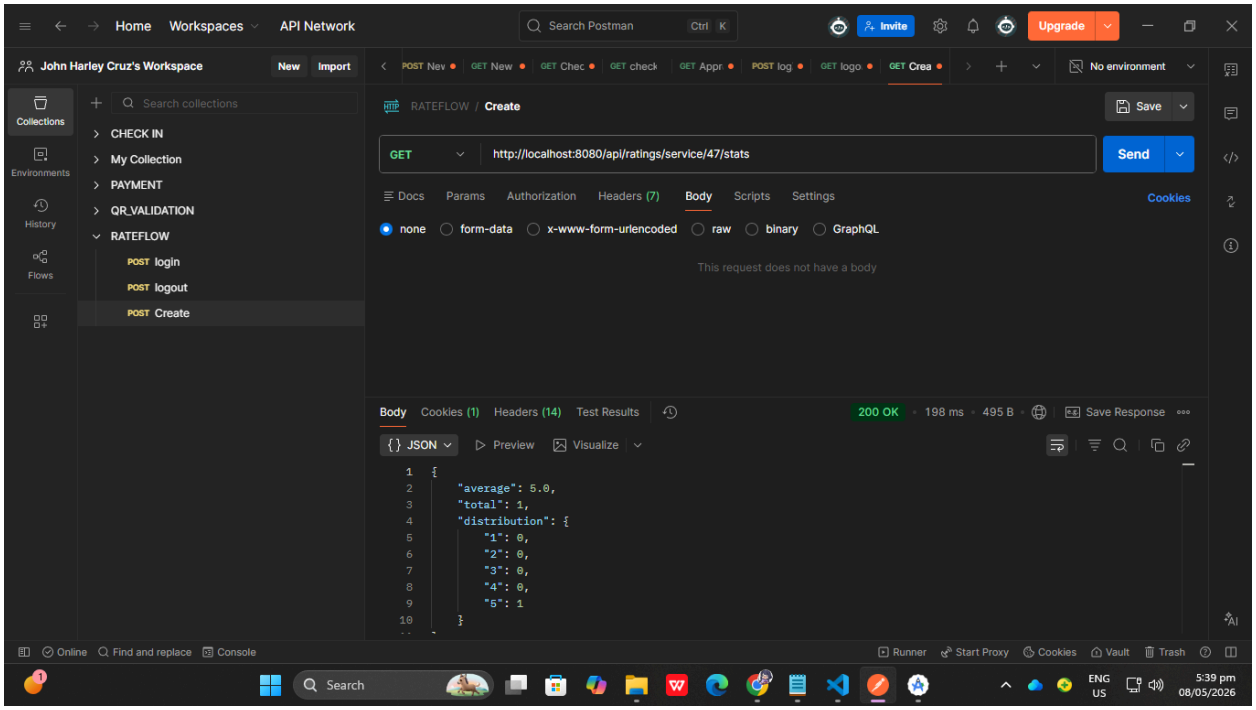
ATC-RATE-003 — Check Existing User Rating

Field	Details
Test ID	ATC-RATE-003
Endpoint	GET `/api/ratings/check/user/{userId}/service/{serviceId}`
Tool	Postman
Objective	Verify duplicate rating detection
Automated Validation	Verify hasRated field
Expected Result	Existing rating detected correctly
Status	Passed



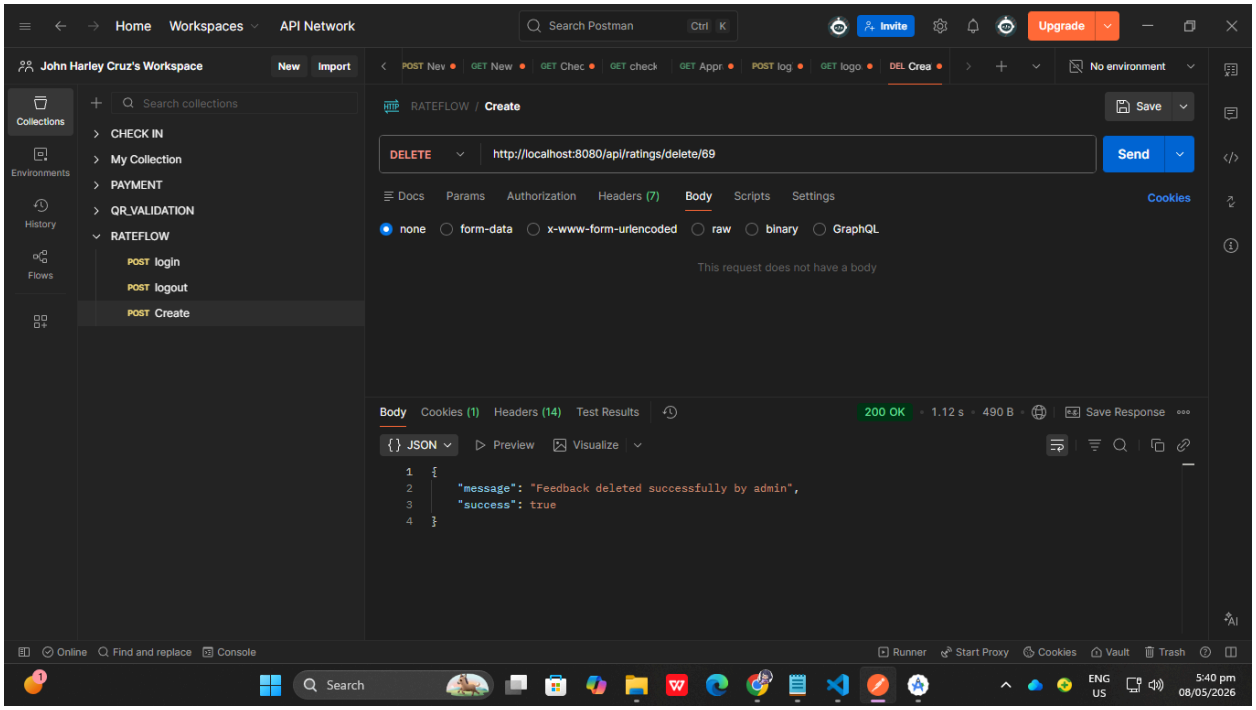
ATC-RATE-004 — Get Rating Statistics

Field	Details
Test ID	ATC-RATE-004
Endpoint	GET `/api/ratings/service/{serviceId}/stats`
Tool	Postman
Objective	Verify rating statistics retrieval
Automated Validation	Verify average and total values
Expected Result	Correct statistics returned
Status	Passed



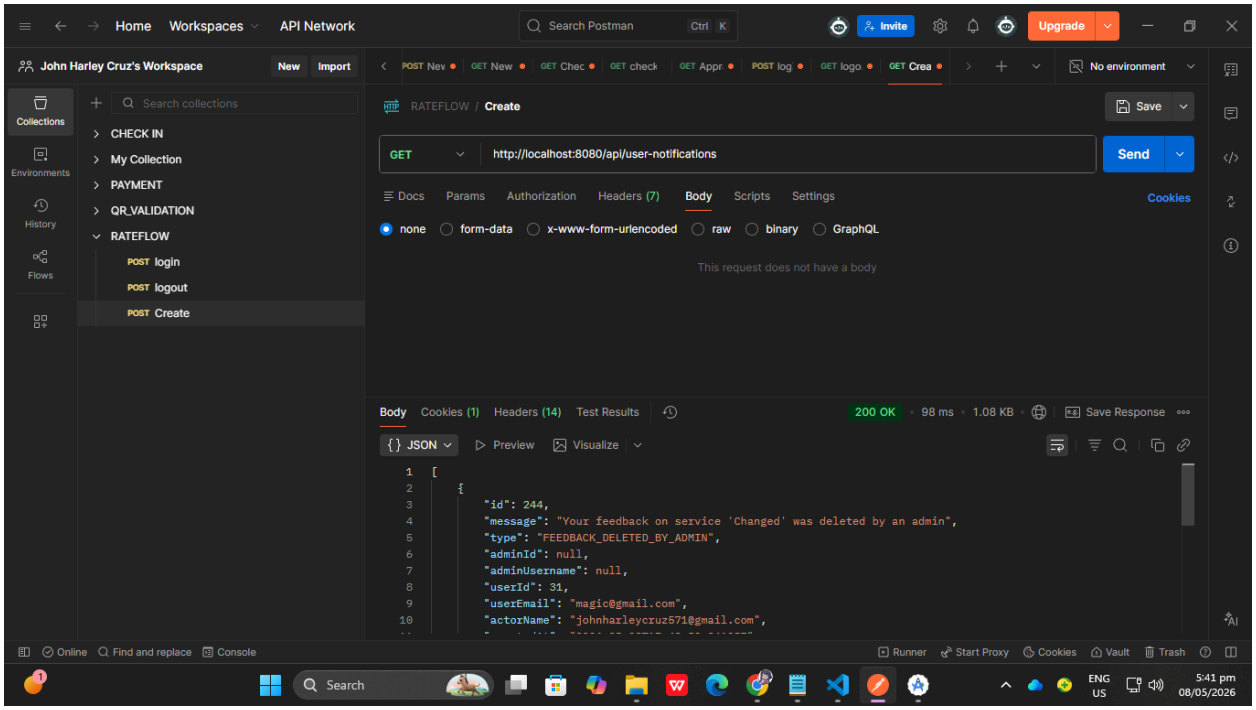
ATC-RATE-005 — Delete Rating

Field	Details
Test ID	ATC-RATE-005
Endpoint	DELETE `/api/ratings/delete/{ratingId}`
Tool	Postman
Objective	Verify rating deletion
Automated Validation	Check deletion success
Expected Result	Rating deleted successfully
Status	Passed



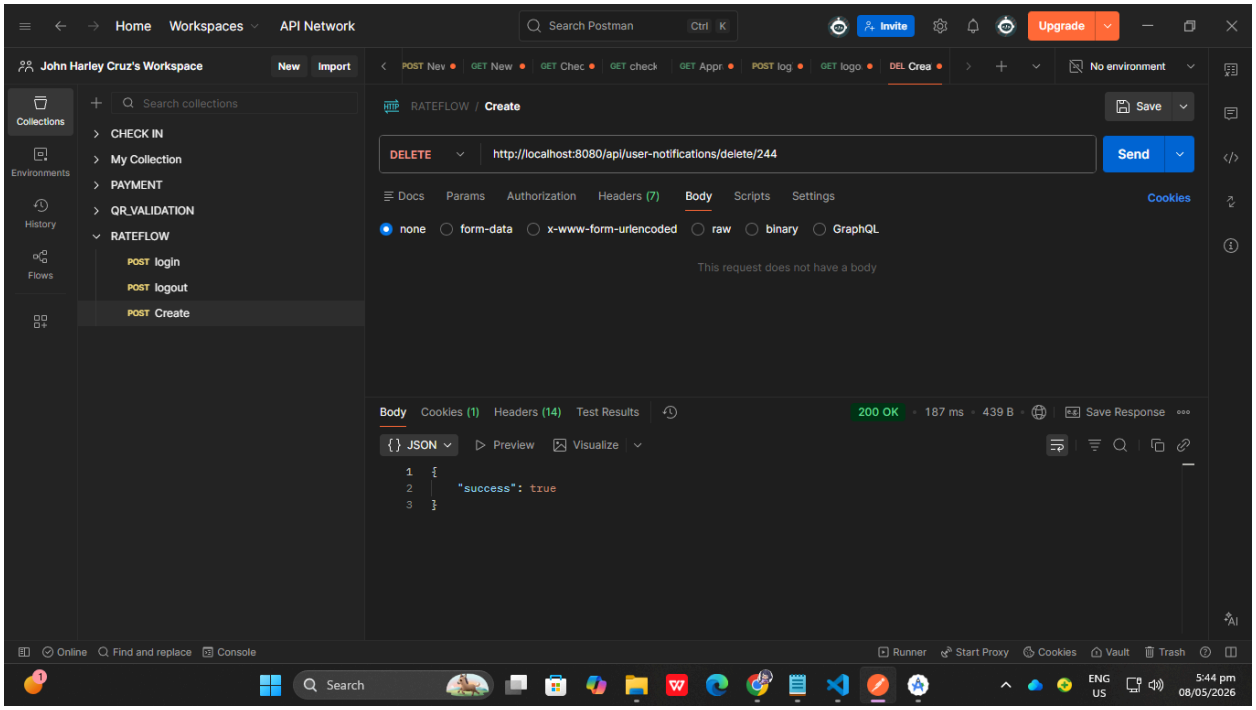
ATC-NOTIF-001 — Get User Notifications

Field	Details
Test ID	ATC-NOTIF-001
Endpoint	GET `/api/user-notifications`
Tool	Postman
Objective	Verify notifications retrieval
Automated Validation	Check notifications array
Expected Result	Notifications displayed
Status	Passed



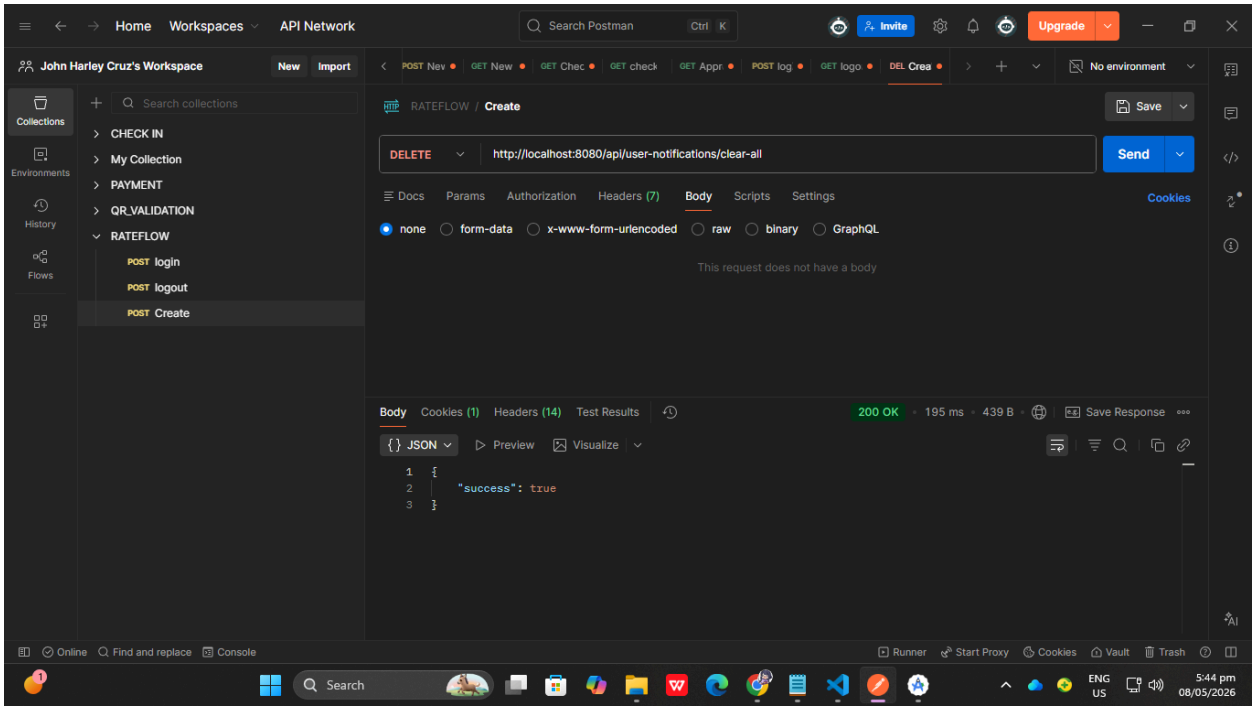
ATC-NOTIF-002 — Delete User Notification

Field	Details
Test ID	ATC-NOTIF-002
Endpoint	DELETE `/api/user-notifications/delete/{notificationId}`
Tool	Postman
Objective	Verify notification deletion
Automated Validation	Check success response
Expected Result	Notification deleted
Status	Passed



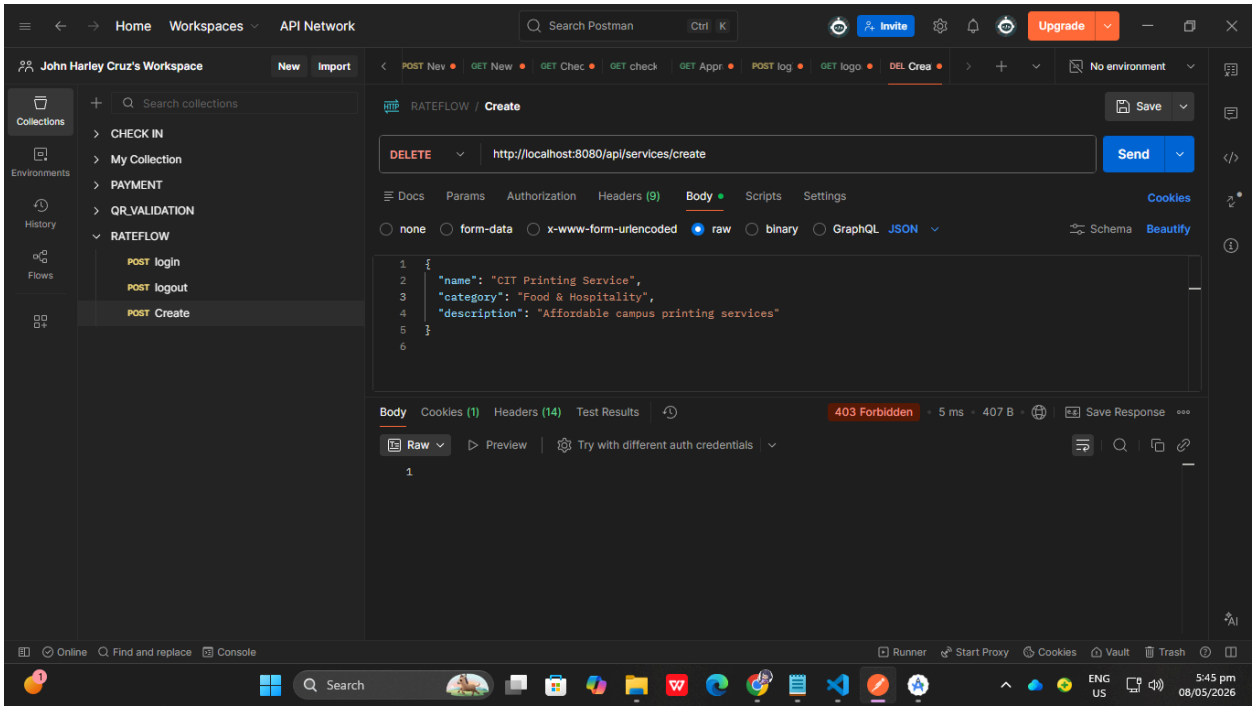
ATC-NOTIF-003 — Clear All Notifications

Field	Details
Test ID	ATC-NOTIF-003
Endpoint	DELETE `/api/user-notifications/clear-all`
Tool	Postman
Objective	Verify clearing notifications
Automated Validation	Check success response
Expected Result	Notifications removed successfully
Status	Passed



ATC-ACCESS-001 — Unauthorized Admin Endpoint Access

Field	Details
Test ID	ATC-ACCESS-001
Endpoint	POST `/api/services/create`
Tool	Postman
Objective	Verify non-admin cannot create service
Automated Validation	Check forbidden response
Expected Result	HTTP 403 Forbidden
Status	Passed



ATC-ACCESS-002 — Authenticated Admin Service Creation

Field	Details
Test ID	ATC-ACCESS-002
Endpoint	POST `/api/services/create`
Tool	Postman
Objective	Verify authenticated admin access
Automated Validation	Check success response
Expected Result	Service created successfully
Status	Passed

Regression Test Results

The following table summarizes the results of the full regression testing conducted after applying the Vertical Slice Architecture refactoring to the RateFlow system. All major functional modules were tested to ensure that existing functionalities continued to operate correctly without introducing regressions or system failures.

Module	Total Tests Executed	Passed	Failed	Status
Authentication Module	5	5	0	Passed
User Profile Module	2	2	0	Passed
Rating System Module	6	6	0	Passed
Admin Service Management Module	4	4	0	Passed
Access Control Module	2	2	0	Passed
Notification Module	3	3	0	Passed
Search & Filter Module	2	2	0	Passed
Automated API Authentication Tests	5	5	0	Passed
Automated API Service Tests	5	5	0	Passed
Automated API Rating Tests	5	5	0	Passed
Automated API Notification Tests	3	3	0	Passed

Automated API Access Control Tests	1	1	0	Passed

Category	Count
Total Tests Executed	43
Total Passed	43
Total Failed	0
Critical Issues Found	0
Regression Failures	0

Conclusion

Full regression testing was successfully completed after the Vertical Slice Architecture refactoring process. All major system functionalities across the backend, web frontend, and mobile application continued to function correctly. No critical regressions, functional failures, or system-breaking issues were identified during testing.

The testing results confirm that the refactoring process preserved the original behavior of the system while improving the maintainability, modularity, scalability, and organization of the project structure.

Issues Found

- No password view eye toggle in login and registration web pages

Fixes Applied

- Applied eye toggle password feature.

TEST PLAN CREATION

Functional Requirements Coverage

Functional Requirement	Module	Test Type
User Registration	Authentication	Functional
User Login	Authentication	Functional
Logout	Authentication	Functional
Google Login	Authentication	Functional
Create Service	Admin Module	Functional
Edit Service	Admin Module	Functional
Delete Service	Admin Module	Functional
Submit Rating	Rating System	Functional
Delete Own Rating	Rating System	Functional
View Feedback History	Feedback System	Functional
Search Services	Service Module	Functional

Notification Retrieval	Notification System	Functional
Grant Admin Access	Access Control	Functional
Session Validation	Security	Security Testing

Test Cases

AUTHENTICATION MODULE TEST CASES

Test Case ID	Test Case Name
TC-AUTH-001	User Registration with Valid Credentials
TC-AUTH-002	User Registration with Existing Email
TC-AUTH-003	User Registration with Invalid Email Format
TC-AUTH-004	User Registration with Weak Password
TC-AUTH-005	User Registration with Password Mismatch
TC-AUTH-006	User Login with Valid Credentials
TC-AUTH-007	User Login with Invalid Password
TC-AUTH-008	User Login with Unregistered Email
TC-AUTH-009	User Logout Successfully
TC-AUTH-010	Session Invalid After Logout
TC-AUTH-011	Google Sign-In with Registered Google Account
TC-AUTH-012	Google Sign-In with Unregistered Google Account
TC-AUTH-013	Forgot Password Request
TC-AUTH-014	Password Reset with Valid Token

TC-AUTH-015	Password Reset with Expired Token
TC-AUTH-016	Access Protected Page Without Login
TC-AUTH-017	Session Persistence During Active Usage
TC-AUTH-018	Session Expiration After Inactivity

USER PROFILE MODULE TEST CASES

Test Case ID	Test Case Name
TC-PROFILE-001	View User Profile Information
TC-PROFILE-002	Edit User Profile Successfully
TC-PROFILE-003	Prevent Empty Username Update
TC-PROFILE-004	Prevent Invalid Email Update
TC-PROFILE-005	Save Updated Profile Information
TC-PROFILE-006	Display User Role Correctly

SERVICE RATING & FEEDBACK MODULE TEST CASES

Test Case ID	Test Case Name
TC-RATE-001	Submit Rating with Valid Data
TC-RATE-002	Submit Rating Without Feedback Text
TC-RATE-003	Submit Rating Without Selecting Stars
TC-RATE-004	Prevent Duplicate Rating Submission
TC-RATE-005	View Service Feedback List
TC-RATE-006	View Average Rating Statistics
TC-RATE-007	Edit Existing Feedback Successfully

TC-RATE-008	Delete Own Rating Successfully
TC-RATE-009	Prevent User from Deleting Other User Ratings
TC-RATE-010	Check User Existing Rating Status
TC-RATE-011	Rating Statistics Updates After Submission
TC-RATE-012	Rating Statistics Updates After Deletion
TC-RATE-013	Display Feedback History Correctly
TC-RATE-014	Search Specific Service
TC-RATE-015	Filter Services by Category
TC-RATE-016	View Rated Services in My Ratings Page

ADMIN AUTHENTICATION MODULE TEST CASES

Test Case ID	Test Case Name
TC-ADMINAUTH-001	Admin Login with Valid Credentials
TC-ADMINAUTH-002	Admin Login with Invalid Credentials
TC-ADMINAUTH-003	Prevent Non-Admin Access to Admin Pages
TC-ADMINAUTH-004	Admin Logout Successfully
TC-ADMINAUTH-005	Access Admin Dashboard After Successful Login

ADMIN SERVICE MANAGEMENT TEST CASES

Test Case ID	Test Case Name
TC-SERV-001	Create Service Successfully
TC-SERV-002	Prevent Service Creation with Empty Name
TC-SERV-003	Prevent Duplicate Service Name per Admin

TC-SERV-004	Upload Service Image Successfully
TC-SERV-005	View Created Services
TC-SERV-006	Edit Service Successfully
TC-SERV-007	Update Service Category Successfully
TC-SERV-008	Update Service Description Successfully
TC-SERV-009	Delete Service Successfully
TC-SERV-010	Delete Service Removes Associated Ratings
TC-SERV-011	Cancel Service Deletion
TC-SERV-012	Search Services from Admin Dashboard
TC-SERV-013	Filter Services by Category
TC-SERV-014	View Service Rating Statistics
TC-SERV-015	View Ratings of Selected Service

ACCESS CONTROL MODULE TEST CASES

Test Case ID	Test Case Name
TC-ACCESS-001	View Registered Users List
TC-ACCESS-002	Grant Admin Access Successfully
TC-ACCESS-003	Prevent Unauthorized User from Granting Admin Access
TC-ACCESS-004	Display Updated User Role After Grant
TC-ACCESS-005	Confirm Admin Access Prompt Appears

NOTIFICATION MODULE TEST CASES

Test Case ID	Test Case Name
TC-NOTIF-001	View User Notifications

TC-NOTIF-002	Delete Single Notification
TC-NOTIF-003	Clear All Notifications
TC-NOTIF-004	Display Notification After Rating Submission
TC-NOTIF-005	Display Notification After Service Creation
TC-NOTIF-006	View Admin Notifications
TC-NOTIF-007	Prevent Unauthorized Notification Access

MOBILE APPLICATION TEST CASES

Test Case ID	Test Case Name
TC-MOB-001	Mobile Login Works Properly
TC-MOB-002	Mobile Registration Works Properly
TC-MOB-003	Mobile Dashboard Loads Services
TC-MOB-004	Mobile Rating Submission Works
TC-MOB-005	Mobile Notifications Display Correctly
TC-MOB-006	Mobile Profile Page Loads Correctly
TC-MOB-007	Mobile Logout Works Properly
TC-MOB-008	Mobile Navigation Drawer Works
TC-MOB-009	Mobile Layout Responsive on Different Screen Sizes
TC-MOB-010	Mobile Touch Controls Function Correctly

Test Scripts / Test Steps

AUTHENTICATION MODULE

TS-AUTH-001 — User Registration

Field	Details
Module	Authentication
Objective	Verify that a new user can register successfully
Preconditions	User is not yet registered
Test Steps	1. Open the Registration Page 2. Enter valid username 3. Enter valid email 4. Enter strong password 5. Confirm password correctly 6. Click “Register” button
Expected Result	User account is created successfully and redirected to login page

TS-AUTH-002 — User Login

Field	Details
Module	Authentication
Objective	Verify user login functionality
Preconditions	Registered user account exists
Test Steps	1. Open Login Page 2. Enter registered email 3. Enter correct password 4. Click “Login”
Expected Result	User successfully logs in and dashboard page loads

TS-AUTH-003 — Invalid Login Credentials

Field	Details
Module	Authentication
Objective	Verify system rejects invalid login

Preconditions	Registered user account exists
Test Steps	1. Open Login Page 2. Enter invalid password 3. Click “Login”
Expected Result	Error message “Invalid credentials” is displayed

TS-AUTH-004 — User Logout

Field	Details
Module	Authentication
Objective	Verify logout functionality
Preconditions	User is logged in
Test Steps	1. Click Logout button 2. Confirm logout
Expected Result	User session is terminated and redirected to login page

TS-AUTH-005 — Google Sign-In

Field	Details
Module	Authentication
Objective	Verify Google authentication login
Preconditions	Google account already registered in system
Test Steps	1. Open Login Page 2. Click “Sign in with Google” 3. Select registered Google account
Expected Result	User successfully authenticated and redirected to dashboard

USER PROFILE MODULE

TS-PROFILE-001 — View User Profile

Field	Details
Module	User Profile
Objective	Verify profile information is displayed
Preconditions	User is logged in
Test Steps	1. Open Profile Page
Expected Result	Username, email, and role are displayed correctly

TS-PROFILE-002 — Edit User Profile

Field	Details
Module	User Profile
Objective	Verify user can edit profile
Preconditions	User is logged in
Test Steps	1. Open Profile Page 2. Click “Edit Profile” 3. Update profile details 4. Save changes
Expected Result	Updated profile information is saved successfully

SERVICE RATING MODULE

TS-RATE-001 — Submit Rating

Field	Details
Module	Rating System

Objective	Verify user can submit rating
Preconditions	User logged in and service exists
Test Steps	1. Open Service Page 2. Select star rating 3. Enter feedback text 4. Click “Submit Rating”
Expected Result	Rating is stored successfully and confirmation message appears

TS-RATE-002 — Submit Rating Without Feedback

Field	Details
Module	Rating System
Objective	Verify optional feedback submission
Preconditions	User logged in
Test Steps	1. Open Service Page 2. Select star rating only 3. Click “Submit Rating”
Expected Result	Rating successfully submitted without written feedback

TS-RATE-003 — Prevent Duplicate Rating

Field	Details
Module	Rating System
Objective	Verify user cannot rate same service twice
Preconditions	User already rated service
Test Steps	1. Open already-rated service 2. Attempt second rating submission
Expected Result	System prevents duplicate rating submission

TS-RATE-004 — Delete Own Rating

Field	Details
Module	Rating System
Objective	Verify user can delete own rating
Preconditions	User has existing rating
Test Steps	1. Open My Ratings Page 2. Select rating 3. Click Delete button 4. Confirm deletion
Expected Result	Rating deleted successfully

TS-RATE-005 — View Feedback History

Field	Details
Module	Rating System
Objective	Verify user feedback history display
Preconditions	User has submitted ratings
Test Steps	1. Open My Ratings Page
Expected Result	All submitted ratings displayed correctly

TS-RATE-006 — View Rating Statistics

Field	Details
Module	Rating System
Objective	Verify average rating statistics display
Preconditions	Service has ratings
Test Steps	1. Open Service Detail Page

Expected Result	Average rating and rating distribution displayed correctly

SERVICE MANAGEMENT MODULE

TS-SERV-001 — Create Service

Field	Details
Module	Admin Service Management
Objective	Verify admin can create service
Preconditions	Admin logged in
Test Steps	1. Open Admin Dashboard 2. Click “Create Service” 3. Fill required fields 4. Upload image 5. Click “Create”
Expected Result	Service created successfully and displayed in dashboard

TS-SERV-002 — Edit Service

Field	Details
Module	Admin Service Management
Objective	Verify admin can edit service
Preconditions	Existing service available
Test Steps	1. Open Manage Services Page 2. Select service 3. Click “Edit” 4. Update fields 5. Save changes
Expected Result	Updated service information displayed successfully

TS-SERV-003 — Delete Service

Field	Details
Module	Admin Service Management
Objective	Verify admin can delete service
Preconditions	Existing service available
Test Steps	1. Open Manage Services Page 2. Select service 3. Click Delete button 4. Confirm deletion
Expected Result	Service removed successfully from system

TS-SERV-004 — View Created Services

Field	Details
Module	Admin Service Management
Objective	Verify admin can view created services
Preconditions	Services exist
Test Steps	1. Open Admin Dashboard
Expected Result	All admin-created services displayed

ACCESS CONTROL MODULE

TS-ACCESS-001 — Grant Admin Access

Field	Details
Module	Access Control
Objective	Verify admin can grant admin privileges
Preconditions	Admin logged in
Test Steps	1. Open Access Control Page 2. Select user 3. Click “Grant Admin Access” 4. Confirm action

Expected Result	User role updated to ADMIN successfully

TS-ACCESS-002 — Prevent Unauthorized Access Control

Field	Details
Module	Access Control
Objective	Verify normal user cannot access admin control
Preconditions	User logged in as regular user
Test Steps	1. Attempt to access Access Control page URL
Expected Result	Access denied or redirected

NOTIFICATION MODULE

TS-NOTIF-001 — View Notifications

Field	Details
Module	Notifications
Objective	Verify notifications are displayed
Preconditions	User has notifications
Test Steps	1. Open Notifications Page
Expected Result	Notifications displayed correctly

TS-NOTIF-002 — Delete Notification

Field	Details
Module	Notifications

Objective	Verify single notification deletion
Preconditions	Existing notification available
Test Steps	1. Open Notifications Page 2. Click Delete icon
Expected Result	Notification removed successfully

TS-NOTIF-003 — Clear All Notifications

Field	Details
Module	Notifications
Objective	Verify clearing all notifications
Preconditions	Multiple notifications available
Test Steps	1. Open Notifications Page 2. Click “Clear All”
Expected Result	All notifications removed successfully

SEARCH & FILTER MODULE

TS-SEARCH-001 — Search Service

Field	Details
Module	Search
Objective	Verify search functionality
Preconditions	Services exist
Test Steps	1. Enter service keyword in search bar
Expected Result	Matching services displayed

TS-SEARCH-002 — Filter Services by Category

Field	Details

Module	Filter
Objective	Verify category filtering
Preconditions	Multiple categorized services exist
Test Steps	1. Click category filter button
Expected Result	Only matching category services displayed

REGRESSION AUTOMATED TEST CASES

ATC-REG-001 — Authentication Regression Test

Field	Details
Automated Test ID	ATC-REG-001
Module	Regression Testing
Tool	Postman
Objective	Verify authentication still works after refactor
Automated Action	Execute login/register/logout tests
Expected Result	All authentication tests pass
Status	Passed

ATC-REG-002 — Service Management Regression Test

Field	Details
Automated Test ID	ATC-REG-002
Module	Regression Testing
Tool	JUnit
Objective	Verify CRUD operations after refactor
Automated Action	Create/update/delete service
Expected Result	CRUD features operate correctly

Status	Passed

ATC-REG-003 — Rating System Regression Test

Field	Details
Automated Test ID	ATC-REG-003
Module	Regression Testing
Tool	JUnit
Objective	Verify ratings still function after refactor
Automated Action	Submit/delete ratings
Expected Result	Rating system works properly
Status	Passed

Automated Test Cases

ATC-AUTH-001 — User Registration API

Field	Details
Test ID	ATC-AUTH-001
Endpoint	POST `/api/auth/register`
Tool	Postman
Objective	Verify successful user registration
Request Body	username, email, password
Automated Validation	Check if status code is 201
Expected Result	User account created successfully
Status	Passed

ATC-AUTH-002 — User Login API

Field	Details
Test ID	ATC-AUTH-002
Endpoint	POST `/api/auth/admin/login`
Tool	Postman
Objective	Verify successful user login
Automated Validation	Verify status code and returned role
Expected Result	User authenticated successfully
Status	Passed

ATC-AUTH-003 — Invalid Login Credentials

Field	Details
Test ID	ATC-AUTH-003
Endpoint	POST `/api/auth/admin/login`
Tool	Postman
Objective	Verify invalid login rejected
Automated Validation	Check unauthorized response
Expected Result	HTTP 401 Unauthorized
Status	Passed

ATC-AUTH-004 — User Logout

Field	Details
Test ID	ATC-AUTH-004
Endpoint	POST `/api/auth/logout`
Tool	Postman

Objective	Verify session invalidation
Automated Validation	Check successful logout
Expected Result	Session terminated
Status	Passed

ATC-AUTH-005 — Google Authentication Login

Field	Details
Test ID	ATC-AUTH-005
Endpoint	POST `/api/auth/google`
Tool	Postman
Objective	Verify Google login
Automated Validation	Check success response
Expected Result	User authenticated via Google
Status	Passed

SERVICE MANAGEMENT AUTOMATED TEST CASES

ATC-SERV-001 — Get All Services

Field	Details
Test ID	ATC-SERV-001
Endpoint	GET `/api/services`
Tool	Postman
Objective	Verify retrieval of services
Automated Validation	Check returned array
Expected Result	Services list returned successfully

Status	Passed

ATC-SERV-002 — Get Service By ID

Field	Details
Test ID	ATC-SERV-002
Endpoint	GET `/api/services/{serviceId}`
Tool	Postman
Objective	Verify service details retrieval
Automated Validation	Check service object returned
Expected Result	Correct service data returned
Status	Passed

ATC-SERV-003 — Create Service

Field	Details
Test ID	ATC-SERV-003
Endpoint	POST `/api/services/create`
Tool	Postman
Objective	Verify admin service creation
Automated Validation	Verify success message
Expected Result	Service created successfully
Status	Passed

ATC-SERV-004 — Update Service

Field	Details
Test ID	ATC-SERV-004
Endpoint	PUT `/api/services/update/{serviceId}`

Tool	Postman
Objective	Verify service editing
Automated Validation	Check update response
Expected Result	Service updated successfully
Status	Passed

ATC-SERV-005 — Delete Service

Field	Details
Test ID	ATC-SERV-005
Endpoint	DELETE `/api/services/delete/{serviceId}`
Tool	Postman
Objective	Verify service deletion
Automated Validation	Verify deletion message
Expected Result	Service removed successfully
Status	Passed

RATING SYSTEM AUTOMATED TEST CASES

ATC-RATE-001 — Submit Rating

Field	Details
Test ID	ATC-RATE-001
Endpoint	POST `/api/ratings/submit`
Tool	Postman
Objective	Verify rating submission
Automated Validation	Check success response

Expected Result	Rating saved successfully
Status	Passed

ATC-RATE-002 — Get Service Feedbacks

Field	Details
Test ID	ATC-RATE-002
Endpoint	GET `/api/ratings/service/{serviceId}/feedbacks`
Tool	Postman
Objective	Verify retrieval of feedback list
Automated Validation	Check array response
Expected Result	Feedback list displayed
Status	Passed

ATC-RATE-003 — Check Existing User Rating

Field	Details
Test ID	ATC-RATE-003
Endpoint	GET `/api/ratings/check/user/{userId}/service/{serviceId}`
Tool	Postman
Objective	Verify duplicate rating detection
Automated Validation	Verify hasRated field
Expected Result	Existing rating detected correctly
Status	Passed

ATC-RATE-004 — Get Rating Statistics

Field	Details
Test ID	ATC-RATE-004
Endpoint	GET `/api/ratings/service/{serviceId}/stats`
Tool	Postman
Objective	Verify rating statistics retrieval
Automated Validation	Verify average and total values
Expected Result	Correct statistics returned
Status	Passed

ATC-RATE-005 — Delete Rating

Field	Details
Test ID	ATC-RATE-005
Endpoint	DELETE `/api/ratings/delete/{ratingId}`
Tool	Postman
Objective	Verify rating deletion
Automated Validation	Check deletion success
Expected Result	Rating deleted successfully
Status	Passed

NOTIFICATION SYSTEM AUTOMATED TEST CASES

ATC-NOTIF-001 — Get User Notifications

Field	Details
Test ID	ATC-NOTIF-001

Endpoint	GET `/api/user-notifications`
Tool	Postman
Objective	Verify notifications retrieval
Automated Validation	Check notifications array
Expected Result	Notifications displayed
Status	Passed

ATC-NOTIF-002 — Delete User Notification

Field	Details
Test ID	ATC-NOTIF-002
Endpoint	DELETE `/api/user-notifications/delete/{notificationId}`
Tool	Postman
Objective	Verify notification deletion
Automated Validation	Check success response
Expected Result	Notification deleted
Status	Passed

ATC-NOTIF-003 — Clear All Notifications

Field	Details
Test ID	ATC-NOTIF-003
Endpoint	DELETE `/api/user-notifications/clear-all`
Tool	Postman

Objective	Verify clearing notifications
Automated Validation	Check success response
Expected Result	Notifications removed successfully
Status	Passed

ACCESS CONTROL AUTOMATED TEST CASES

ATC-ACCESS-001 — Unauthorized Admin Endpoint Access

Field	Details
Test ID	ATC-ACCESS-001
Endpoint	POST `/api/services/create`
Tool	Postman
Objective	Verify non-admin cannot create service
Automated Validation	Check forbidden response
Expected Result	HTTP 403 Forbidden
Status	Passed

ATC-ACCESS-002 — Authenticated Admin Service Creation

Field	Details
Test ID	ATC-ACCESS-002
Endpoint	POST `/api/services/create`
Tool	Postman
Objective	Verify authenticated admin access
Automated Validation	Check success response
Expected Result	Service created successfully

Status	Passed

REGRESSION AUTOMATED TEST CASES

ATC-REG-001 — Authentication Regression Test

Field	Details
Test ID	ATC-REG-001
Module	Regression Testing
Tool	Postman Collection Runner
Objective	Verify authentication still works after refactor
Automated Validation	Execute all authentication endpoints
Expected Result	All tests pass
Status	Passed

ATC-REG-002 — Rating System Regression Test

Field	Details
Test ID	ATC-REG-002
Module	Regression Testing
Tool	Postman Collection Runner
Objective	Verify rating system still works after refactor
Automated Validation	Execute rating APIs
Expected Result	Rating features remain functional
Status	Passed

ATC-REG-003 — Service Management Regression Test

Field	Details

Test ID	ATC-REG-003
Module	Regression Testing
Tool	Postman Collection Runner
Objective	Verify CRUD operations still work
Automated Validation	Execute create/update/delete operations
Expected Result	CRUD functionality remains stable
Status	Passed

REGRESSION AUTOMATED TEST CASES

ATC-REG-001 — Authentication Regression Test

Field	Details
Automated Test ID	ATC-REG-001
Module	Regression Testing
Tool	JUnit + Postman
Objective	Verify authentication still works after refactor
Automated Action	Execute login/register/logout tests
Expected Result	All authentication tests pass
Status	Passed

ATC-REG-002 — Service Management Regression Test

Field	Details
Automated Test ID	ATC-REG-002
Module	Regression Testing
Tool	JUnit

Objective	Verify CRUD operations after refactor
Automated Action	Create/update/delete service
Expected Result	CRUD features operate correctly
Status	Passed

ATC-REG-003 — Rating System Regression Test

Field	Details
Automated Test ID	ATC-REG-003
Module	Regression Testing
Tool	JUnit
Objective	Verify ratings still function after refactor
Automated Action	Submit/delete ratings
Expected Result	Rating system works properly
Status	Passed